

分类号: TP242.2

单位代码: 10335

密 级: _____

学 号: 21960177

浙江大学

硕士专业学位论文



中文论文题目: 深度学习框架中的
GPU 高效利用问题研究

英文论文题目: GPU efficient utilization issues
in deep learning frameworks

申请人姓名: 欧岳枫

指导教师: 杜歆

合作导师: _____

专业学位类别: 工程硕士

专业学位领域: 电子与通信工程

所在学院: 信息与工程学院

论文提交日期 2022 年 1 月 6 日

深度学习框架中的

GPU 高效利用问题研究



论文作者签名: 欧岳枫

指导教师签名: 杜毅

论文评阅人 1: 匿名

评阅人 2: 匿名

评阅人 3: 匿名

评阅人 4:

评阅人 5:

答辩委员会主席: 夏永祥

委员 1: 沈会良

委员 2: 叶德信

委员 3: 皇甫江涛

委员 4: 杨勇勤

委员 5:

答辩日期: 2022 年 3 月 11 日

致谢

时光如箭，光阴如梭，两年半的硕士研究生生涯如白驹之过隙，转眼间便走到了终点。在这段求是之路上，我度过了人生中一段重要而难忘的时间。在此，特向过去给予我关心、帮助和指导的老师、同学、朋友和家人致以我最由衷的敬意和最衷心的感谢。

首先，感谢我的研究生导师杜歆老师。杜老师十分随和、有亲和力，重视学生的想法，为课题组营造了自由而高效的科研氛围。在治学方面，杜老师严谨而又博学，常常能指出我所做工作中考虑不周之处，给予我改进的思路与方向。在进行科研的过程中，杜老师会鼓励和支持我去做我所感兴趣的方向。杜老师不仅是一位良师，同时也是一位益友，在日常生活和待人接物方面也常给予我指导与教诲，我一直铭记在心。

其次，感谢钱徽老师、李石坚老师和陈岭老师对我的科研工作给予的指导和帮助；几位老师严谨的治学态度和独到的观点，也让我获益匪浅。

还要感谢科研之路上同行的伙伴们，蒋宇波，宋宇达，郑祥谱，樊瑞新，邓尧慷，江舒婷，吕泽嘉，经豪灿，李飞，苗兴宇，陶阳阳，张海杰，何竹青，王云峰，李智健。同窗之间，友谊长存。

最后，感谢我的父亲与母亲，我成长路上最无私的灌溉者，为我倾注了无数心血。你们或许不善表达，但却是最有力的支持者，使我能够不断向前。

国有成均，在浙之滨。如今的我即将离开生活了七年的求是园，踏入社会。回首往昔，既有日夜兼程之奋进，也有清风明月之闲暇。在这个即将步入人生下一个阶段的时刻，我才发现自己竟从求学之旅上汲取了如此多的营养和力量，而这些力量将会鼓舞我一生的时间。感谢母校和国家的栽培，也感恩这个伟大的时代，愿我们都能前程似锦，不负韶华！

2022年1月4日于求是园

摘要

GPU 作为普遍用于深度学习模型训练的硬件平台, 在应用时常面临着可用内存不足、核心利用率低等问题。本文以深度学习框架中的 GPU 高效利用问题作为研究对象, 研究了提升 GPU 核心利用率以及降低 GPU 内存消耗的方法, 并开发了 HelloDeep 深度学习框架作为实验平台进行验证。

深度学习框架是执行深度神经网络搭建、训练和推理的软件平台。为实现和验证本文提出的 GPU 高效利用方法, 本文提出了一个面向 GPU 高效利用的深度学习框架 HelloDeep。该框架以静态计算图为核心, 以异步并行执行引擎保证计算效率, 以静态内存池统一管理全局内存, 提供用户友好的算子封装和数据处理方法。

数据加载经常会成为神经网络训练 workflow 中的瓶颈, 一个较慢的数据加载速度常会使 GPU 处于等候数据就绪的闲置状态, 降低了 GPU 总体利用率。本文提出了一个高效数据 IO 模块, 该模块基于生产者-消费者模型, 采用多线程异步加载的方式加快数据处理速度。通过实验证明, 该方法可以将神经网络训练过程中 GPU 的平均利用率从同步加载模式下的 25% 提高到 90% 以上。

GPU 可用内存不足是导致深度学习任务失败的重要原因之一。本文提出了一个 GPU 内存优化方法, 提供 3 种可选的优化级别, 方便用户根据实际情况选用。该 GPU 内存优化方法采用了变量重计算和内存交换策略来减少静态计算图中特征图的大小。通过在 HelloDeep 框架上的实验表明, 本文提出的 GPU 内存优化方法相较于基线, 实现了最小的特征图内存占用空间大小, 并在多数情况下取得了 GPU 运行时内存占用的最小值或次小值, 验证了本文提出方法的有效性。

本文所提出工作的创新之处在于: 其一, 提出了一个新的、强调 GPU 高效利用的深度学习框架 HelloDeep; 其二, 考虑了前人工作中较少关注到的 Cudnn 辅助空间对 GPU 内存占用的影响, 并对 Cudnn 辅助空间进行限制; 其三, 考虑了用户对不同级别的 GPU 内存优化的需求, 提出了具有 3 种可选级别的 GPU 内存优化方法; 最后, 在 HelloDeep 上进行 GPU 高效利用方法的实现, 其实验结果更接近实际环境, 说服力更强。

关键词: 深度学习框架, GPU 利用率, GPU 内存优化

Abstract

As a hardware platform commonly used for deep-learning model training, GPUs are often faced with problems such as insufficient available memory and low kernel utilization during deep-learning application. This thesis takes the efficient utilization of GPU in deep-learning framework as the research subject, studies the method to improve GPU kernel utilization and reduce GPU memory consumption, and develops the HelloDeep deep learning framework as an experimental platform for verification.

Deep-learning framework is a software platform that performs the construction, training, and inference of deep neural networks. In order to implement and verify the GPU efficient utilization method proposed, this thesis proposes a deep-learning framework HelloDeep. The framework takes the static computation graph as the core, uses the asynchronous parallel execution engine to ensure the computing efficiency, uses the static memory pool to uniformly manage the global memory, and provides user-friendly operator encapsulation and data processing methods.

Data loading often becomes a bottleneck in the neural network training workflow. A slow data loading speed often leaves the GPU in an idle state waiting for data to be ready, reducing the overall GPU utilization. This paper proposes an efficient data IO module, which based on the producer-consumer model and uses multi-threaded asynchronous loading to speed up data processing. Experiments show that this method can increase the average utilization of GPUs in the process of neural network training from 25% in synchronous loading mode to more than 90%.

Insufficient available memory on GPU is one of the important reasons for the failure of deep-learning tasks. This thesis proposes a GPU memory optimization method that provides 3 optional optimization levels, which is convenient for users to choose according to the actual situation. This GPU memory optimization method employs variable recomputation and memory swapping strategies to reduce the size of feature maps in static computation graphs. Experiments on the HelloDeep framework show that, compared with the baseline, the GPU memory optimization method proposed achieves the smallest feature map memory footprint, and in most cases achieves the minimum or sub-smallest GPU runtime memory footprint value, which verifies the effectiveness of the method proposed in this paper.

The innovations of the work proposed in this thesis are as follows. Firstly, a new deep learning framework HelloDeep is proposed, which emphasizes the efficient utilization of GPU. Secondly, the GPU memory occupation by Cudnn auxiliary space which is less concerned in previous work, is considered and optimized. Thirdly, considering the user's requirements for different levels of GPU memory optimization, a GPU memory optimization method with 3 optional levels is proposed. Finally, since the implementation of the GPU efficient utilization methods is on HelloDeep deep-learning framework, the experimental results are closer to the actual environment and more convincing.

Keywords: Deep-learning framework, GPU kernel utilization, GPU memory consumption

目录

致谢..... I

摘要..... II

Abstract..... III

目录..... IV

图目录..... VI

表目录..... VIII

1 绪论..... 1

 1.1 研究背景和意义..... 1

 1.2 国内外研究现状..... 3

 1.2.1 主流深度学习框架中的 GPU 利用率提升方法..... 3

 1.2.2 模型压缩(量化)..... 4

 1.2.3 内存共享 (Memory sharing) 5

 1.2.4 重计算 (Recompute) 5

 1.2.5 内存交换 (Memory Swap) 6

 1.3 本文主要研究工作和组织结构..... 6

2 HelloDeep: 面向 GPU 高效利用的深度学习框架..... 8

 2.1 引言..... 8

 2.2 HelloDeep 框架架构及其 workflow 9

 2.3 框架内部模块介绍..... 10

 2.3.1 数据结构与内存管理模块..... 10

 2.3.2 用户接口和计算图模块..... 14

 2.3.3 执行器和并行引擎模块..... 17

 2.4 本章小结..... 21

3 HelloDeep 高效 IO 设计..... 22

 3.1 引言..... 22

 3.2 GPU 数据传输、读写最佳实践..... 23

 3.3 IO 模块设计..... 24

 3.4 IO 模块实现..... 27

 3.5 实验结果分析..... 31

 3.6 本章小结..... 32

4 HelloDeep GPU 内存优化方法..... 33

 4.1 引言..... 33

4.2 总体方法 33

4.3 模型试运行和重计算 checkpoint 候选点选取 36

4.4 启发式重计算和内存交换方案搜索 38

 4.4.1 方法描述 38

 4.4.2 具体实现 44

 4.4.3 实验验证 47

4.5 基于静态计算图的内存申请方案 50

 4.5.1 方法描述 50

 4.5.2 具体实现 53

 4.5.3 实验验证 54

4.6 Cudnn 卷积算法辅助空间限制 55

 4.6.1 方法描述 55

 4.6.2 实验验证 58

4.7 总体方法与 baseline 对比实验 58

4.8 本章小结 62

5 总结与展望 65

 5.1 本文主要工作总结 65

 5.2 未来工作展望 66

参考文献 68

作者简介 71

攻读硕士期间所取得科研成果 72

图目录

图 1.1 GPU 架构图 (NVIDIA TURING 架构) [3]	2
图 2.1 HelloDeep 框架架构图	9
图 2.2 HelloDeep 工作流程图	10
图 2.3 数据结构模块 (Tensor) 功能图	11
图 2.4 数据结构模块 (Tensor) 工作流程图	12
图 2.5 内存管理模块 (Storage) 功能图	13
图 2.6 内存管理模块 (Storage) 架构图	13
图 2.7 用户接口模块 (Symbol) 功能示意图	14
图 2.8 用户接口模块 (Symbol) 工作流程图	15
图 2.9 计算图模块 (ComGraph) 功能示意图	16
图 2.10 计算图模块 (ComGraph) 工作流程图	17
图 2.11 执行器模块 (Executor) 功能示意图	18
图 2.12 执行器模块 (Executor) 工作流程图	18
图 2.13 并行执行引擎模块 (Parallel) 功能示意图	19
图 2.14 并行执行引擎模块 (Parallel) 工作流程图	20
图 3.1 GPU 利用率-时刻曲线图 (直接使用 OpenCV 加载数据)	22
图 3.2 数据加载模块 (IO) 功能示意图	25
图 3.3 数据加载模块 (IO) 工作流程图	26
图 3.4 数据加载模块 Dataset 类 UML 图	27
图 3.5 数据加载模块 Normalizer 类及 Sampler 类 UML 图	28
图 3.6 数据加载模块 Batchifier 类 UML 图	29
图 3.7 数据加载模块 DataLoader 类 UML 图	29
图 3.8 GPU 利用率-时刻曲线图 (使用 HelloDeep-IO 加载数据)	32
图 4.1 HelloDeep GPU 内存优化总体方法示意图	35
图 4.2 ResNet-18 逐层运行时间柱状图	36
图 4.3 ResNet-18 逐层运行时间柱状图 (降序排列)	37
图 4.4 神经网络重训练 Checkpoint 示意图	39
图 4.5 神经网络重训练 Checkpoint 及内存交换示意图	40
图 4.6 启发式 Checkpoint 方案搜索算法流程图	43
图 4.7 神经网络重计算策略下计算图变化示意图	45
图 4.8 神经网络重计算及内存交换策略下计算图变化示意图	46
图 4.9 启发式 checkpoint 搜索算法不同阈值下内存占用散点图	48

图 4.10 一次前向-反向传播过程中 GPU 内存使用量随时间变化柱状图 49

图 4.11 神经网络内存分配过程示意图 50

图 4.12 基于静态计算图的内存申请方案算法流程图 52

图 4.13 内存申请算法不同 match_range 取值下 MemChunk 大小分布柱状图 55

图 4.14 重计算策略下 ResNet-50 不同 batchsize GPU 内存占用对比图 56

图 4.15 不同模型和不同方法下总体特征图大小值变化图 59

图 4.16 不同模型和不同方法下峰值内存大小值变化图 60

图 4.17 不同模型和不同方法下运算时长变化图 61

表目录

表 3-1 实验主要软硬件环境列表 31

表 4-1 内存重计算策略最低内存占用搜索 47

表 4-2 Cudnn 卷积前向计算不同算法的辅助空间大小 57

表 4-3 Cudnn 卷积前向计算不同限制下的效率和辅助空间大小 57

表 4-4 内存重计算策略最低内存占用搜索 58

表 4-5 基线对比实验结果 62

1 绪论

1.1 研究背景和意义

深度学习的迅速发展，给各个行业的应用带来了许多积极变化：人脸识别，语音助手，古籍修复，自动驾驶，智能推荐等等。深度学习的主要研究内容是深度神经网络（DNN）在图像、文本、语音等各个课题上的应用，但神经网络并非近些年的首创，而是从最早可追溯到1943年的沃伦·麦卡洛克等人的研究发展而来。Ian Goodfellow等人在其合著的《Deep learning》^[1]一书中认为，深度学习之所以有别于经典的统计机器学习，除了其结构层次比较深、比较庞大之外，更在于学习特征的提取方式。对于决策树、支持向量机等统计学习方法，最终模型学习到的特征（feature）是人为设置（hand-designed）的，如决策树的分支属性、向量机的核函数等；深度学习在此基础上更进一步，使得模型所要学习的特征由人工选择变为由模型自身学习得到，即深度学习学习到的不只是模型的参数，更学习到了何种特征才是对决策最有效的，这是所谓“深度”的更深层的含义——这一点对计算机视觉、自然语言处理等研究课题而言尤为重要。

在历史上，神经网络的发展经过了几个阶段，但最终因为当时的计算资源不足以支持神经网络的计算而停滞。到了21世纪10年代，学者发现可以利用GPU对神经网络进行有效的训练^[2]，并在图像分类领域取得明显优于传统方法的结果，从此开启了深度神经网络的研究应用热潮。

GPU (Graphics Processing Unit, 图形处理单元)，是一种广泛应用于个人计算机、工作站、游戏机以及一些移动设备上的微处理器，通常负责渲染并输出显示图形。GPU的芯片架构如图1.1所示，以NVIDIA Turing架构为例，包含最高4608个CUDA Cores以及576个Tensor Cores，同时还拥有最高384比特的内存位宽，这意味着GPU拥有强大的并行计算能力和高速的内存访问能力；不仅如此，GPU在一个时钟周期内还能提供比CPU更高的每秒浮点运算次数（FLOPs）。这些特性都十分适合于并行浮点数运算，这也就使得GPU迅速成为最适合进行深度神经网络计算的硬件设备。

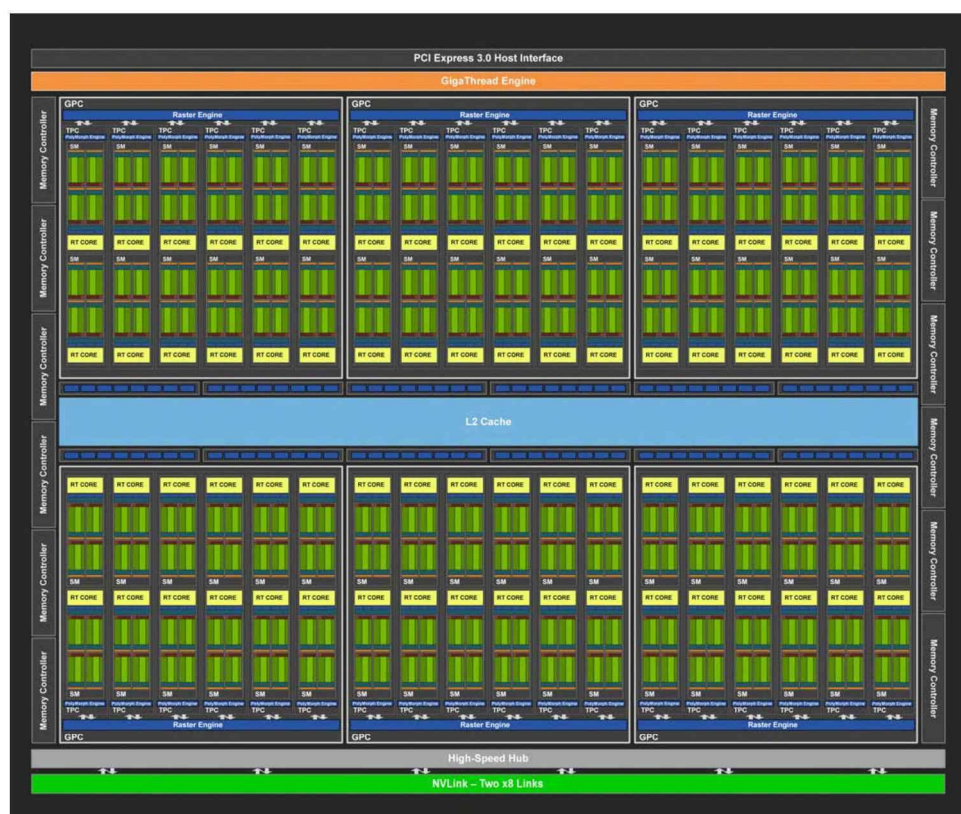


图 1.1 GPU 架构图（NVIDIA TURING 架构）^[3]

由于深度神经网络计算密集的特性，通用的做法是将深度网络模型在高性能 GPU 主机（或集群）上进行训练，随后部署到不同的应用场景中。为了支持深度学习应用的开发，软件工程师们开发了一套易于研究人员使用的工具，称深度学习框架，如 TensorFlow^[4]，Pytorch^[5]，MXNet^[6]等。相较于神经网络研究人员关注模型的设计和达到的效果，神经网络框架开发人员更关注神经网络在计算机中的表示、训练和执行过程中的内存开销和性能优化。

时至今日，GPU 可用内存不足（out of memory, OOM）仍然是多数研究者在训练神经网络时遇到的主要困扰之一。究其原因，主要是因为神经网络模型的参数量十分庞大，且仍在飞快增长当中。以常用的 ResNet-50^[7]和 Vgg-16^[8]为例，它们的参数量分别达到了 0.46 亿和 1.38 亿；而由 OpenAI 在 2020 年提出的 GPT-3^[9]模型更是达到了 1750 亿的参数量，模型占用约 700GB 空间，使用的数据集最大达 45TB。常见的深度学习服务器的内存配置为 128GB，GPU 显存为 44GB（RTX 2080Ti*4），相较于模型和数据集的体量而言可谓是杯水车薪，可见深度学习总是内存饥渴的。

不论是什么方面的深度学习课题，GPU的高效利用都是绕不开的一步。不难看出，在内存资源固定的情况下，在软件层面做出改进，提升GPU使用效率，降低神经网络训练过程中的内存使用量，具有重要研究价值。

本研究旨在研究深度学习框架中GPU使用效率高效利用问题，以实现减少GPU内存需求、提升训练效率的目的。本研究的意义在于，通过提高GPU使用效率，来降低研究人员和企业的计算资源和存储资源的开支，提升训练的速度，最大限度地发挥已有资源的作用。为测试本研究提出的GPU高效利用方法，本文还提出了一个自主研发的深度学习框架HelloDeep，并在该深度学习框架上进行本研究中提出的各类GPU优化方法的测试。

1.2 国内外研究现状

基于深度神经网络的方法相较传统方法虽然有很高的精确度，但其计算量需求十分巨大。Yanjie Gao等人详细研究了深度学习中GPU的使用情况^[10]，指出在所有失败的深度学习任务中，大约8.8%的原因来自于GPU资源的耗尽；同时，GPU的实际占用情况和根据神经网络特征图大小来进行估算得出的占用情况有着较大差距，其差距主要来源于Cudnn辅助空间、Cuda数据结构以及一些临时变量。

1.2.1 主流深度学习框架中的 GPU 利用率提升方法

目前，主流深度学习框架都应用了若干提升GPU利用率的技术。以Caffe^[11]、Torch^[12]为代表的静态图框架利用计算图已知的特性，通过预先的分析得到变量间的依赖情况，以便在训练过程中实时释放不再被依赖的变量。

TensorFlow使用BFC (best-fit with coalescing)算法进行GPU动态内存管理，该算法是Doug Lea's malloc^[13]动态内存分配算法的一个简单实现版本。BFC算法的基本思想是维护一个双链表数据结构，其基本元素称Chunk，包含了内存块的大小、地址、使用状态等一些基础信息，利用该数据结构实现了内存申请、释放以及内存碎片回收等功能。它的核心思想是将大的内存块通过split操作分成若干的小内存块，通过merge操作将内存块合并为更大的内存块，以满足不同大小内存申请的需求。

Pytorch则为每个变量绑定了一个Allocator和Deleter，它们都是动态绑定的。所谓的动态绑定，就是根据运行时该变量所处的上下文环境来选择恰当的申请器和释放器，例如CPU Memory Allocator，MKLDNN Memory Allocator，CUDA Memory Allocator等。特别地，对于GPU内存，Pytorch的内存分配器设计了一个内存缓存池，并试图在每次内存分配时应用最佳适应（Best-Fit）策略，即分配一个最小的满足需求的内存块。其中，大的内存块（>1MB）和小的内存块分别由两个不同的内存池进行分配。

MXNet在Caffe的静态图分析的基础之上，加入了内存重计算策略，该策略也由MXNet的主要代码贡献者之一的陈天奇发表在2016年的一篇文章^[14]中。内存重计算的核心是以计算换内存，通过舍弃一部分变量来减少运行时的内存占用峰值，并在反向传播时通过重新计算这些变量来使得梯度传递得以完成。理论上，该方法可以将特征图总体占用降低到次线性级。

除了神经网络框架中应用的策略之外，不少学者也单独对GPU内存优化作出研究。这些研究，可以大略分为四类：

1.2.2 模型压缩(量化)

模型压缩指的是在不过多损失模型准确度的前提下，降低模型表示的精度，这项方法也通常被称作模型量化。早在20世纪90年代，就已经有学者开展关于低精度数据进行神经网络计算的研究^{[15][16]}；然而受资源限制，彼时的工作都是在小规模的网络和数据集上进行，不能很好地说明该方法在当下超大规模神经网络中是否继续有效。2011年，Vanhoucke等人在研究^[17]中指出，使用8位固定精度的数据进行运算，相较于使用float型浮点数而言，可以获得3倍的速度提升。^[18]中，M. Courbariaux等人使用半精度浮点数(FP16)和混合精度浮点数来代替原来的32位浮点数，比较了不同精度在多个基线数据集中的结果；同时，作者研究了在量化过程中产生的量化误差，指出在模型准确度和模型精度之间存在一个Tradeoff，在不同的应用场景下有不同的应用。为了解决模型准确度随数据精度的下降而下降的问题，Christopher等人提出了一种低精度时仍能保持模型的高准确度的训练算法HALP^[19]。该算法应用了动态数据精度策略，它的核心思想是bit centering，即随着训练进行，梯度的变化值越来越小，此时可供压缩的数据空间就更大；利

用这一事实,就可以通过动态调整数据精度,来获得在数据低精度时保持模型高准确度的效果。

1.2.3 内存共享 (Memory sharing)

内存共享是指不同变量对同一块内存进行复用的技术,通常有两种形式:第一种是原地(In-Place)操作,即一个运算在输入变量的内存中直接做修改来得到输出变量。但在DNN中,支持原地操作的算子较少;共享内存另一种形式是指两个生命周期不重叠的变量,在不同的时间上利用同样的内存空间的技术,也常称作内存池技术。主流神经网络框架如MXNet、Pytorch和TensorFlow等都运用了内存池技术。

利用内存池进行动态内存分配的问题可以称作DSA(Dynamic Storage Allocation)问题^[20],分为在线DSA和离线DSA两种情况。在线DSA需要在不预先提供任何信息的情况下满足每一次内存申请,离线DSA则是在预先知道每个内存申请到达的时机以及释放时机。在线DSA是一个典型的NP-Hard问题^{[21][22]},而离线DSA则提供了更多优化的空间。Junzhe Zhang等提出了SmartPool^[23],一种基于静态计算图分析的智能离线DSA算法,该算法旨在降低内存申请过程中的峰值使用量,提供了First-Fit和Best-Fit两种内存分配方案。Sentinel^[24]这项工作中还考虑了内存分布对内存池性能的影响,指出高频访问的变量和低频访问的变量应该分别存放在内存池中的不同物理区域,否则将会影响操作系统Page对齐,从而影响整体内存访问的性能。不仅如此,内存中数据结构的设计也可以提升内存池性能^[25],主要考虑利用内存对齐、空间一致性和硬件预读取来提升TLB和缓存的性能。

1.2.4 重计算 (Recompute)

重计算方法是计算过程中一些可复用的临时对象在不使用时释放,以换取更多内存的策略,被释放掉的变量将会在下一轮迭代中被重新计算、加载。Memonger^[14]和SuperNeurons^[26]中采用的正是这样一种方法:研究者发现POOL, ACTIVATION, LRN和BN这些算子平均使用了50%的内存,在一次前向计算中占用的时间却不到10%;那么如果不保留这些计算的临时结果,就可以节约50%的

内存，所付出的代价就是在反向传播时进行重新计算而耗费的额外时间。在该过程中，保留在GPU内存中的节点（层）一般称checkpoint。

1.2.5 内存交换（Memory Swap）

在使用GPU对神经网络进行训练时，为了减少峰值内存消耗，可以将GPU中暂时用不到的变量交换到CPU中，在需要的时候再重新交换回来。值得注意的是，GPU和CPU之间的内存交换，和GPU正在进行的运算，应该尽量并行流水化，以免因为所需变量尚未交换完成而堵塞GPU的计算，降低整体效率。

GeePS^[27]和SuperNeurons^[26]中都应用了内存交换技术，但这些工作中，可被交换的算子或者操作，都需要由深度神经网络开发人员手动指定，这要求开发者对神经网络有较好的理解。Capuchin^[28]将内存交换的颗粒度进一步细化到Tensor级别，并根据Tensor自身的属性（而非它们的算子类型）来决定交换方案，避免依赖过多的先验知识。[23]中，研究者提出了AutoSwap算法，能根据变量自身的属性，包括Absence时长（即可以被交换到CPU中的时长）、Absence面积以及它们之间的加权和，定量地给出一个分数，并根据该分数来决定一个变量是否应该被交换到CPU内存。SwapAdvisor^[29]则使用了遗传算法来进行搜索，以找到一个合适的内存交换方案。

1.3 本文主要研究工作和组织结构

本文主要对深度学习框架中的GPU高效利用问题进行研究，提出优化GPU利用率和GPU内存占用的方法。为验证这些方法，本研究提出了一个面向GPU高效利用的深度学习框架HelloDeep。GPU的高效利用包括两方面，其一是使得GPU利用率保持在较高水平，而另一方面则是降低深度模型训练过程中的GPU内存占用率，降低硬件成本，提升GPU使用效率，使得训练得以高效运行。本文介绍了提升GPU利用率和降低GPU内存占用的方法，并说明了这些方法在深度学习框架HelloDeep中的实现，同时通过实验来验证优化方法的有效性。

本研究总体按照如下方式进行组织：首先，介绍面向GPU高效利用的深度学习框架HelloDeep；其次，介绍基于生产者-消费者模式的异步数据加载模块；随

后,对基于静态图分析的多级别GPU内存优化方法进行介绍;最后,对本文所做工作进行总结与展望。具体章节安排如下:

在第二章中,本文首先介绍了HelloDeep——一个面向GPU高效利用的深度学习框架。之所以先对框架进行介绍,是因为该框架是后面三、四章将要介绍的工作的平台。本文对GPU高效利用的研究成果被实现到HelloDeep当中,并通过在HelloDeep框架中进行神经网络训练来评估本文所提出方法的效果。在该章节中,首先将介绍HelloDeep的整体工作流程,展现HelloDeep内部的各个模块以及它们之间的联系;随后,将分别介绍几个组成HelloDeep框架的核心模块。这些模块不仅在第二章中被提及,在第三、第四章的实现部分中,也会基于这些模块来对实现工作进行说明。

在第三章中,本文介绍了基于生产者-消费者模式的异步数据加载模块(IO模块)。在该章节中,首先说明了数据加载和GPU利用率之间的联系,证明一个高效的数据IO对于高GPU利用率的必要性,同时也介绍了提升NVIDIA GPU性能的一些技巧;随后,通过流程图来说明高效IO模块设计的思路,并以UML类图的形式介绍高效IO模块的实现过程。最后,通过在HelloDeep深度学习框架上的实验来验证IO模块对GPU利用率的提升,观察GPU利用率是否符合预期。

在第四章中,本文介绍了基于静态图分析的多级别GPU内存优化方案。首先,介绍GPU内存优化的总体流程,以及总体流程在基础-通常-极致三个不同优化级别下的区别;随后,对总体流程中的每个步骤进行详细的展开说明,包括checkpoint候选点选择、启发式checkpoint方案搜索、基于静态图分析的离线DSA内存池,以及基于特征图大小的Cudnn辅助空间限制。其中,对于比较复杂的启发式搜索和离线DSA内存池,还展开说明了其在HelloDeep框架中的具体实现。在此过程中,通过在HelloDeep框架上的测试来分别验证各个方法的有效性以及总体三个优化级别的有效性,并与主流深度学习框架进行对比。

在第五章中,对本文所做工作做一次回顾与总结,整理出还可以继续进行提升的工作点,以期后续进行改进提升。

2 HelloDeep: 面向 GPU 高效利用的深度学习框架

2.1 引言

深度学习框架是研究深度学习课题的主要工具,无论是学术界的研究,还是工业界的应用,所有的深度学习任务都运行在深度学习框架上。在国际上,第一代深度学习框架以Caffe、TensorFlow 1.0为代表,第二代深度学习框架则是以Pytorch、TensorFlow 2.0为代表。国内也有一系列的国产深度学习框架,如百度的PaddlePaddle^[30],华为的MindSpore^[31],清华大学的Jittor^[32]等。同时,在工业界,也存在一些和上述通用型框架不同的、针对业务场景专门优化的框架,如阿里巴巴的XDL^[33]和腾讯的NCNN^[34]等。

纵观深度学习框架近年来的发展历程,其发展方向大概有如下几点:其一,由单一语言前段向多语言前端发展;其二,由通用型框架细分为场景特化型框架,以TensorFlow为例,除了主要的TensorFlow API之外,还提供了面向服务端部署的TensorFlow Serving组件,面向边缘部署的TensorFlow Lite组件,面向端到端生产环境的TensorFlow Extended组件等。总而言之,其发展趋势就是从GPU单机训练部署,向云端分布式训练和边缘计算发展,打造完整生态;其三,底层利用算子融合^[35]、子图划分^[36]、代码自动优化^[37]、即时编译^[32]等技术,不断优化性能表现;其四,底层支持更多深度学习加速硬件,包括FPGA、Google TPU^[38],以及华为海思昇腾^[39]、寒武纪MLU^[40]等部分国产加速芯片,提升对更多编译器和平台的兼容性,等等。

由于深度学习框架的重要性以及实用性,其得到了国内外开发人员的广泛重视和研究。考虑到TensorFlow、Pytorch等成熟框架内部模块耦合的复杂度,在这些框架上开展本文所研究的GPU高效利用问题将变得比较困难;同时,往成熟的框架中添加新的特性,往往会对框架本身的运行效率产生负面影响,甚至产生致命错误。因此,为更好地实现和评估本研究所提出的GPU高效利用方法,本章提出了一个自主开发的深度学习框架HelloDeep。HelloDeep支持基本的数据加载和模型训练、推理功能,以及模型并行、数据并行等特性。在软件国产化的大背景下,开发拥有自主知识产权的深度学习框架HelloDeep,也能为软件的国产化做出一点贡献。

2.2 HelloDeep 框架架构及其 workflow

HelloDeep 的架构如图 2.1 所示的框架架构图所示，共分为九个模块。其中，对外接口是指 HelloDeep 的用户直接使用的接口，包括网络表示模块 Symbol，数据结构模块 Tensor，以及数据加载模块 IO。内部模块则是 HelloDeep 内部用于实现神经网络训练、推理功能所用到的模块，用户不会直接接触它们。内部模块包括了静态计算图 ComGraph 模块，图执行器 Executor 模块，异步并行执行引擎 Parallel 模块，统一内存管理 Storage 模块，神经网络算子 Operator 模块，以及神经网络权值更新 Optimizer 模块。

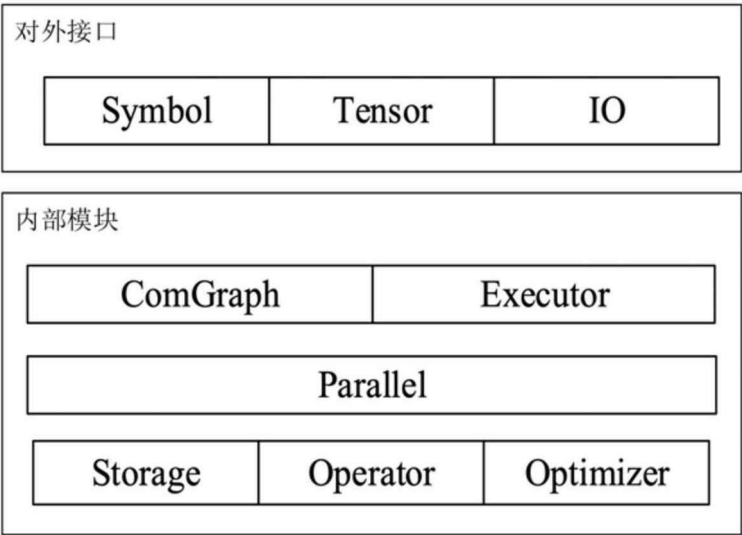


图 2.1 HelloDeep 框架架构图

HelloDeep 的工作流如图 2.2 所示。用户通过用户接口定义好网络结构，该网络在框架内部表示为计算图，图中的节点是各类操作，即卷积、池化、归一化等算子，而连接各个节点的边则是输入输出的数据。HelloDeep 所支持的算子由 Operator 模块决定，而网络的输入数据则来源于用户定义的数据加载器实例。在操作和数据都准备完毕后，执行器 Executor 便依据计算图的信息，完成中间变量的形状、数据类型、运行时上下文的推理，并使用内存池技术分配中间输出变量的内存空间。随后，执行器根据前向计算图来生成反向传播的计算图，将两张计算图合并后，以拓扑顺序将整张计算图拆分为具有前后依赖关系的若干计算任务，按顺序进行前向传播和反向传播计算。执行器 Executor 所做的工作是分发计算任务而并非实际参与计算，实际的计算将由异步并行运算引擎 Parallel 模块进行。

Parallel 模块将根据各个计算任务之间的依赖关系,尽可能地将没有相互依赖的计算进行并行化处理。在 **Parallel** 模块完成所有计算后,得出的梯度信息将会由优化器 **Optimizer** 模块进行模型参数的更新。若干次迭代后,即可完成模型的训练工作。

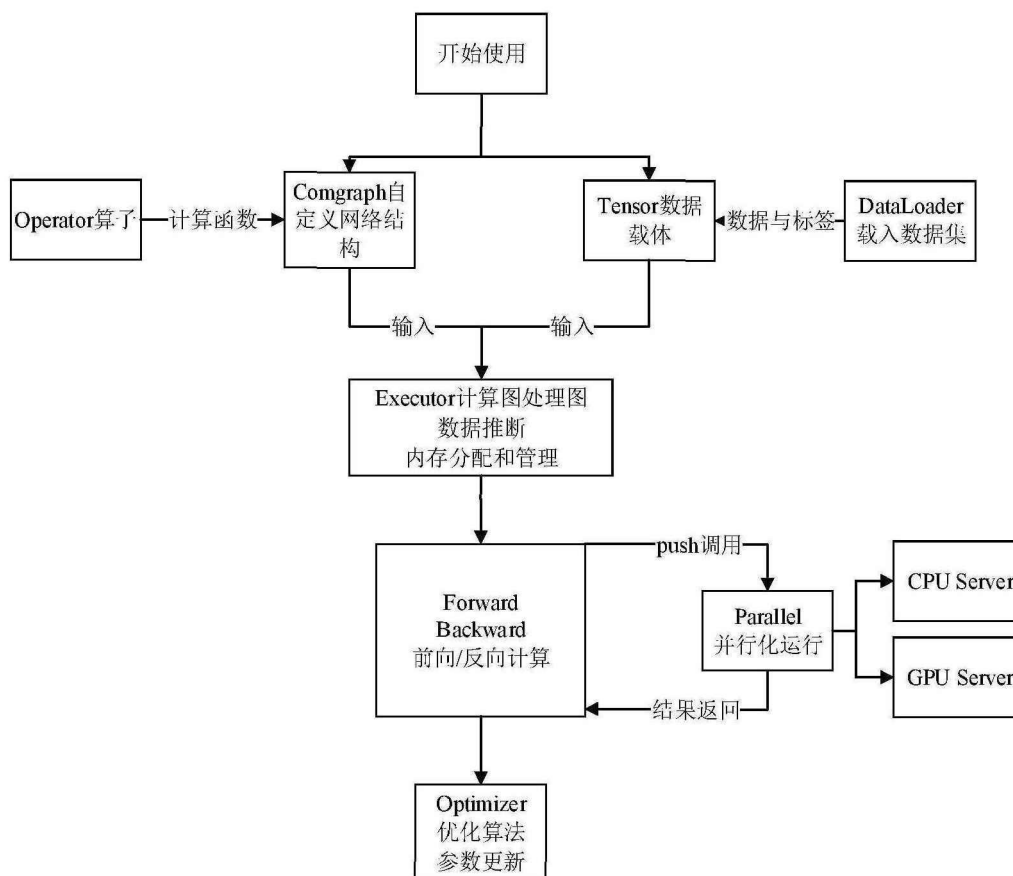


图 2.2 HelloDeep 工作流程图

2.3 框架内部模块介绍

本节将展开介绍 HelloDeep 内部 9 个模块中比较重要的 6 个模块,这些模块是本文第三、四章中的实现部分的基础,所以有必要对其进行说明。

2.3.1 数据结构与内存管理模块

HelloDeep 提供了一个多维数组数据结构 Tensor,用以代表神经网络中的数据,包括输入输出、标签、权值、偏置矩阵等。Tensor 是用户直接接触的接口,

也其他模块操作的对象，所以对Tensor模块的设计需要同时兼顾用户的易用性和被其他模块调用的可用性。具体而言，如图2.3所示，Tensor模块的设计特点为：

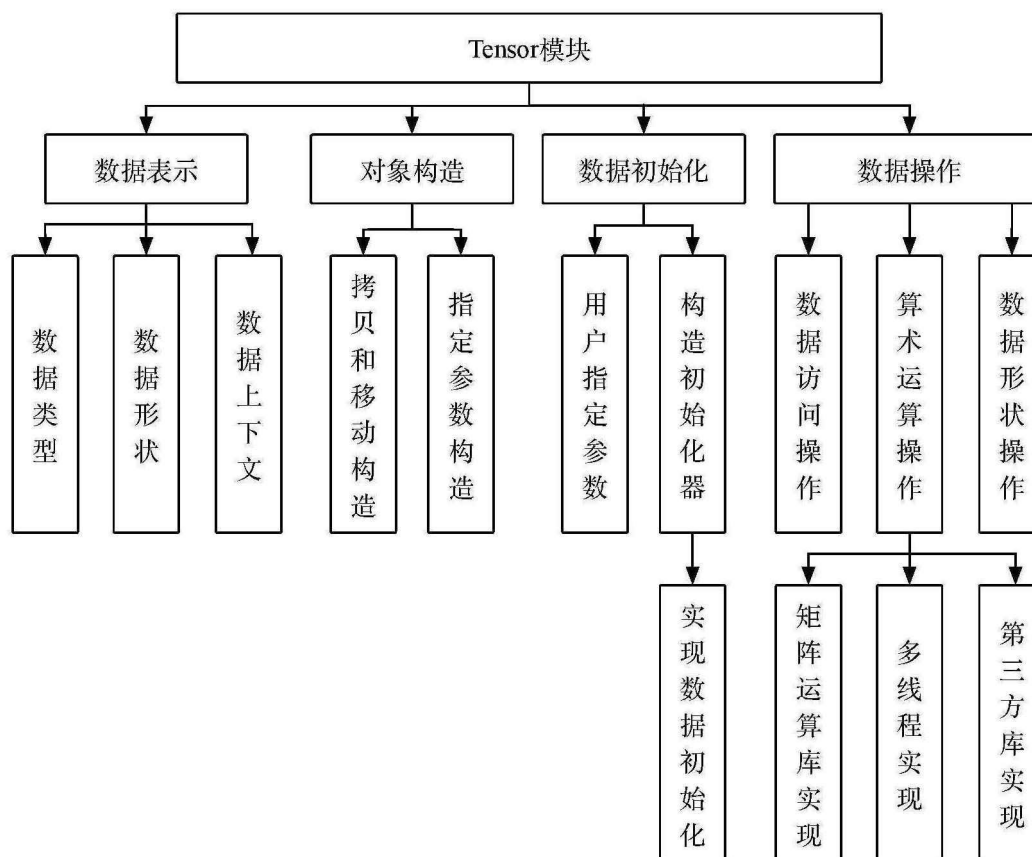


图 2.3 数据结构模块 (Tensor) 功能图

- 1) **简单易用的接口**：符合用户编程直觉的接口，例如指定数据类型、数据形状和数据上下文的构造，支持对对象任意的拷贝和移动操作，获取Tensor的形状等；
- 2) **高效的内存分配/回收机制**：通过内部的MemChunk封装实现内存的自动回收机制，调用Storage模块实现不同设备内存的分配和释放；
- 3) **多样化的初始化方式**：默认情况下向内存申请的数据是未经初始化的，Tensor提供如常数初始化、列表初始化、符合指定分布的随机初始化等各种初始化方式；
- 4) **多样化的变换函数**：对已经构建的Tensor对象，提供View、Reshape、切片、改变上下文设备等各种变换；

- 5) **支持多种数据类型的泛型函数**: 支持不同精度的浮点数和有符号、无符号整型数, 内部所有与数据类型有关的成员函数, 都支持对这些不同数据类型进行泛型操作;
- 6) **支持多种设备上下文**: 支持表示GPU以及CPU和后续将扩展支持的其他芯片等设备中的数据;
- 7) **支持基本四则运算**: 包括元素级的加减乘除运算;
- 8) **支持不同设备间拷贝**: 在不同的设备之间都能实现高效的数据拷贝传输操作;
- 9) **支持保存和导入**: 可以将内容保存到二进制文件以及从二进制文件导入。

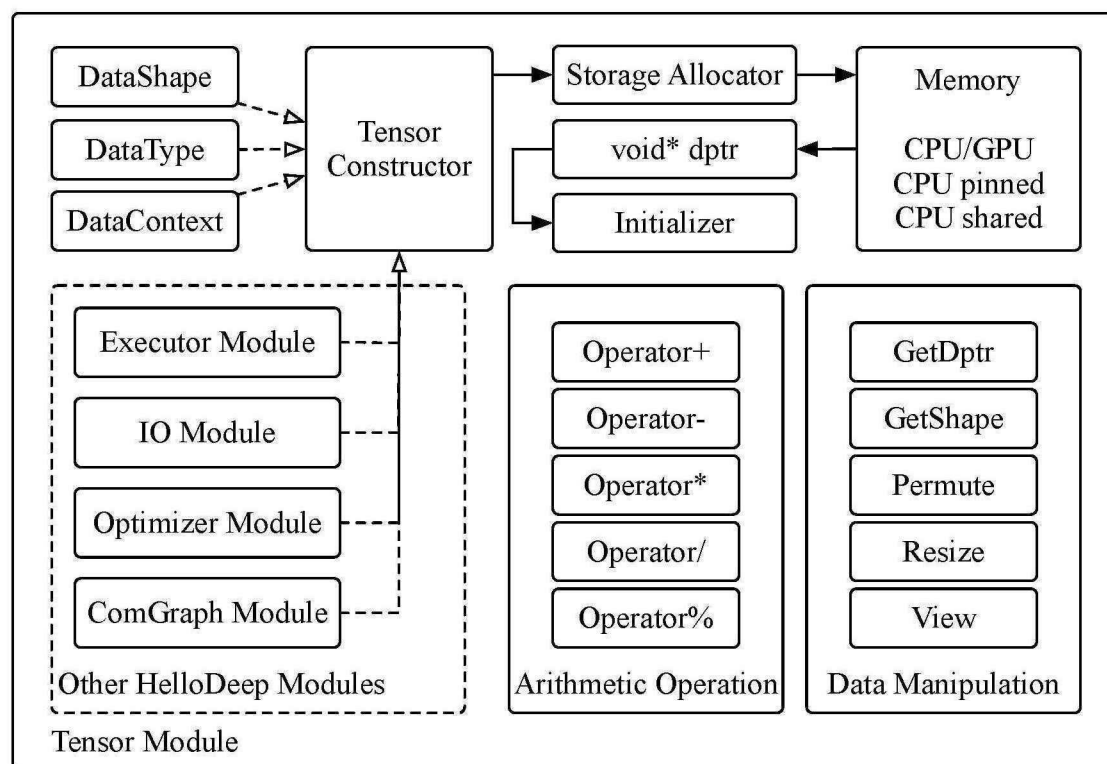


图 2.4 数据结构模块 (Tensor) 工作流程图

Tensor模块的工作流程如图2.4所示。用户或者HelloDeep中的其他模块（执行器模块，数据加载模块，优化器模块，计算图模块等），在某个时刻都会提供不同的信息（形状、类型、上下文），调用Tensor的构造函数来获取若干个Tensor对象；Tensor的构造函数会调用Storage Allocator来向CPU或GPU申请内存，并返回指向一块内存的StorageHandle。

对于已经存在的Tensor，HelloDeep提供了两类主要的操作。第一类是数据的算术运算，包括加减乘除等，通常应用于Optimizer中，以求解梯度和更新权值；

第二类是Tensor属性和数据的获取操作,一系列元素操作包括Slice(), At(), View(), 以及拷贝到不同设备和初始化数据等数据操作。

Tensor模块是对数据的封装,但其并不直接管理内存。内存的申请和释放,都由Storage模块负责。Storage模块的主要任务是封装底层不同设备的内存管理方法,对上层提供统一的内存分配与释放接口,并且在模块内部统一管理已分配的内存资源,这样开发者可以方便地使用内存管理模块来申请不同设备上的内存资源,而且不需要手动管理内存。内存管理模块中内存分配器(Allocator)的易于扩展性使框架在支持新的硬件设备时,框架其他模块的代码不需要重构,减少了框架内模块之间的耦合性,方便框架未来的开发使用。

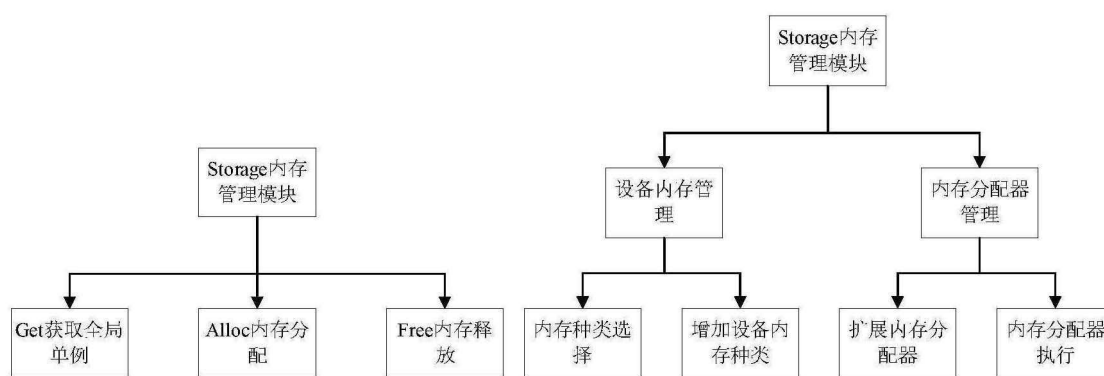


图 2.5 内存管理模块(Storage) 功能图

图 2.6 内存管理模块(Storage) 架构图

Storage模块提供的功能如图2.5所示。为管理全局HelloDeep框架内全局内存,Storage被实现为单例模式,开发者(指HelloDeep的开发者,而非用户)可以通过Get()方法来获取全局的Storage单例对象。Storage单例对象提供了Alloc()和Free()两个接口,分别用于获取和归还内存。在Storage内部,Alloc()方法实际上被继承为多种不同内存池的接口,Storage模块会根据开发者的设置,从多种内存池中选择一个,然后调用其重写的Alloc()接口;Release()方法同样如此。

Storage内部的模块架构如图2.6所示。为了支持不同种类的设备内存和具体的不同的内存分配器,Storage模块分为设备内存管理子模块和内存分配器管理子模块。设备内存管理模块会针对不同的设备平台,如CPU、GPU,以及可扩展的更多设备,提供不同的方法封装。例如,对于CPU平台,就可以使用posix_memalign()方法来获取字节对齐的内存,以获取更好的内存性能。而内存分配器子模块,则包含了多种不同的内存分配方案,包括直接分配、内存池分配等。在内存池内部,

也存在着多种不同的分桶策略。通过这样的架构，Storage模块实现了统一的、多设备的内存管理接口。

2.3.2 用户接口和计算图模块

HelloDeep 是基于静态计算图的深度学习框架，在用户视角下，用户使用 HelloDeep 的用户接口模块，即 Symbol 模块来搭建静态计算图；在框架内部，通过 ComGraph 计算图模块来表示静态计算图。

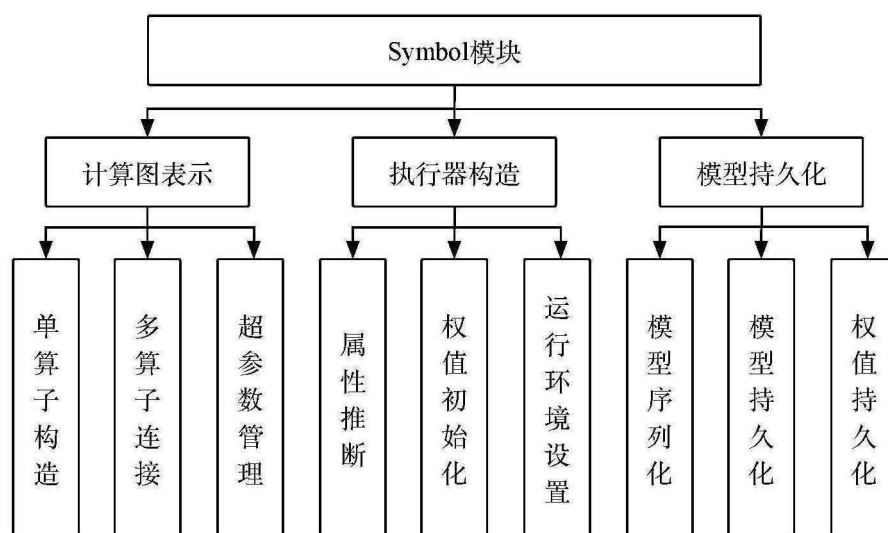


图 2.7 用户接口模块 (Symbol) 功能示意图

用户接口模块 (Symbol) 的功能如图 2.7 所示。Symbol 模块面向用户，其设计目标是为 HelloDeep 的用户提供一个使用逻辑上与 Pytorch 的 `nn.layer` 相近的一套神经网络表示接口。利用 Symbol 模块，用户可以方便地定义神经网络的每一个算子，为它们设置超参数，并将它们组合成完整的网络。对于 Symbol 接口表示的神经网络模型，Symbol 还给用户提供了模型参数的查看、调试和序列化功能，序列化后的 Symbol 模型和权值文件可以导出到文件系统进行持久化保存。

对于 HelloDeep 的底层模块如 ComGraph、Executor 等，Symbol 提供了对计算图更高级别的抽象和封装，同时完成所有网络内部变量的形状和数据类型的推断工作，在指定的上下文设备中申请并初始化训练权值的 Tensor 对象。Symbol 将用户提供的 Operator 和超参数信息转换为计算图，调用执行器 Executor 的构造函数，返回给用户一个可以运行前向推理和反向传播的 Executor 对象。

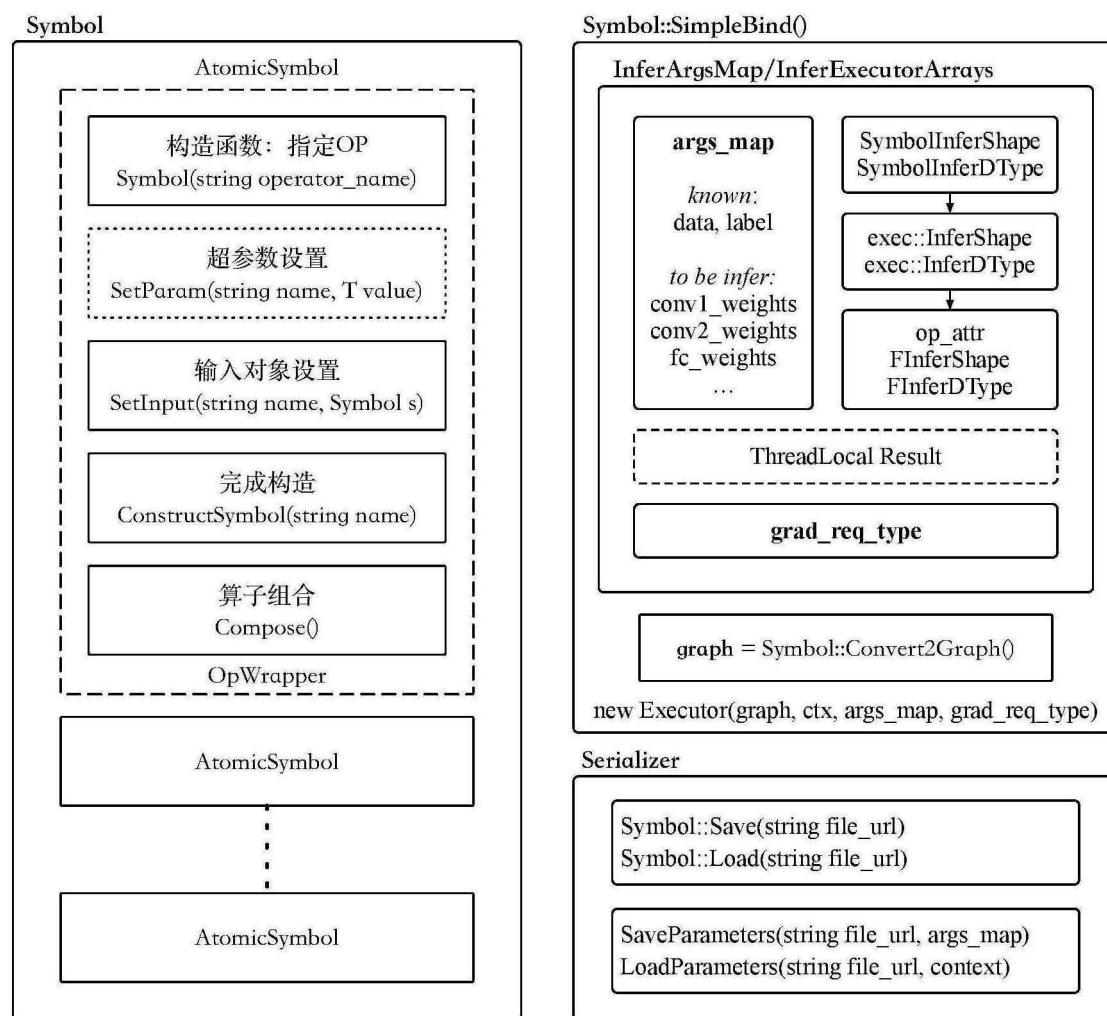


图 2.8 用户接口模块 (Symbol) 工作流程图

Symbol 模块的工作流程如图 2.8 所示。图中左半部分是建立单个算子，并将多个算子组合成计算图的工作流程。AtomicSymbol 表示最小的、不可分的计算图单元，即算子。在其构造函数中，首先需要传入一个表示算子名称的字符串，如“Convolution”；随后为该算子设置超参数，这一步是非必需的，因为部分算子不需要设置超参数。在此之后，通过 SetInput() 接口设置当前 Symbol 的输入，从网络视角来看就是当前层的上一层 Symbol。最后，在 ConstructSymbol() 方法中完成 Symbol 的构造。Compose() 方法在 ConstructSymbol() 中被调用，它实际上完成了所有单个算子的连接，使之成为一张完整的计算图。为了方便用户的使用，HelloDeep 通过 OpWrapper 接口封装了 Symbol 的构造过程，使得本段文字中上述所有函数都是对用户隐藏的，提升了接口的易用性。

在上图的右侧，描述了 Symbol 另外两个主要功能的流程。SimpleBind() 函数

在 **Symbol** 模块中负责推理计算图信息，构造并返回执行器对象。在 **InferArgsMap()**和 **InferExecutorArrays()**两个函数中，通过输入的数据和标签信息，推理出所有中间变量的大小、数据类型、运行设备等信息。随后，用 **Symbol::Convert2Graph()**函数将 **Symbol** 对象转换为 **HelloDeep** 内部计算图的表示形式，即马上将要介绍的 **ComGraph** 模块。最后还要根据当前是否为训练模式来设置梯度需求，这些数据一并作为参数传给 **Executor** 的构造函数，并向用户返回得到的执行器。用户执行若干次训练后，可以调用 **Symbol** 模块的序列化接口，将训练好的模型及其权值保存到文件系统中。

在 **HelloDeep** 框架内部，计算图由 **ComGraph** 模块表示。**ComGraph** 模块的主要功能是接收上层用户通过 **Symbol** 模块定义的计算流程（通常是深度神经网络），抽象成为计算图，并且为 **Executor** 模块提供分析整体流程的函数。如图 2.9 所示，**ComGraph** 可以细分为计算流程抽象化、计算图序列化以及计算图分析优化三大功能模块。

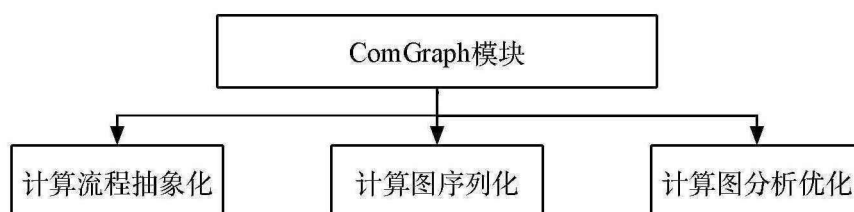


图 2.9 计算图模块 (**ComGraph**) 功能示意图

计算流程抽象是指将用户自定义的计算流程（通常是深度神经网络）抽象成有向无环图的数据结构，进而得到计算图。

计算图序列化是将图数据结构序列化，为图中的节点以及有向边提供唯一索引值。不仅如此，计算图序列化模块还为节点以及有向边提供了双向索引函数，使得每一条边可以找到与之相关的节点，也使得每个节点可以找到与之相连的有向边，大大扩展了便捷性。

计算图分析优化为 **Executor** 模块提供了对于计算图整体分析优化的相关函数。主要包括：构建反向计算图的 **Gradient()**函数，实现自动微分的逻辑；一系列推断函数，用来完善整个计算图的数据信息；内存分配计划函数，分析整体计算流程，达到内存复用节省内存的效果。有关内存分配的详细方法，将在第四章展开介绍。

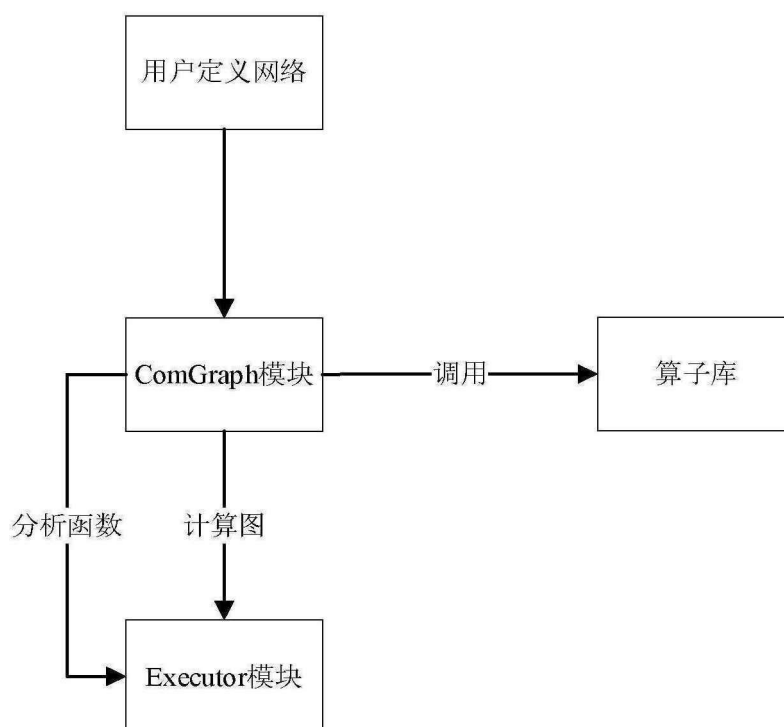


图 2.10 计算图模块 (ComGraph) 工作流程图

图 2.10 说明了计算图模块的工作流程。首先，用户通过 Symbol 接口进行神经网络的定义；随后，框架内部将 Symbol 对象转换为 ComGraph 对象，计算图 Graph 由节点 Node 和边 Edge 组成，其中计算图中的各个节点代表算子，各个边代表中间生成的特征图。各个算子的特性由算子库提供，包括算子的前向传播计算函数，梯度生成函数，反向计算函数，超参数解析函数等等。ComGraph 模块蕴含了执行神经网络运算的所有信息，这些信息将会交由执行器 Executor 模块进行进一步处理。

2.3.3 执行器和并行引擎模块

上节介绍了用户如何通过 Symbol 模块来定义神经网络，以及神经网络如何在 ComGraph 模块中表示为计算图。本节中将介绍使用 ComGraph 静态图信息进行任务打包的执行器 Executor 模块，以及执行任务的异步引擎 Parallel 模块。

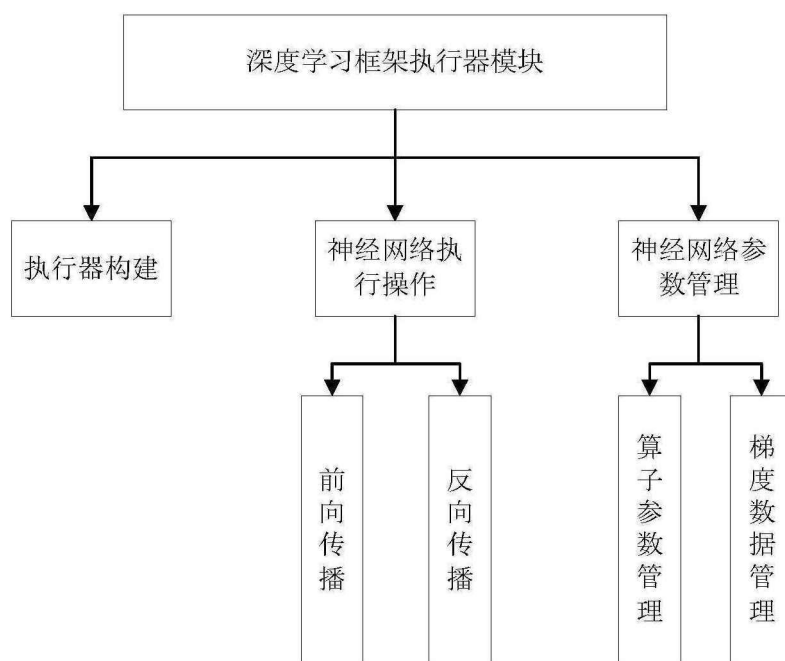


图 2.11 执行器模块 (Executor) 功能示意图

执行器 Executor 模块是 HelloDeep 框架给用户提供的操作“把手”，通过执行器模块，用户可以执行整个网络的前向推理和反向传播。在框架内部，Executor 模块承上启下，负责加工计算图信息，打包计算任务，并将这些任务推送至并行引擎。在引擎完成计算之后，得出的梯度信息也由执行器进行管理。其模块功能如图 2.11 所示。

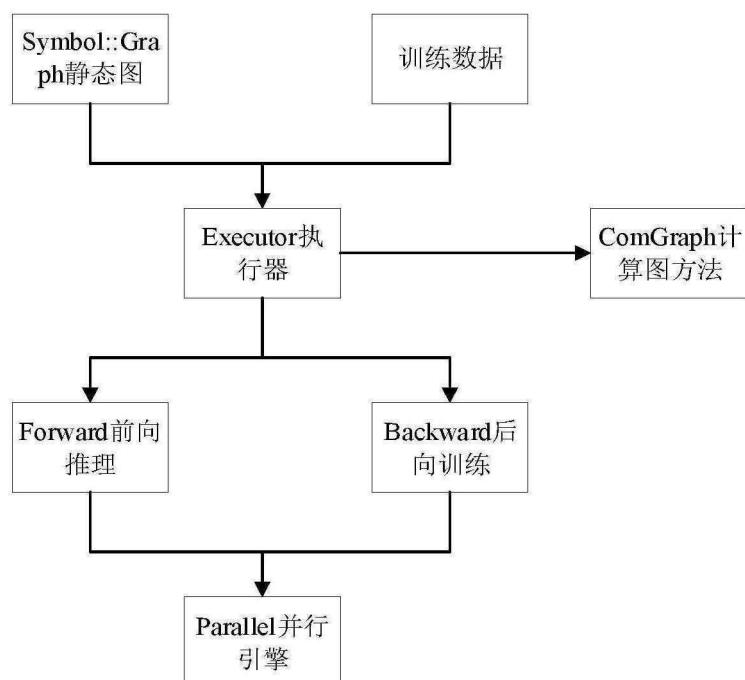


图 2.12 执行器模块 (Executor) 工作流程图

图 2.12 体现了执行器模块的工作流程。在构造 **Executor** 对象的过程中，先根据前向计算图推导出反向计算图并将二者合并，然后通过拓扑排序得到所有算子的计算顺序。根据算子间的相互依赖关系，调用分析函数给出中间变量的内存共享方案，即第四章将要详细介绍的内容；然后，将各个算子打包成一个个计算任务，每个任务内包含了来自算子库的当前算子对应的计算函数和反向计算函数，以及每个计算任务的输入数据和输出变量。最后，执行器依次将打包好的任务推送至并行引擎 **Parallel** 模块进行计算。

并行执行引擎 **Parallel** 模块是一个采用 **ProActor** 模式的 **IO** 多路复用多线程模型，在框架内负责执行经由 **Parallel::Push()** 接口提交的任务，并试图在满足任务间依赖关系的前提下尽可能地并行执行。为了实现这一要求，**Parallel** 模块内部需要提供两个子功能，一是防止资源在多线程情况下发生冲突的资源同步功能，二是执行计算所需要的线程调度功能，如图 2.13 所示。

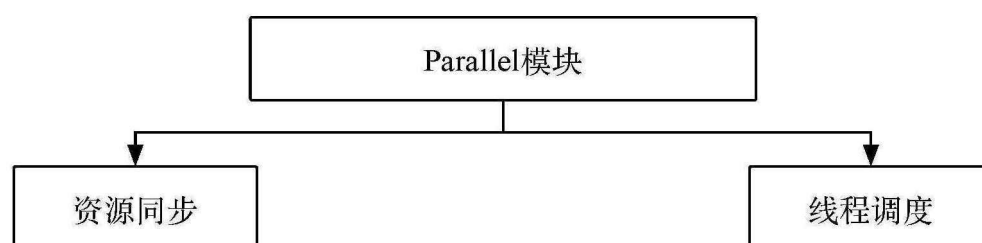


图 2.13 并行执行引擎模块 (**Parallel**) 功能示意图

资源同步功能是所有计算正确计算的关键。在这里，资源主要指内存中的 **Tensor** 变量。在每一个计算任务当中，对应的 **worker** 线程都需要读写资源；而读写资源的顺序需要得到严格保证，才能使得计算结果正确。因此，资源同步至关重要，这需要通过依赖分析和等待函数两个方法来完成。依赖分析使用依赖队列来处理资源在计算读写中的顺序读入和写出。框架内的其他模块通过 **Push()** 接口提交计算任务进入 **Parallel** 模块内时，依赖分析功能会自动启动，以保证传入 **Parallel** 模块的任务执行的顺序不会使得资源出现冲突。等待函数完成资源的同步，保证资源在该函数返回的时候已经满足依赖条件。

线程调度功能负责处理任务执行过程中的计算分配问题以及多个线程资源的管理。在分配计算任务时，被执行的任务必须满足上文所述的资源不冲突前提。而在线程调度功能中，**Parallel** 模块提供了不同的线程管理策略来保障线程的重复利用：一种是不区分设备的线程池设计，通过对 **CPU** 的线程调度来完成

所有设备的计算并行；另一种是分设备的线程池设计，对不同的设备，例如 CPU 和 GPU，分别使用单独的任务队列和线程管理，来保证更大程度的并行。

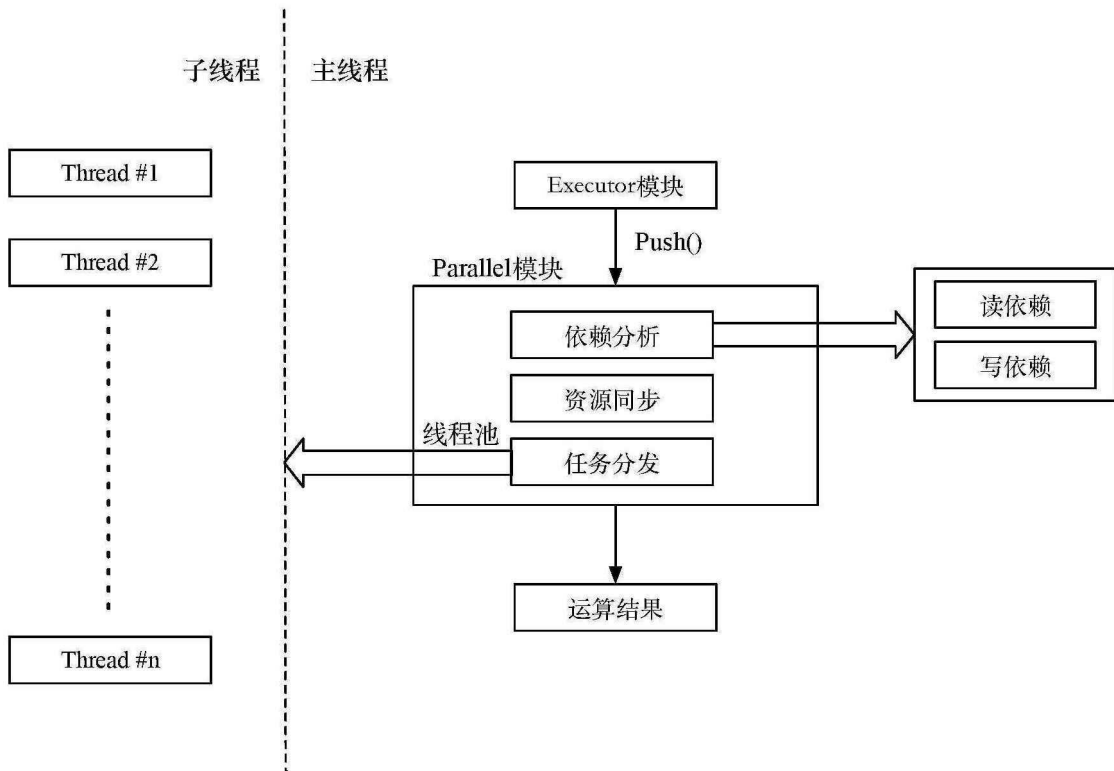


图 2.14 并行执行引擎模块 (Parallel) 工作流程图

如图 2.14 所示，在实际的工作流程中，Parallel 主线程接受并调度计算任务，子线程执行计算任务并在任务结束时通过回调函数通知主线程。接受计算任务时，主线程会根据当前计算任务的读对象（输入变量）和写对象（输出变量），来分别更新主线程内维护的读依赖队列和写依赖队列，然后将计算任务挂到任务队列当中。由于 Executor 向 Parallel 模块推送的计算任务是依据拓扑顺序进行的，所以 Parallel 模块只需检查任务队列首部的元素是否满足运行条件即可：资源依赖尚未得到满足时，Parallel 主线程便会对队列头的资源进行同步等候；资源就绪时，便使用线程池将任务分发至子线程。子线程完成计算任务时，通过回调函数通知主线程，主线程相应地更新读写依赖队列，并重新检查是否存在同时满足读写依赖的任务，以进行下一个计算任务的分发。在此过程中，所有对读依赖和写依赖的读取和更新都在主线程中完成，不会出现线程安全问题。最后的运算结果返回到预定的输出变量中，由 Optimizer 模块进行模型参数更新。

2.4 本章小结

本章介绍了一个面向 GPU 高效利用的深度学习框架 HelloDeep。首先，2.1 节介绍了国内外主流深度学习框架的发展情况，强调了本章介绍的 HelloDeep 框架与其他框架的不同；其次，2.2 节对 HelloDeep 框架的整体架构和总体工作流程进行了说明，列举了框架的 9 大模块以及它们之间的相互关系；随后，2.3 节中选取了比较重要的数据结构模块、内存管理模块、用户接口模块、计算图模块、执行器模块以及并行引擎模块，分别对其模块功能以及工作流程进行了展开说明。最后，本文将在本章之后的第三、第四章中，具体展示 HelloDeep 框架如何做到 GPU 的高效利用。

3 HelloDeep 高效 IO 设计

3.1 引言

在深度学习以及其他 HPC（高性能计算）系统中，IO 往往会成为整个系统的瓶颈。以图 3.1 为例，当直接使用 OpenCV 加载图像作为数据加载方式时，得到的 GPU 利用率随时间变化的曲线图。可以看到，只有少数时刻 GPU 利用率才会高于 75%，说明大部分时候 GPU 都在等候数据的加载，然后才能对应地进行运算。整体的 GPU 平均利用率只在 25%左右的水平，大大影响了深度神经网络的训练速度。

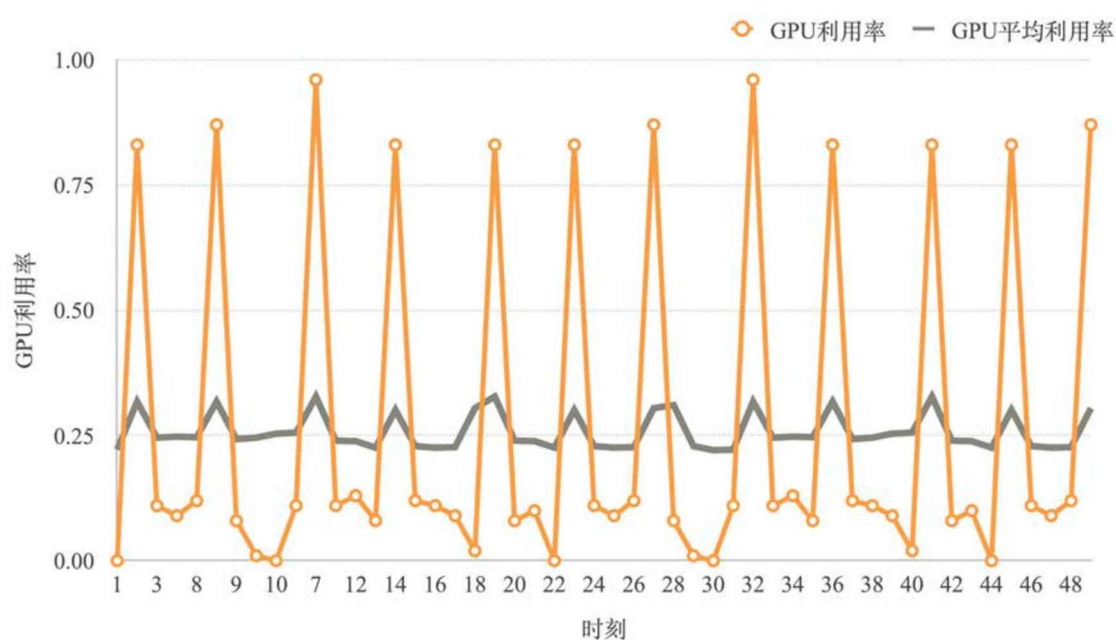


图 3.1 GPU 利用率-时刻曲线图（直接使用 OpenCV 加载数据）

为了缓解这一情形，研究者发展了各种技术来提升 IO 接口的性能。Thakur 等^{[41][42]}提出了利用消息传递接口（MPI）来进行并行 IO 以提升其性能，这是一个底层的、面向非结构化数据的 IO 接口。同时，一些高效的文件存储格式也被提出，例如 HDF5 以及 NetCDF^[43]等。通过将数据集转换为这些高效存储格式，以及配合相应的 API，就可以获得高效的数据输入/输出性能。关于 HDF5 的综述可以参见^[44]。为进一步提升 HDF5 文件读取的性能，Pumma 等^[45]提出将 HDF5 文件通过 Linux 的 `mmap()` 接口映射到内存中，以改善 Caffe 及其基于 HDF5 文件的 LMDB 接口在大规模数据集下的性能。

在本章中，先介绍 GPU 编程的最佳实践，然后介绍 HelloDeep 中的数据加载 IO 模块的设计——一个基于生产-消费者的多线程异步数据加载模型；3.5 节的实验将展现它如何将 GPU 利用率提升至足够高的水平。

3.2 GPU 数据传输、读写最佳实践

在正式介绍 IO 模块设计之前，本节先对 GPU 数据传输和读写的一些最佳实践^{[46][47][48][49]}进行说明，这些是提高 GPU 数据传输效率有效且容易被忽视的一些方法。

在 NVIDIA GPU 中，不论是核（CUDA kernel）计算还是数据传输，都通过 CUDA Stream 进行。CUDA API 支持 kernel 计算、CPU 到 GPU 数据传输、以及 GPU 到 CPU 数据传输分别使用不同的 stream，并且每个 stream 分别绑定到不同的线程上；与此同时，CUDA 也提供了对 stream 进行同步的接口。这些特点意味着可以将 GPU 的数据传输和 GPU 的计算重叠起来以提升效率。

将 GPU 计算和 GPU 数据传输重叠，实现起来也非常简单：在 kernel stream 执行计算时，CPU 到 GPU 的数据传输 stream 就可以开始将下一批数据传输到 GPU 中来，GPU 到 CPU 的传输 stream 也可以将上一批完成计算的结果返回到 CPU，这三者是并行不悖的。但是在这个过程中，有几点需要注意：首先，尽量将多个小的数据传输合并为一次大的数据传输，以最小化数据传输次数以及内存对齐带来的额外传输比特数；其次，GPU 不能直接从 CPU 可分页内存（Pageable Memory）中读取数据，必须经过不可分页内存（Pinned Memory）进行中转。因此，直接将数据存放在不可分页内存中，可以加快 CPU 和 GPU 之间的数据传输。GPU 直接读取 CPU Pinned Memory 的过程，也称内存直接访问（Direct Memory Access, DMA）技术^[52]。最后，受 GPU 和 CPU 之间连接的总线带宽的物理限制，使用超过一个以上的 Copy Stream 并不能提高数据传输的效率。

GPU 内部的内存分为全局内存和共享内存，全局内存位于 DRAM，而共享内存位于 GPU 芯片上（on chip）。DRAMs 的数据读取速度（纳秒级）相对于数据访问速度（亚纳秒级）是非常慢的，现代 DRAMs 通常会使用并行化来提高数据访问速度，即内存访问吞吐量（memory access throughput）。GPU 设备每次访

访问 DRAMs 中的一个位置时，实际上包含目标请求位置在内的一系列连续位置都会被访问到。由此，对于全局内存进行访问时，GPU 会进行内存的合并（Global Memory Coalescing）。

所谓的内存合并，实际上合并的不是内存本身，而是合并多个线程对内存的访问。当所有线程对访问全局内存的位置是连续的时，将实现最佳的内存访问模式。当一次任务中的所有线程执行 load 指令时，GPU 会检测它们访问的全局内存位置是否是连续的；如果是的话，硬件会将这些访问合并成一个对连续位置的访问。因此，一个线程在其生命周期内是否读取整行数据并不重要，重要的是，一次任务内的所有线程在每次内存访问时是否可以合并。这就是为什么深度学习框架要求输入数据在 CPU 向 GPU 传输的时候，将数据排布由 NHWC 变换为 NCHW。

对于 GPU 共享内存而言，其访问速度本身就高于全局内存；同时，共享内存不仅支持对连续内存的合并访问，还支持对跳跃内存（Stride Access）的合并访问，从而使得跳跃访问时不损失性能。这对于一些具备跳跃内存访问的计算，例如神经网络中 Stride 超参数不为 0 的卷积算子，有着很好的性能表现。

3.3 IO 模块设计

第二章介绍了 HelloDeep 框架的整体架构以及部分模块，而本章介绍的 IO 模块也是其中之一。在 HelloDeep 中，IO 模块负责从磁盘加载数据集，对其进行解码和预处理，然后为用户提供这些处理好的数据的迭代器，以便用户在顶层代码逻辑中将数据输入到神经网络中进行计算。

前文已经阐述了 IO 模块需要提供高效的数据加载性能来实现 GPU 的高效利用。除了性能之外，方便和灵活性也是需要考虑的因素。在一个深度学习任务中，对于输入数据的处理，是极为重要的第一环。不论是训练数据、验证数据还是推理数据，它们都需要同样的加载和处理流程；但是它们彼此之间也会存在流程上的区别，这就需要 HelloDeep 提供一个灵活、支持多种数据格式、用户自定义程度高的高性能数据加载模块。

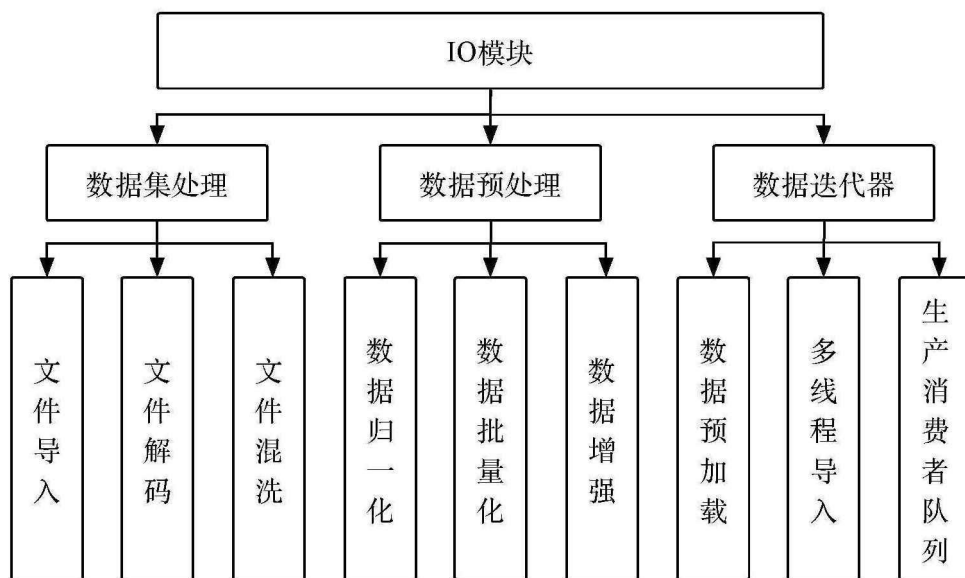


图 3.2 数据加载模块 (IO) 功能示意图

根据高效性和灵活性的要求，IO 模块提供如图 3.2 所示的功能：

- 1) **数据集处理**：IO 模块提供对各类数据集的导入功能，包括未处理的图片数据集、视频数据集，以及经过处理的 XML 数据集，HDF5 数据集等。在导入过程中，可以根据用户给出的参数决定是否进行混洗 (Shuffle) 操作，以便在多个训练 Epoch 中更改训练数据的顺序。
- 2) **数据预处理**：对于加载到内存中并存放到 Tensor 对象中的数据，在进入神经网络之前通常还需要一系列的预处理。比较常见的处理是将数据进行归一化，将数据的值映射到[0,1]区间内，并且减去各个通道的均值；同时，由于内存大小的限制，我们不可能在一次前向和反向计算中对全量数据集进行计算，所以通常需要进行 Mini-batch 计算，即每次对 BatchSize 个单位的数据进行计算。Mini-batch 同时提升了计算效率和神经网络的收敛性，IO 模块为此提供了将数据划分为 BatchData 的功能。最后，对于神经网络训练而言，数据增强是很重要的扩展数据、增加模型精确度和泛化能力的手段。IO 模块提供归一化、批量化以及多样的数据增强功能，包括随机剪裁、翻转等，提升了模型的健壮性。
- 3) **数据迭代器**：迭代器是用户操作每个 mini-batch 数据的“把手”。需要强调的是，数据集中所有数据过大，不可能全部存放在 GPU 或者 CPU 内存中，所以迭代器每次进行一个 mini-batch 数据的加载，都需要重新从文件系统中执行数据的加载、解码、预处理的完整流程。因此，迭代器的设计极大影响

了 IO 模块的性能。HelloDeep 的 IO 模块通过三个设计来保障迭代器的高效运转：数据预加载、多线程导入和生产消费者队列。在正式开始训练之前，预先加载若干个 Batch 的数据来填充生产-消费者队列；由于神经网络的运算是异步的，所以 IO 可以在主线程中随时监控队列中每个 Batch 的信息，一旦某个 Batch 被消费完成，IO 马上启动一次数据加载来重新填满队列；在这个过程中，一个 Batch 内部的多个数据单元是多线程并行加载的。通过这个设计，使得数据加载和神经网络计算之间不会互相阻塞，极大提升整体效率。

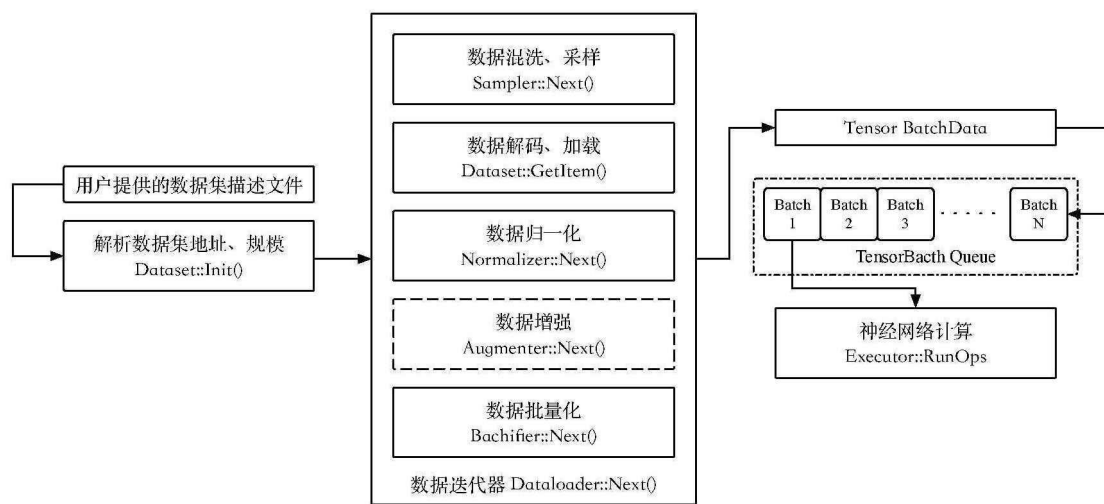


图 3.3 数据加载模块 (IO) 工作流程图

IO 模块的工作流程如图 3.3 所示。首先，用户在顶层提供数据集描述文件，包括路径、是否乱序等，然后由 IO 模块的 Dataset 对象负责进行解析；随后，根据采样顺序（顺序或者随机），由 Sampler 对象生成读取序号；然后由 Dataset 的 GetItem() 方法实现目标序号的数据加载。当解析过的数据加载到内存后，Normalizer 对象会对数据进行归一化，以及可选的 Augmenter 进行的数据增强；最后，由 Batchifier 实现给定 BatchSize 的数据划分，这就是数据迭代器 Dataloader 内部进行一次数据加载所执行的所有流程。

Dataloader::Next() 方法会返回一个 BatchData 对象，这是对 BatchSize 个 Tensor 的封装，称一个 Batch。当用户显式地调用 Dataloader::Next() 方法时，Dataloader 会持续加载数据，直至 BatchData 把生产-消费者序列填满为止，这个过程是异步进行的。与此同时，在消费者一端，即执行器和并行引擎一侧，则会从队列中获取一个 Batch 的数据来进行计算，计算结束后会把完成计算的 Batch 放到空闲列表中，等候 Dataloader 的重新装填。有关生产者-消费者模型的更多细节，将在下

一节模块实现部分进行详述。

本节介绍了高效 IO 模块的设计。在实际运行过程中,除了如本节所述的工作流程之外,还在中间应用了 3.2 节所述的 GPU 数据传输、访问的技巧。例如,通过 CUDA Stream 将计算和 IO 并行化,通过在数据加载过程中调整维度顺序为 NCHW 以加快数据在 GPU DRAMs 上的访问速度,等等。除此之外,在 CPU 侧数据的读取过程中,也应用了 OpenMP,利用硬件支持的最高并发数来同时进行多张图片的读取。

3.4 IO 模块实现

本节通过 UML 类图的形式对 IO 模块内主要的类进行介绍。为了方便灵活性和扩展性的需要,本研究采用了面向对象的设计方式和装饰器设计模式;同时,每个类的私有成员中都存在一个 param 结构体,用于存放各类可以由用户指定的超参数,并控制类内部的逻辑,例如数据集 URL,归一化均值文件 URL 等。

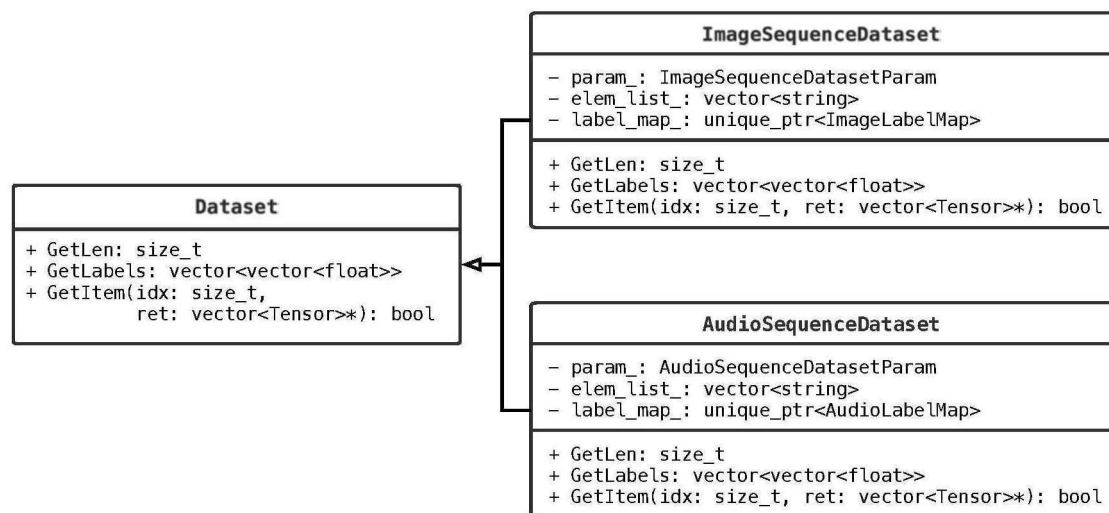


图 3.4 数据加载模块 Dataset 类 UML 图

图 3.4 描述了 Dataset 类的实现。Dataset 作为基类,规定了所有的子类应该具备的接口,包括给出数据集总元素个数的 `GetLen()`, 标签获取接口 `GetLabels()` 以及获取单个元素的接口 `GetItem()`。以此为基类,可以轻松地扩展出处理不同类型数据的子类。以负责处理普通的图片数据集的 **ImageSequenceDataset** 类为例,其 `GetItem()` 的实现中就采取了使用 OpenCV 库来进行图片数据解码的方式。需要强调的是,对于 3.1 中介绍的 HDF5 和 NetCDF 数据,也可以方便地集成到本模块

中：只需要继承并创建相应的 Dataset 类，改写 `GetItem()` 接口为相应的数据加载接口即可。

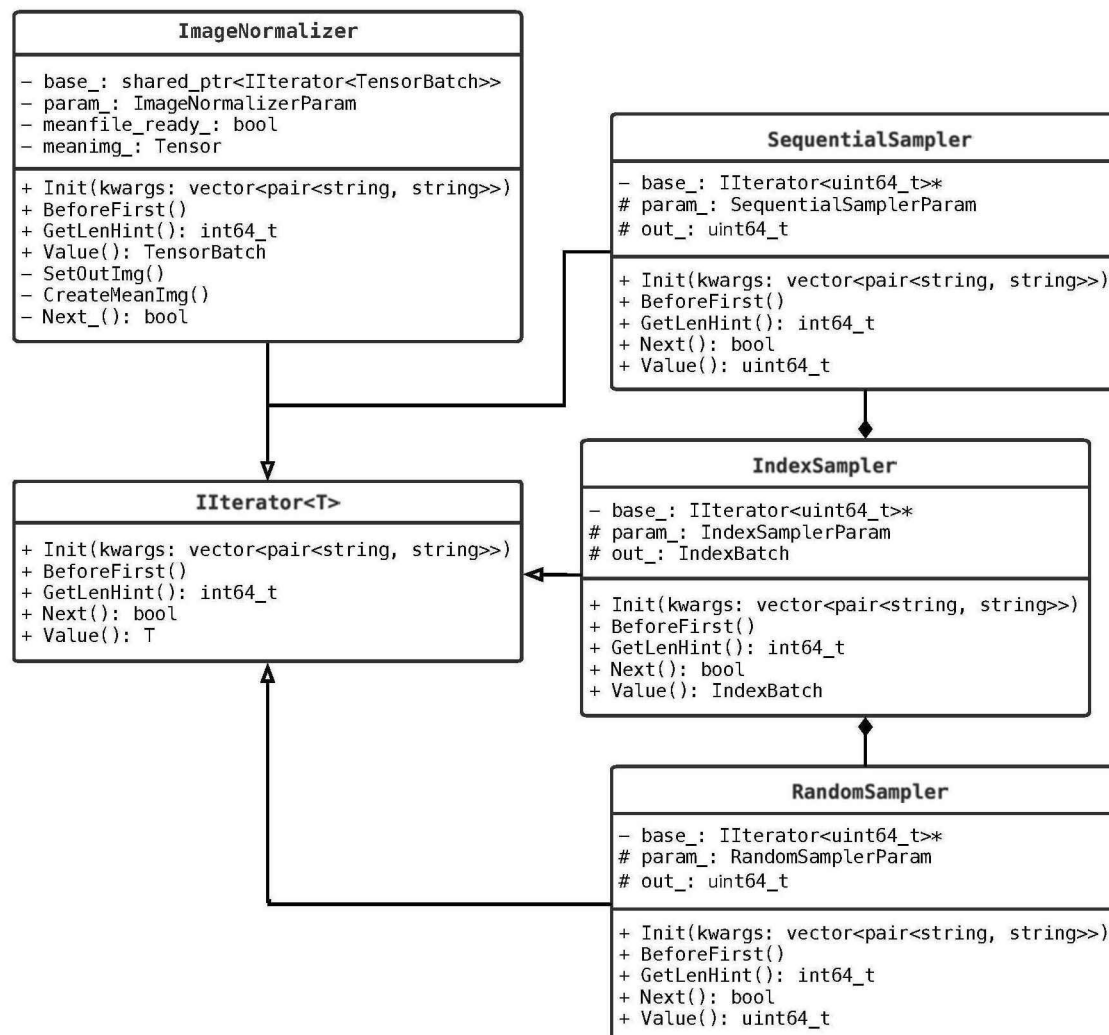


图 3.5 数据加载模块 Normalizer 类及 Sampler 类 UML 图

图 3.5 描述了归一化 Normalizer 类，和采样器 Sampler 类的实现。注意左侧的 `IIterator<T>` 这一类模版，Normalizer、Sampler、Dataloader 都继承自该模版，它规定了最重要的几个必须实现的方法：初始化 `Init()` 用于从参数结构体中解析超参数，`BeforeFirst()` 用于返回首前位置，`Next()` 将迭代器往后移一个位置并返回是否成功的指示，`Value()` 用于返回迭代器指向的元素。

`IndexSampler` 继承 `IIterator` 模版，模版参数特化为无符号 64 位整型数，管理数据集中所有元素的下标，在 `Next()` 方法中返回下一个元素的下标值；其内部有两种实现，分别是顺序下标和随机下标。`ImageNormalizer` 类继承 `IIterator` 模版，模版参数特化为 `TensorBatch`，这是迭代器给用户返回的，包含 `batchsize` 个 `Tensor`

的数据结构。ImageNormalizer 在 Next()方法中对当前的 TensorBatch 应用数据归一化并将其返回。

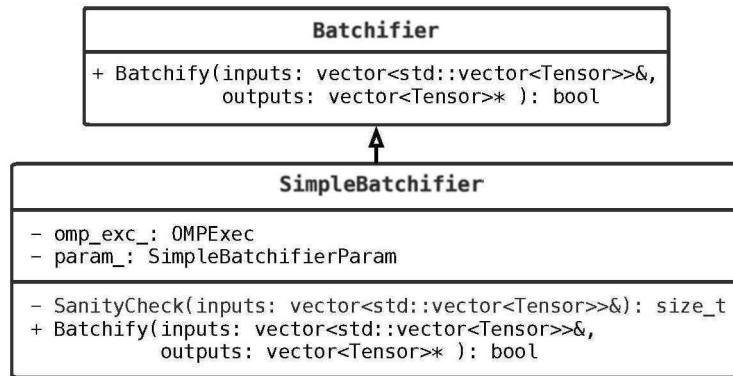


图 3.6 数据加载模块 Batchifier 类 UML 图

图 3.6 描述了 Batchfier 类的实现。Batchfier 类的主要函数是 Batchify(), 该函数先从 Sampler 对象中获取 batchsize 个元素的下标, 随后利用 OpenMP 多线程执行器 OMPExec, 让多个线程同时执行 Dataset 对象的 GetItem()方法, 并行地加载指定数量、指定下标的数据元素。

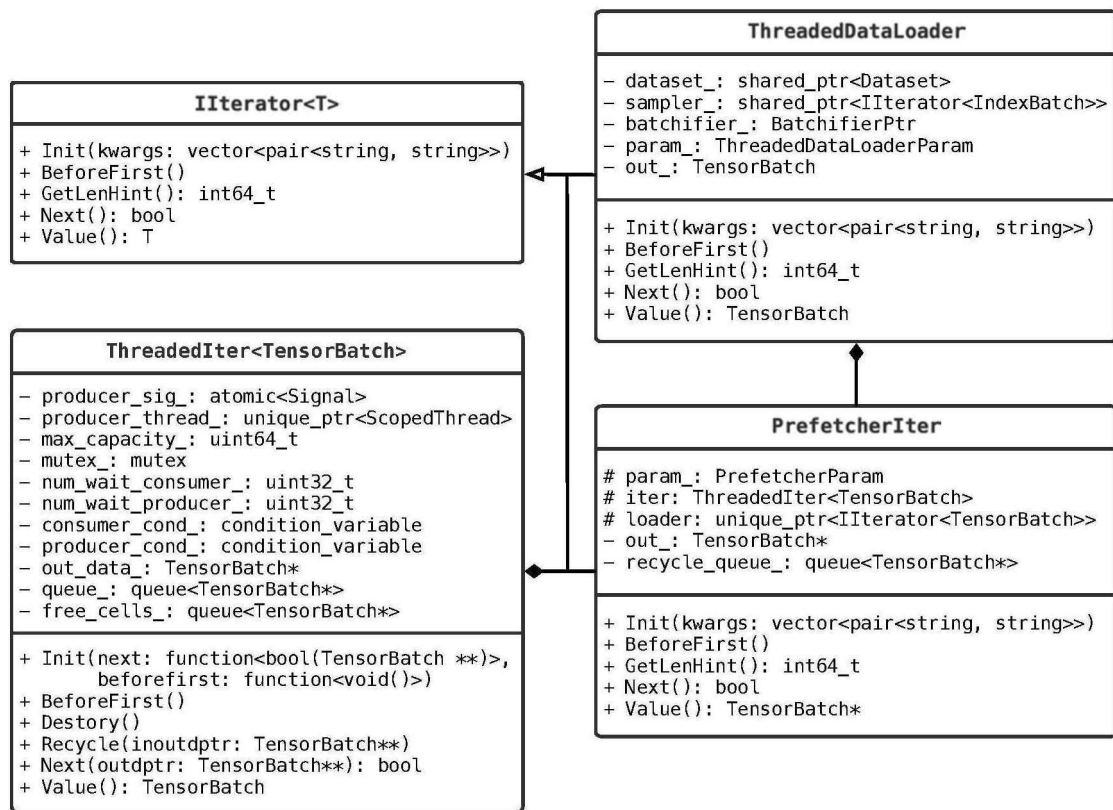


图 3.7 数据加载模块 DataLoader 类 UML 图

图 3.7 描述了 DataLoader 类的实现。同样地, 图中三个类都继承自 IIterator<T>, 这使得 IO 模块中可以使用装饰器设计模式: 指向基类 IIterator<TensorBatch>的

指针，同样也可以指向 `Normalizer`、`Sampler`、`Dataloader`，以及可选的 `Augmenter` 等对象，它们也都拥有迭代器的功能，可以调用 `Next()` 方法和 `Value()` 方法获取并迭代数据。这使得用户定义数据加载和预处理流程时十分地方便灵活。有关装饰器模式在 C++ 中的高效实现，可以参考 Niculescu 等人的工作^[53]。

回到 `DataLoader` 类本身，`PrefetcherIter` 类是提供给用户的迭代器对象，其内部包含了 `ThreadedDataLoader` 和 `ThreadedIter` 成员。`ThreadedDataLoader` 内部包含了指向 `Dataset` 对象、`Sampler` 对象以及 `Batchifier` 对象的指针，负责完成数据从加载到打包成 `TensorBatch` 的过程；`ThreadedIter` 对象则完成生产者-消费者模型的运行。

下面简单描述 `ThreadedIter` 的生产者-消费者模型的运行细节。可以看到，`ThreadedIter` 类的 `Init()` 方法接受两个函数对象作为参数，在运行时分别是 `ThreadedDataLoader::Next()` 和 `ThreadedDataLoader::BeforeFirst()`，`ThreadedIter` 会利用这些方法完成数据 `Batch` 的加载。在进程运行期间，`queue_` 和 `free_cells_` 内的元素之和总保持在 `max_capacity_` 的水平，默认为 16。本模块中采用了单生产者、单消费者的模式，`producer` 在生产过程中会优先选取 `free_cells_` 中的元素，利用 `ThreadedDataLoader::Next()` 方法加载一个 `Batch` 的数据，然后放入 `queue_` 中；消费者从 `queue_` 中取出一个 `Batch` 的数据，完成计算后通过 `Recycle()` 将 `TensorBatch` 对象放回至 `free_cells_` 中进行重新利用。

为了保证生产者和消费者在运行时读写 `queue_` 和 `free_cells_` 的线程安全，需要对二者进行同步。对于 `consumer` 而言，每次 `Next()` 方法被调用时都会主动进行阻塞，结束等待的条件是 `queue_` 非空或者 `producer` 已经停止工作；当以下事件发生时，`consumer` 会被唤醒来检查是否符合结束等待的条件：1. `producer` 完成了一次数据加载，且 `num_wait_consumer` 值不为 0；2. `Destory()` 方法被调用。

对于 `producer` 而言，其默认处于阻塞状态，结束等待的条件是 `queue_` 和 `free_cells_` 内元素之和未达到上限；当 `BeforeFirst()`、`Next()` 或者 `Destory()` 方法被调用时，若 `num_wait_producer` 值不为 0，`producer` 会被唤醒，检查是否满足结束等待的条件，若满足，则进行 `TensorBatch` 的加载。特别地，当整体的深度学习进程初启动时，`PreFretcherIter` 对象会进行数据的预读取，即反复唤醒 `producer` 成员进行数据加载，直至 `queue_` 的元素被填满为止。

3.5 实验结果分析

本节将在 HelloDeep 框架中验证本章提出的高效 IO 的有效性。

本文所有实验，包括第三章和第四章中介绍的所有实验，都基于以下软硬件环境进行：

表 3-1 实验主要软硬件环境列表

CPU	Intel(R) Xeon(R) CPU E5-1620 v4 @ 3.50GHz
CPU 内存	128GB
GPU	NVIDIA GeForce RTX™ 2070 @ 1620 MHz
GPU 内存	8 GB GDDR6
操作系统版本	Ubuntu 18.04
编译器版本	gcc/g++ version 7.5.0, nvcc version 10.2
CUDA 版本	Cuda compilation tools, release 10.2, V10.2.89
Cudnn 版本	7.6.4
C++标准版本	C++17
Python 版本	3.8.5
TensorFlow 版本	2.3.0
Pytorch 版本	1.1.0
MXNet 版本	2.0.0

3.1 节中展现了不使用 IO 模块，直接调用 OpenCV 接口时的情况，GPU 占用率平均在 25%左右，GPU 的大部分时间在等待数据加载。现在，我们对于同样的数据和模型，将数据加载方式变更为基于 HelloDeep IO 模块的 Dataset 接口，底层同样利用 OpenCV 库，来进行网络训练，得到的 GPU 利用率如图 3.8 所示。从中可以看到，整体的 GPU 利用率保持着 96%左右的水平，这显示着 IO 加载的速度高于 GPU 计算的速度，使得 GPU 总能处于有数据可用的状态，内部的计算就不会因为等待数据传输而阻塞。

本节展示了 HelloDeep 的 IO 模块对于图片数据集训练情况下的 GPU 利用率。对于图片之外的数据集，只需要配置相应的 Dataset 接口，也同样可以使得 GPU 利用率保持在较高水平。如果将来遇到数据加载慢于网络计算的情况，那么可以考虑将 raw 数据集转换为更高效的格式，包括 HDF5 等，或者将 CPU-GPU 的连接方式升级为更高速的 PCIE 5.0^[50]或 NVLink^[51]，就可以进一步改善 IO 的瓶颈情况。

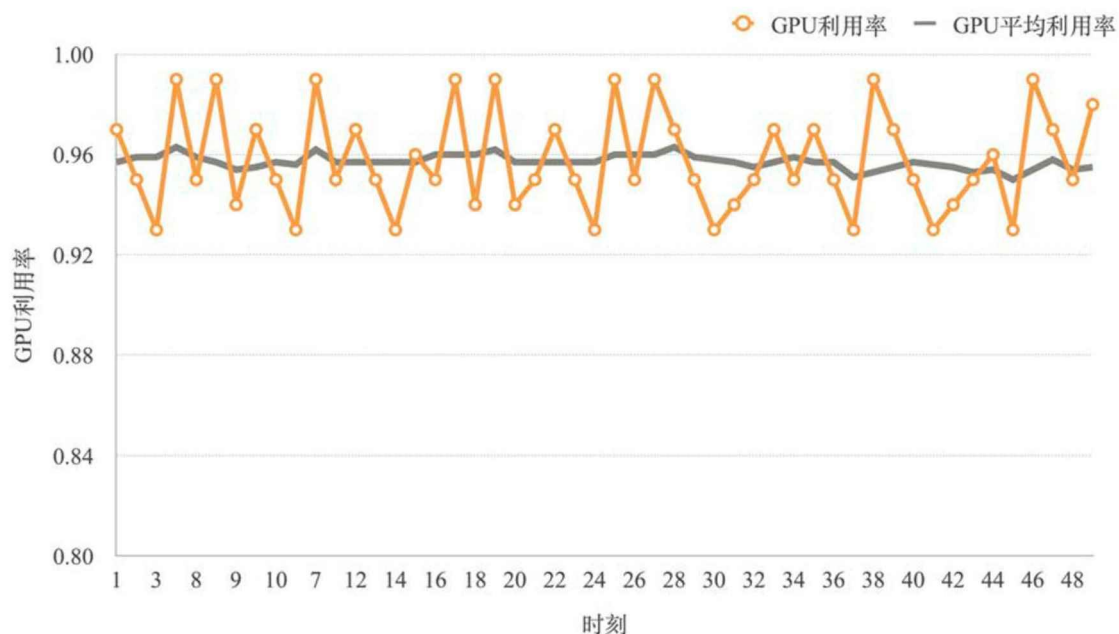


图 3.8 GPU 利用率-时刻曲线图（使用 HelloDeep-IO 加载数据）

3.6 本章小结

本章介绍了 HelloDeep 高效 IO 模块的设计及其实现。HelloDeep 的 IO 模块采用了生产者-消费者的编程模型，利用多线程极大增加了数据读取的速度，通过预读取策略提前加载数据。只要 GPU 进行一次前向、反向计算的速度不超过 IO 模块一个 Batch 的读取速度，即生产者的生产速度不低于消费者的消费速度，那么生产者-消费者队列的可用元素就总是能保持在足够的数量，从而保证 GPU 计算不会因为数据 IO 而堵塞，使得 GPU 利用率始终保持在较高水平。

在模块的内部实现中，本研究采用了面向对象的思想 and 装饰器设计模式。得益于此，本研究的 IO 模块设计中可以将 3.1 节介绍的 HDF5 或者 NetCDF 结合进来，进一步提高 IO 的效率。可以看到，本章介绍的 IO 模块，不仅自身具有优秀的性能，同时还具备了良好的扩展性和灵活性，可以和前人的工作很好地结合。

实验表明，HelloDeep 通过多线程的异步生产者-消费者模型，提高了 IO 模块的加载效率，使得 GPU 占用率可以维持在 90% 以上；同时在 Dataset 内保留了方便的扩展能力，在未来面对更大型数据集时可以扩展使用更高效的数据集格式和加载方法。

4 HelloDeep GPU 内存优化方法

4.1 引言

GPU 高效利用包括两个维度，第三章介绍了如何让 GPU 利用率保持在较高水平，本章则介绍 HelloDeep 中降低 GPU 内存占用的方法。关于如何降低 GPU 内存占用，1.2 节中已经介绍了国内外研究者的一些做法。本文的做法在部分参考前人工作之余，也改进了一些在前人工作中被忽视的点。

本章的内容按照如下方式组织：首先，在 4.2 节中介绍 HelloDeep GPU 内存优化的总体方法，包括基础-通常-极致三个优化级别；其次，在 4.3 节中介绍模型试运行和重计算 checkpoint 候选点选取的具体过程，并说明这一步骤的必要性；随后，在 4.4 节中介绍一个启发式搜索算法，该算法旨在寻找一个最佳的变量重计算和内存交换方案；4.5 节则介绍了基于静态计算图的内存申请方案，该方案将为每个 Tensor 分配一个内存池中的内存块 id；4.6 则介绍神经网络计算中内存占比较大但却经常被忽略的 Cudnn 辅助空间，以及如何对其进行限制。在 4.4 至 4.6 节中介绍的 GPU 内存优化方法，均在每一节的最后一小节中进行了实验验证，以基于 HelloDeep 框架的实验来说明各个方法的有效性。

4.7 节展示了 HelloDeep 3 个不同级别的 GPU 内存优化方案和主流深度学习框架 TensorFlow、Pytorch、MXNet 以及基于 MXNet 实现的 Memonger，在经典 CNN 网络上的对比实验。通过观察实验中不同 GPU 内存优化方案的表现，来评估本研究提出的 GPU 内存优化方法的有效性。

最后，4.8 节对全章内容进行总结。

4.2 总体方法

图 4.1 描述了 HelloDeep GPU 内存优化的总体方法：图左侧描述了整体的执行步骤，右侧则是三个优化级别下各个步骤的不同配置。在本章中约定：CPU 内存，指 CPU 北桥芯片通过总线连接的高速存储器；GPU 内存，指与 CPU 内存相区分的、专属于 GPU 的存储器；checkpoint，指重计算策略中被保留在内存中的 Tensor，在反向传播时从这些 Tensor 重新计算出其余 Tensor。可以看到，GPU 内

存优化整体分五个步骤：

- 1) **试运行**：该步骤对应图 4.1 中的第一步，初次训练时启动试运行，收集设备环境信息与神经网络逐层运行情况，为后续算法决策提供必要信息。具体而言，在试运行过程中，需要收集计算图中每个节点的算子类型、内存占用量、计算时长、辅助空间大小，以及 CPU 和 GPU 之间拷贝操作的执行时长和吞吐量；
- 2) **checkpoint 候选点选择**：该步骤对应图 4.1 中的第二步，根据 1) 的信息，以特定方式选择合适的节点作为重计算 checkpoint 的候选点。在评估一个节点是否适合作为重计算 checkpoint 时，主要考虑其存活时长、计算耗时、特征图大小；评估一个节点是否适合被交换到 CPU 时，主要考虑依赖该节点的两级运算之间是否有足够长的时间间隔，以覆盖 CPU-GPU 之间的拷贝开销；
- 3) **启发式内存申请方案搜索**：该步骤对应图 4.1 中的第三步，在 2) 给出的候选点中选择合适的若干节点作为 checkpoints，利用 4) 描述的算法计算出内存申请方案，该步骤将执行多次以获取最优方案。在此过程中，算法先随机选取若干候选点作为 checkpoint，其余所有节点都被标记为可丢弃；可丢弃的节点在计算完成后不会被保存在内存中，而是直接丢弃；这些节点在反向传播过程中再次由驻留在内存中的 checkpoint 重新计算；此处将由 checkpoint i 重新计算的部分节点称作 Block i ，那么启发式内存申请方案搜索的目标就是：

$$\operatorname{argmin}(\max(\operatorname{Mem}_{\text{Block } i})) \text{ for } i \in [0, n]$$

其中， n 为 checkpoints 的数量。即在多个 checkpoints 选取方案中，找出使得步骤 4) 返回最小内存占用大小的方案；

- 4) **基于静态计算图变量生命周期分析的内存池方案**：该步骤对应图 4.1 中的第四步，根据不同的优化级别，给出每个节点的内存申请方案和整体内存使用情况。对于所有级别的优化，内存池都会启用变量生命周期及其依赖的分析；对于通常和极致级别的优化，启用节点丢弃优化，对非 checkpoint 的节点进行回收和重计算；对于极致级别的优化，启用 checkpoints 自动交换优化，将暂时用不到的 checkpoint 节点交换到 CPU 内存中；
- 5) **Cudnn 辅助空间限制**：该步骤对应图 4.1 中的第五步，在 4) 的基础上，考察 1) 实际运行时 GPU 运算库所占用的额外空间，给出进一步 Cudnn 内存限制方

案。本研究根据每个卷积层输出特征图的大小来决定 Cudnn 卷积算法的额外空间限制：对于基础级别优化，不加限制；对于通常级别的优化，额外空间限制为输出特征图大小的 20 倍；对于极致级别的优化，额外空间限制为输出特征图大小的 0.05 倍。

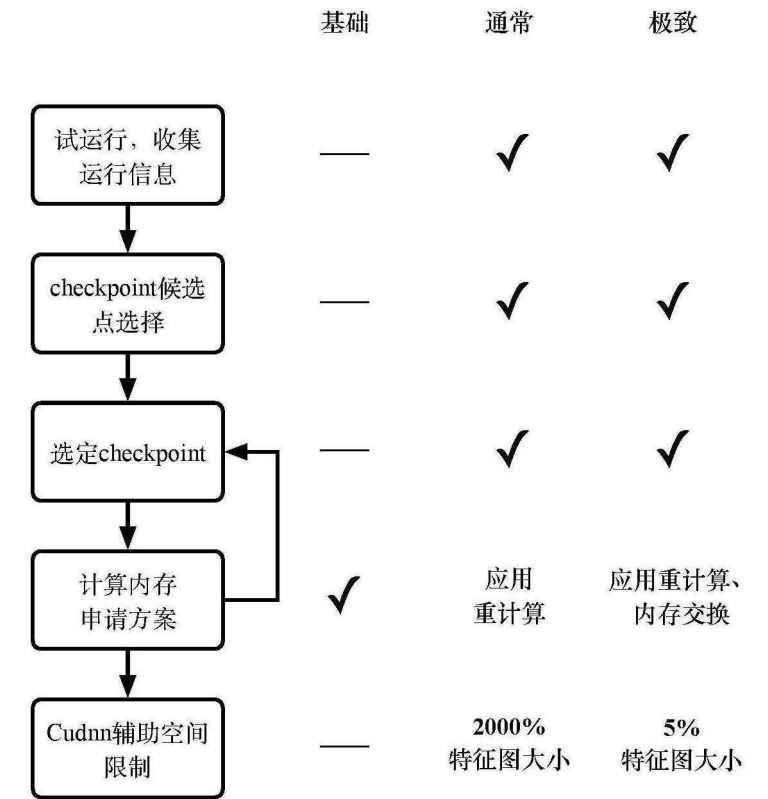


图 4.1 HelloDeep GPU 内存优化总体方法示意图

将 GPU 内存优化等级分为 3 个等级，主要原因是在前人工作中缺乏这样一个可供选择的特性：要么就不进行优化，要么就只能应用所有的优化方法，缺乏灵活性。图 4.1 中描述的各个方法，除了第四步之外，都需要付出额外的时间花销，以换取 GPU 内存的节省。本章介绍的方法给用户提供了 3 个级别的优化：基础级别优化只启用了最基础的，利用变量生命周期不重叠的内存共享，几乎不需要付出额外时间；在通常级别优化中，应用了变量重计算策略和较为宽松的 Cudnn 辅助空间限制；在极致级别的优化中，在前者的基础上加入了对 checkpoint 变量的内存交换，以及更严格的 Cudnn 辅助空间限制，进一步压缩内存的使用，同时也进一步增加了额外时间开销。用户可以结合硬件资源的实际情况，采用不同的 GPU 内存节省策略。有关图 4.1 中各项策略方法的具体效果将在相应章节的实验过程中进行介绍。

4.3 模型试运行和重计算 checkpoint 候选点选取

在 Memonger^[14]和 SuperNeurons^[26]中介绍的变量重计算方法,在对 checkpoint 的选取上都采用了固定的策略,即选取所有的 Convolution 层、FullyConnected 层、Concat 层以及 Embedding 层作为候选点。这样的做法存在两个不足:其一,这样的选取方法在颗粒度上过于大,并且不够灵活;其二,依靠先验知识来完成。在 HelloDeep 中,将 checkpoint 候选点的粒度细化到 layer 级别,同时依赖模型试运行来收集模型中所有 layer 计算情况以决定最终的候选点,避免对先验知识的依赖。

在试运行过程中，本研究主要采用 C++ 的 `chrono` 库，以及 NVIDIA Nsight System 工具^[54]来收集 Tensor 级别的运行情况。Nsight System 是 NVIDIA 推出的新一代 CUDA 可视化性能分析工具，具有比上一代工具 `nvprof` 更精确更全面，同时开销更小的优势。有关如何在 CPU 和 GPU 上进行性能分析的更多方法，可以参考 Yang 的工作^[55]。

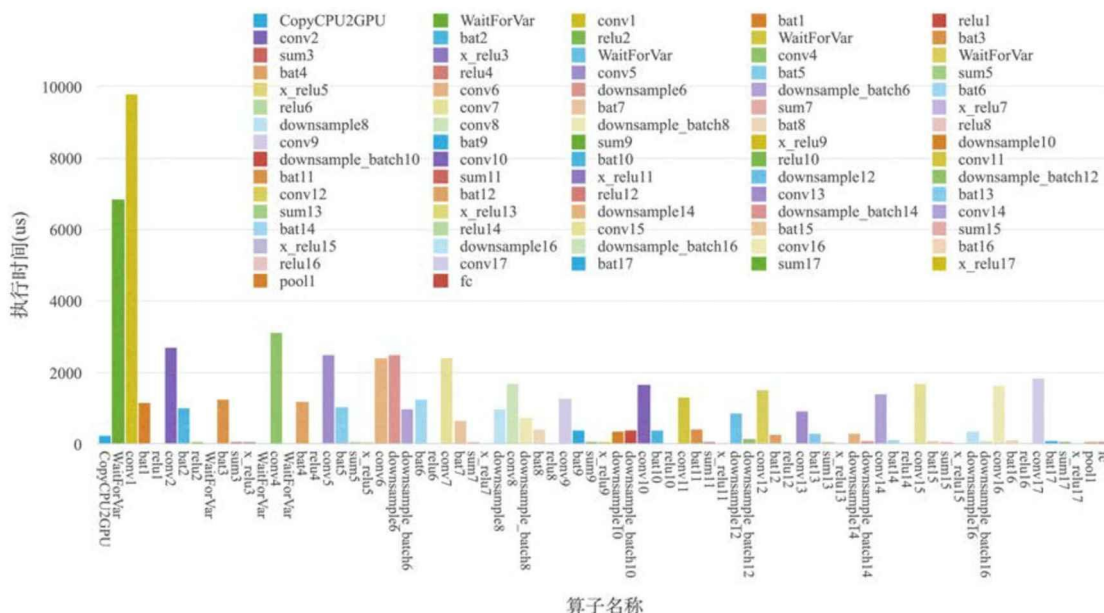


图 4.2 ResNet-18 逐层运行时间柱状图

为说明试运行收集到的信息，以图 4.2 所示的柱状图进行说明。在 HelloDeep 框架中对 ResNet-18 模型执行一次试运行，输入图像尺寸为 224*224，通道数为 1，BatchSize 为 4，得到其逐层运行时间。图 4.2 说明了在试运行过程中，前向推理各层消耗的时间（微秒）。可以看到，耗时较长的层都是各个卷积层，包括残

差模块中的 `downsample` 层。例外的是紧跟在 `CopyCPU2GPU` 算子之后的 `WaitForVar` 操作，耗时同样超过了 6000 us，这代表了将 $4*1*224*224$ 的 Float 型输入数据从 CPU 拷贝到 GPU 所需的通信时间，由此可以计算出 CPU 和 GPU 之间的通信带宽。

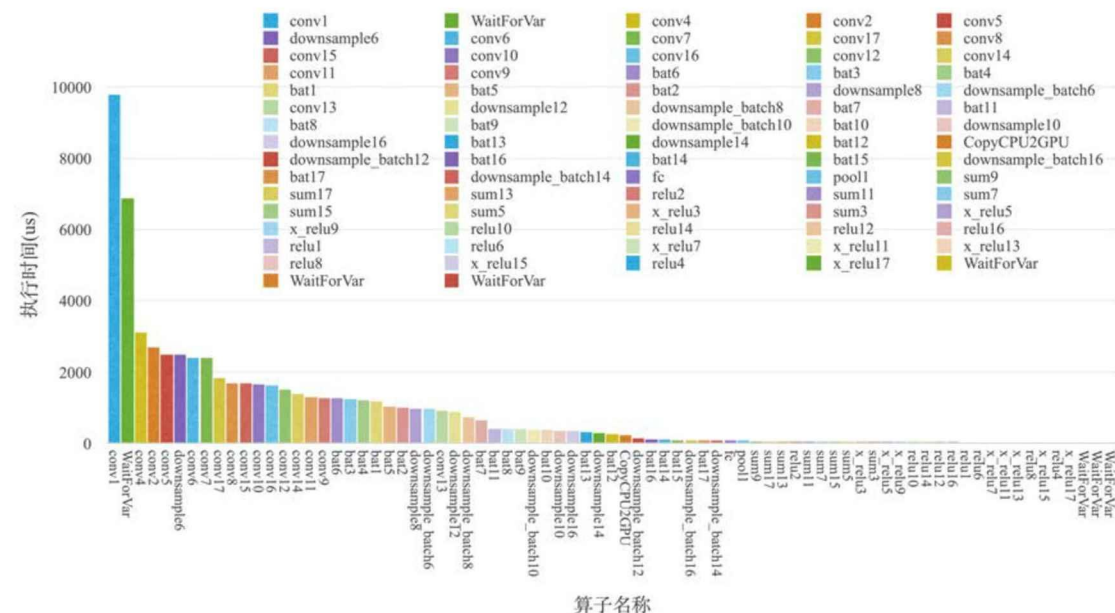


图 4.3 ResNet-18 逐层运行时间柱状图（降序排列）

将 ResNet-18 逐层运行时间进行降序排列，可以更清晰地看到计算用时较长的 layer，这些层更适合作为重计算的 checkpoint 而被保留；而靠近尾部的 layer，例如 ReLU 层，计算耗时较短，更适合被舍弃掉，在需要的时候再进行重新计算。注意到第二高的柱属于 CPU-GPU 数据传输时间，除以特征图大小之后可以得到传输速率，这将作为 Tensor 内存交换的阈值参考。

HelloDeep 通过试运行收集了若干 profile 信息，这主要是为了后续三个方面的工作：其一，收集 Tensor 的运行信息，以决定哪些 Tensor 可以作为 checkpoint 的候选点；其二，收集 CPU-GPU 数据传输的带宽和速率信息，以帮助 Tensor 在内存交换和计算之间做好 overlap；其三，收集 Cudnn 不同卷积算法之间占用额外内存空间的信息，为 Cudnn 辅助空间限制做准备。

现在具体介绍如何利用上文收集到的 profile 信息来进行 checkpoint 候选点 Tensor 的选取：首先，过滤掉所有输入变量和参数变量，包括 weights, bias 等，以及生命周期较短的临时变量。其次，选取的 Tensor 主要考虑三方面因素：第一，

考虑该 Tensor 计算时长 t_{com} 与该 Tensor 大小 S_{fea} 的比值, 该比值越大, 说明重新计算的代价越大, 越应该被考虑作为 checkpoint; 第二, 考虑该 Tensor 前后两次依赖之间的时间间隔, 通常是该 Tensor 在前向计算和反向计算时的间隔, 该间隔越长, 越适合被交换到 CPU 内存; 第三, 考虑该 Tensor 自身的大小 S_{fea} , 该值越大, 说明该 Tensor 越有可能是整体训练过程中的内存瓶颈, 越值得被交换到 CPU 内存。考虑到只有极致级别的优化才会启用内存交换, 所以前文所述三项数据归一化得分的权值之比为 10:1:1, 以此得到所有 Tensor 的分数排名, 从中选取分数最高的适量 Tensor 作为 checkpoint 的候选点。再次回到 ResNet-18 的例子, 在这个网络中, 我们选取了所有的卷积层作为 checkpoints 的候选点。

需要再次强调, 虽然本研究在 ResNet-18 选取了卷积层作为候选点, 与 Memonger^[14]和 SuperNeurons^[26]的选择一致, 但是这是经过对 Layer 级别的性能测试而得出的结果, 而非依赖固定的先验知识。在以 Transformer 为代表的新的网络结构不断推陈出新之际, 采用固定策略的内存优化方法会迅速变得不再适用, 但 HelloDeep 仍可以利用试运行方法来给出合适的 checkpoint 候选点方案。

4.4 启发式重计算和内存交换方案搜索

4.4.1 方法描述

Tensor 重计算的思想, 主要是基于这样一个发现: 在神经网络训练过程中, 反向计算依赖前向计算的结果来计算梯度, 所以在前向传播过程中所有 Tensor 都需要保留在内存中以便在反向计算中被使用; 如果可以舍弃一部分前向结果, 那么就可以有效降低整体训练 GPU 内存占用的峰值。为了取得内存节省和计算效率的平衡, 我们不能舍弃所有结果, 而需要每隔一定层数设置一个 Tensor 作为重计算的 checkpoint, 意为检查点, 两个检查点之间的部分, 称一个 Block; 在反向传播过程中, 每当需要已经被释放的 Tensor 作为计算依赖时, 就从该 Tensor 所在的 Block 所对应的检查点开始, 重新进行整个 Block 的前向计算, 以满足反向传播的需要。

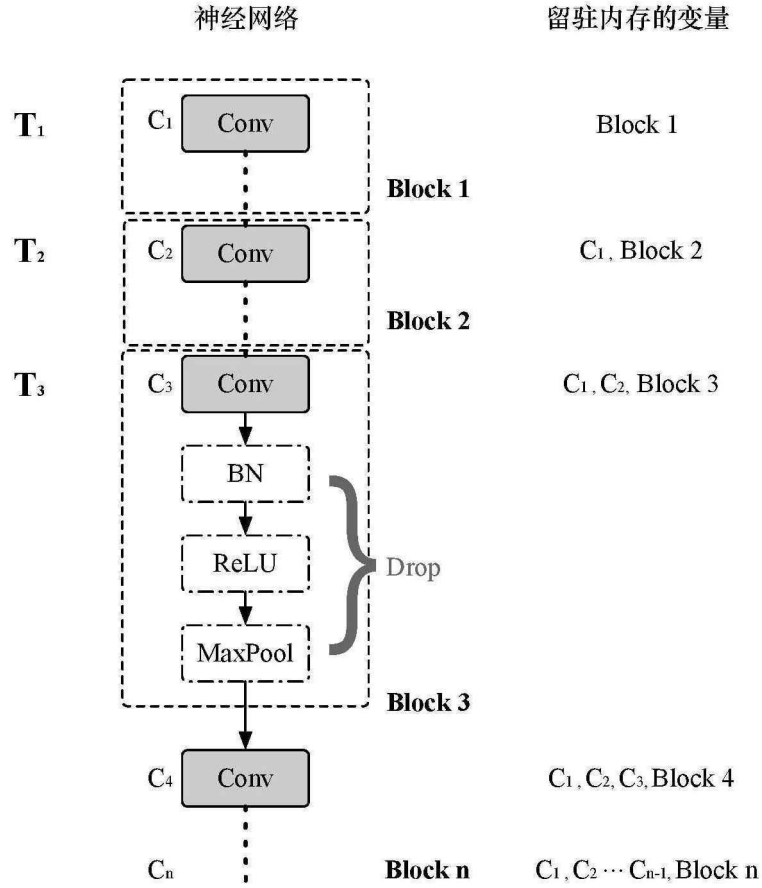


图 4.4 神经网络重训练 Checkpoint 示意图

图 4.4 描述了应用 Tensor 重计算策略后,神经网络变量在内存中的驻留情况。在重训练逻辑中,完整的神经网络被分为若干 Block,Block 中的首层是 checkpoint,其后是可以被丢弃的 Tensor。以 Block 3 为例,Conv 层作为 checkpoint,后续的 BN 层、ReLU 层、MaxPool 层均可在结束运算后回收内存;在反向传播时,再从 Conv 层重新计算出后续 3 层数据以计算梯度。根据定义,对于某个时刻 T_i ,内存中留驻的 Tensor 包括当前 Block 的所有 Tensor 以及该 Block 之前所有 Checkpoint Tensor。此时内存占用量为:

$$MemAlloc_{T_i} = MemAlloc_{Block_i} + \sum_{j=0}^{i-1} MemAlloc_{ckpt_j}, i \in [1, n] \quad (1)$$

本节所介绍方法的目的是,就是找到一个 Checkpoint 的组合,使得所有时刻中内存留驻 Tensor 总量的最大值最小(内存占用峰值最小),即:

$$checkpoints = argmin_{checkpoints} \left(\max \left(MemAlloc_{Block_i} + \sum_{j=0}^{i-1} MemAlloc_{ckpt_j} \right) \right), i \in [1, n] \quad (2)$$

根据图 4.4 以及式(2)，不难看出，想要得到最佳的 checkpoints 组合，需要使得顶部的 Block 尽量大，底部的 Block 尽量小；即，在这样的条件下得出的 checkpoints 的分布不是均匀的，而是前疏后密。同时，从图 4.2 可以观察到，本研究通过试运行得到的 checkpoints 候选点比较均匀地分布在网络中。因此，如果能让最佳 checkpoints 分布也接近这样的均匀形式，那么 GPU 内存优化的空间将会进一步变大。

将内存交换技术加入到当前方法中，可以使得 checkpoints 的最佳分布变得更均匀。内存交换技术基于这样一个事实：前向计算中的输出变量，在反向传播中再次被访问之前，存在一段不被任何计算依赖的空闲时间。如果在这段时间内将其交换到 CPU 内存空间中，那么就可以节约出一部分的 GPU 内存。更为重要的是，在图 4.4 所展示的情形中，所有的 checkpoints 共同提高了 GPU 内存使用的峰值，如果对这些 checkpoints 应用内存交换，那么减少的 GPU 内存，正好对应于 GPU 内存使用的峰值。

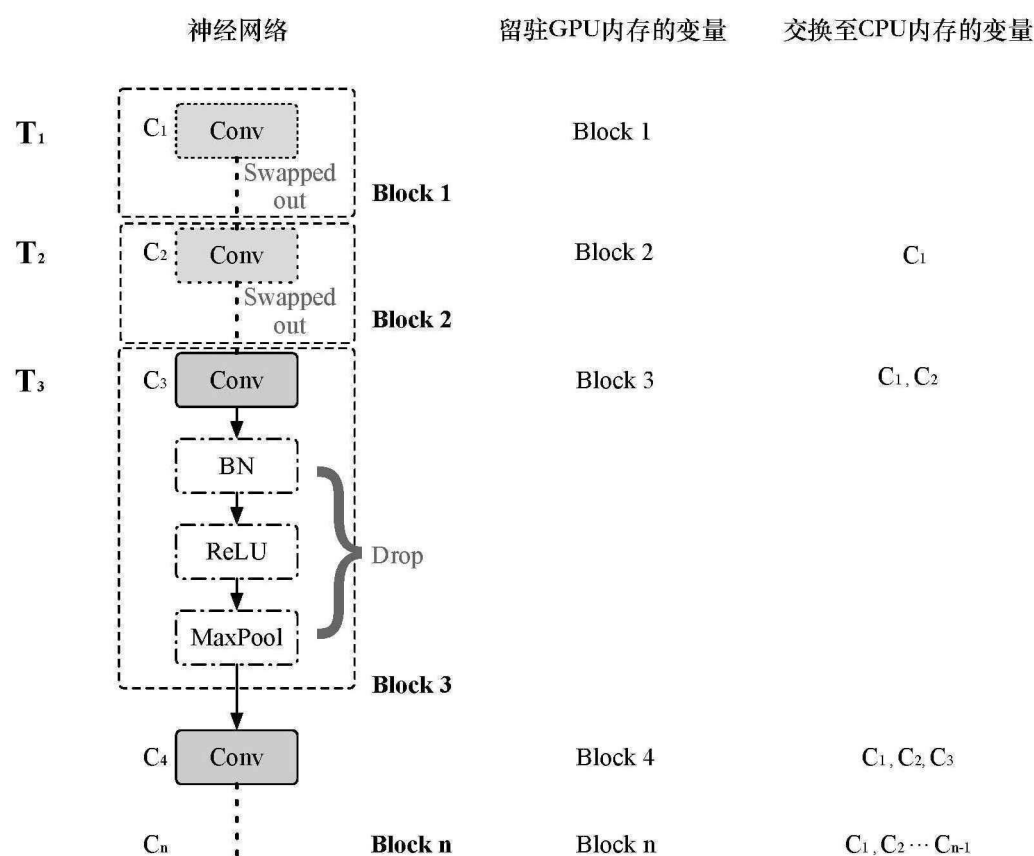


图 4.5 神经网络重训练 Checkpoint 及内存交换示意图

图 4.5 展示了应用内存交换技术后的内存占用情况。在 T_3 时刻， C_1 和 C_2 都已

经被交换到 CPU 内存中, 此时 GPU 内存中只存在 Block 3 的 Tensor; 并且 Block 3 完成计算后, C_3 将被交换至 CPU 内存, 除此之外的其他变量将被回收内存。此情况也适用于任意时刻 T_i : 在任意时刻, GPU 内存中驻留的变量, 都只有当前时刻的检查点及其对应的 Block。在这种情况下, (2) 式将被简化为:

$$checkpoints = argmin_{checkpoints} (\max(MemAlloc_{Blocki})), i \in [1, n] \quad (3)$$

在同时应用重计算和内存交换的情况下, 只需要寻找一个使得所有 Block 中的最大值最小的 checkpoints 方案即可。需要注意的是, 如果按照上式, 只需将所有候选点均设置为 checkpoint, 即可获得最小的 Block 大小, 即最小峰值内存使用。然而, 该式成立的前提是所有非当前 Block 的 checkpoints 都能被交换到 CPU, 并且在适当的时候交换回 GPU 内存中。这需要保证一个 Block 的计算时间需要大于邻近 checkpoint 的交换时间, 包括换出和换入的时间, 还要加上调度和同步的时间开销, 以此作为 checkpoint 选取的限制条件。那么这个限制条件的具体取值如何得到呢? 幸运的是, 在 4.3 节描述的试运行过程中, 我们已经得到了 CPU-GPU 之间数据传输的速率, 以此为依据, 同时考虑被交换的检查点的大小, 便可以得出两个 checkpoint 之间需要满足的最小时间间隔。

下面开始描述如何在 4.3 节给出的候选点中搜索最佳 checkpoints 方案的过程。按照图 4.6 的流程所示, 最佳 checkpoints 方案的搜索需要依以下步骤进行:

- 1) 初始化阈值为 0, 启动 n 轮迭代, 通常 n 取 5 或 7 即可使得阈值收敛;
- 2) 每轮迭代中, 以拓扑顺序考察神经网络中的每个 Tensor。神经网络的输入(data 层)作为第一个 checkpoint, 同时也是第一个 Block; 同时, 神经网络的参数变量(weights, bias 等)不加入 Block;
- 3) 检查当前 Tensor 是否为候选点, 如果不是则直接加入当前 Block;
- 4) 如果当前 Tensor 是候选点, 则检查当前 Block 是否超过阈值: 如果没超过阈值, 则将当前 Tensor 加入当前 Block, 否则创建新 Block 并将当前 Tensor 标记为 checkpoint;
- 5) 重复 3) 和 4), 直至所有 Tensor 考察完毕, 得到当前的 checkpoint 组合, 以此更新阈值, 同时运行一次 4.5 节介绍的静态计算图内存申请算法, 得到实际内存占用量;
- 6) 以新的阈值, 重复 2) 至 5), 直至完成预定次数的搜索, 输出使得实际内存占

用量最小的 checkpoint 方案。

这里需要展开说明 5) 中阈值更新的细节。为启发式求解式(2)(3)，我们希望对于每个 T_i 时刻，其内存使用量 $MemAlloc_{Ti}$ 都维持在一个相似的水平，故阈值更新的方式为：

$$Threshold = \sqrt{\frac{\left(\frac{\sum_{i=1}^n MemAlloc_{Ti}}{n}\right)^2}{2}} \quad (4)$$

该阈值代表的是每个 Block 应该保持的最小大小。由于神经网络每层特征图的尺寸是离散的，所以理论上不存在所有 Block 大小严格相等的方案。为了简便起见，本研究简单地将 Block 的最小大小限制为上一次迭代中所有 Block 的算术平均数除以 $\sqrt{2}$ 。对于通常级别优化（启用变量重计算）和极致级别优化（启用变量重计算和内存交换）， $MemAlloc_{Ti}$ 会分别取不同值。注意，图 4.6 所示算法的搜索方向是公式(2)(3)，这是一个理论上的内存占用值，实际的内存规划需要由 4.5 节介绍的内存规划算法给出。在 6) 中最终输出的 checkpoint 方案，定义为由图 4.1 的第四步，即内存规划算法所给出的实际内存申请值最低的方案，而非使得式(4)最小的方案。

本节介绍了 HelloDeep 中应用变量重计算和内存交换的方法。与前人工作相比，本研究的进步是把重计算和内存交换应用到了一起，在 checkpoint 方案搜索时同时考虑应用两个技术带来的影响，而非将其割裂成两种无关的方法。与使用遗传或者模拟退火算法进行搜索的方式相比，本节提供的启发式搜索能更快地给出足够好的方案。同时，如 4.3 节所述，本节算法的搜索对象也并非由先验知识指定的特定算子层，而是在每次训练进行时所动态获取的具体网络层。

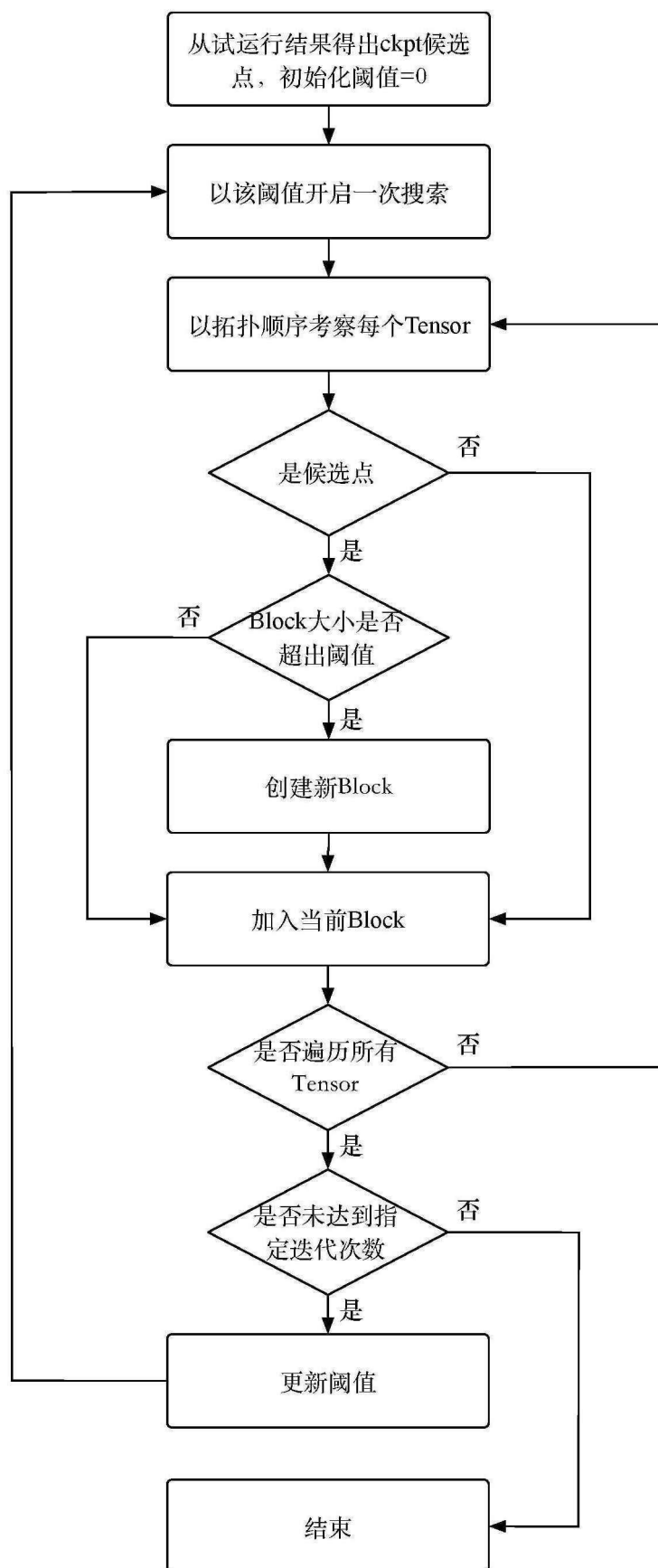


图 4.6 启发式 Checkpoint 方案搜索算法流程图

4.4.2 具体实现

上节介绍了 HelloDeep 启发式重计算和内存交换方案的具体实施方法，本节将介绍其在 HelloDeep 中的具体实现，包括图 4.6 所示算法的细节，以及变量重计算以及内存交换的具体实施过程。

伪代码 1 SearchCkptScheme 描述了启发式搜索的过程：该函数接受神经网络 net 和 4.3 节方法给出的候选点 candidates，返回最佳的 checkpoint 方案。第 6 行中的 GenCkpts() 函数，对应 4.4.1 节中 2) 至 5) 步描述的流程，即通过给定的候选点和阈值，给出一个 checkpoint 方案；第 7 行 MakeMemScheme() 函数是我们即将于 4.5 节介绍的基于静态计算图的内存申请方案生成函数，GetCost() 函数则给出了总体的 GPU 内存占用。第 8 至第 12 行则体现了本算法的搜索目标，即最小化最终 GPU 内存占用，而非最小化 threshold。第 13 行的阈值更新则使用了如式(4)所示的方法。第 5 到第 14 行，基本上就对应了图 4.6 所示的算法流程。

Algorithm 1. SearchCkptScheme

Input: net, candidates

Output: best_scheme

```

1:  min_cost ← Inf
2:  threshold ← 0
3:  best_threshold ← 0
4:  best_scheme ← []
5:  for i ← 0 to search_times do
6:    blocks, ckpts ← GenCkpts(net, candidates, threshold)
7:    cost ← GetCost(MakeMemScheme(net, ckpts))
8:    if cost < min_cost do
9:      min_cost ← cost
10:     best_scheme ← ckpts
11:     best_threshold ← threshold
12:   end if
13:   threshold ← UpdateThreshold(blocks)
14: end for
15: lower_bound ← best_threshold /  $\sqrt{2}$ 
16: upper_bound ← best_threshold *  $\sqrt{2}$ 
17: step ← (upper_bound - lower_bound) / grid_search_times
18: for i ← 0 to grid_search_times do
19:   blocks, ckpts ← GenCkpts(net, candidates, lower_bound)
20:   cost ← GetCost(MakeMemScheme(net, ckpts))
21:   if cost < min_cost do
22:     min_cost ← cost

```

```

23:     best_scheme ← ckpts
24:   end if
25:   lower_bound ← lower_bound + step
26: end for
27: return best_scheme

```

除了利用式(4)来更新阈值以进行搜索之外,本研究还可以对搜索到的最佳 checkpoints 方案再进行第二轮细粒度的搜索,以期找到更接近于理论最优的结果,这一想法被实现为 15 到 26 行的代码。在这部分代码中,依据上一步搜索到的最佳阈值,分别乘除 $\sqrt{2}$ 以得到细粒度搜索 grid_search 的上下界;然后在这段区间内平均分为 grid_search_times 段来进行搜索。最后,在 27 行返回两轮搜索得到的最佳方案。

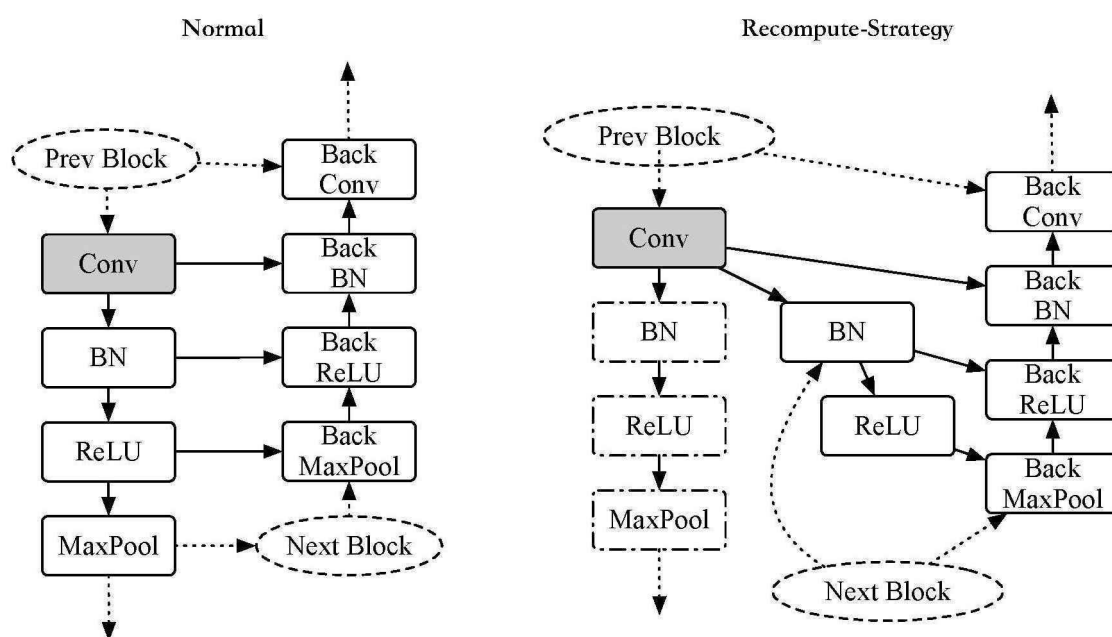


图 4.7 神经网络重计算策略下计算图变化示意图

上文介绍了 HelloDeep 如何从候选点中搜索最佳 checkpoints 方案,现在开始介绍如何利用该信息进行变量的重计算和内存交换。本文在第二章介绍过 HelloDeep 的框架和主要模块,其神经网络的表示方式是静态计算图,而计算的过程是 ProActor 模式的异步多线程计算。现在要实现重计算和内存交换,直观的想法是在运行时判断当前 Tensor 是否为 checkpoint,若是则考虑对其进行内存交换,若否则考虑在计算结束时回收其内存。这样的做法虽然直观,但是在多线程的环境下会增加非常多冲突的风险,为此要增加许多同步工作,从开发效率上来说非常不值得。HelloDeep 采用了另一种做法,即根据 checkpoints 方案,直接修

改原有的计算图，从开发角度而言，由于不需要考虑多线程间的同步问题而更加方便；从运行角度而言，修改后的计算图交由 **Parallel** 模块进行调度，也比上一种在运行时更改其行为的方案更加高效。

图 4.7 展示了神经网络训练过程中正常模式下的计算图与重计算策略下计算图的区别。在正常模式下，所有变量都常驻内存，反向计算时可以直接从前向时的变量处获取数据。在重计算策略下，除了 **checkpoint** 之外的变量都被回收内存。当执行反向梯度传播时，当 **Next Block** 完成计算时，从 **Conv** 到 **BN** 的计算的两个输入依赖都被满足，此时会自动执行 **Conv-BN-ReLU** 的重计算，从而使得反向传播的依赖得以满足。此时内存中保留的变量有 **Conv**，**BN**，**ReLU** 以及 **BackMaxPool**。当 **BackMaxPool** 计算结束之后，它以及它的输入变量 **ReLU** 的内存都可以被回收，以便为 **BackReLU** 申请内存空间并进行计算。可以看到，在反向传播过程中，内存中留驻的变量占用的空间仍然满足 4.4.1 中所讨论的值，式 (2)和(3)的正确性在反向传播过程中也得到验证。

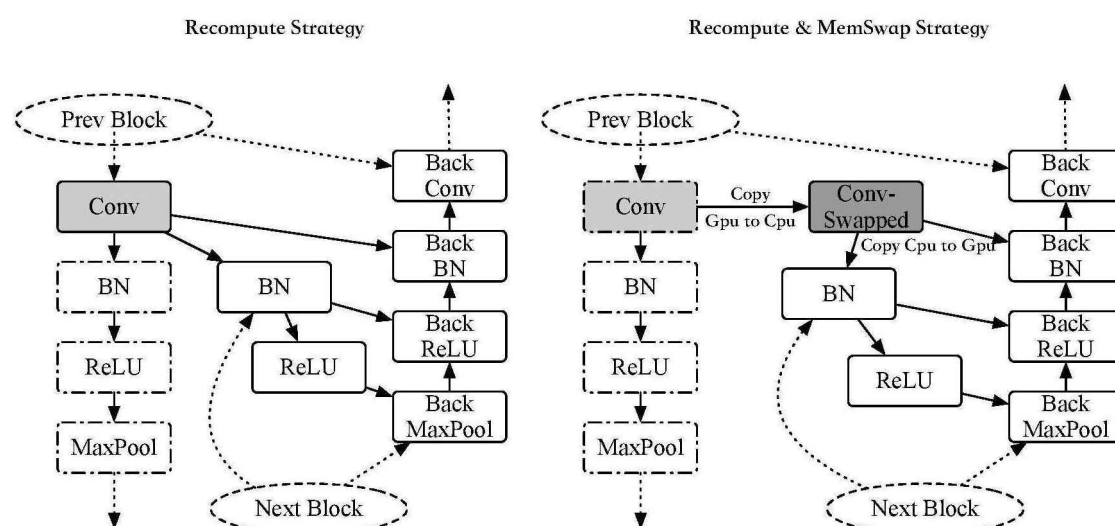


图 4.8 神经网络重计算及内存交换策略下计算图变化示意图

图 4.8 展示了应用内存交换技术后计算图的变化，本研究通过将 **checkpoint** 变量交换至 **CPU** 内存中以减少 **GPU** 内存的占用。可以看到，应用该策略之后，整个前向过程中所有变量都可以在计算完成后进行内存的回收。被交换至 **CPU** 的变量，会在被需要时重新拷贝回 **GPU**。值得注意的是，本文在之前的 3.2 节中介绍过 **GPU** 数据传输的最佳实践，所以在这里我们也应用了这些技巧：分别使用专门的两个 **GPU Stream** 处理数据的拷出和拷入，并且传输时目的地设置为 **CPU**

的不可分页内存，利用 DMA 技术提升传输效率。

至此，我们已经完整介绍了 HelloDeep 的启发式重计算和内存交换方案搜索的具体方法和实现过程。可以看到，和前人工作相比，重计算和内存交换两个方法不仅在搜索过程中紧密结合，在具体实现时也是同样结合到一起的。

4.4.3 实验验证

本节中展开对上文介绍的启发式重计算和内存交换方案搜索进行测试。

我们以 Vgg-19 网络为例展示启发式搜索过程中的细节。根据 4.3 节介绍的试运行方法，我们选取了所有的 Conv 层和 FullyConnected 层作为 checkpoint 的候选点；输入图像的尺寸为 3*256*256，batchsize 为 16。在不进行任何优化的前提下，该网络的中间特征图大小为 2505MB；在启用基于静态计算图分析的内存申请方案后，特征图大小可以减少至 1689MB；在此基础上，采用变量重计算策略后，特征图大小可以减小为 1144MB。这些数据意味着，变量重计算策略可以将特征图大小降低至原来的 67.7%(1144MB/1689MB)。下面来看如何通过启发式搜索实现 1144MB 的最终内存占用。

如表 4-1 所示，对于给定的 checkpoint 候选点，我们首先进行了 5 轮搜索，在搜索过程中，阈值的变化按 4.4.1 节(4)式所示进行。在初次的 5 轮搜索过程中，第 3 轮开始搜索结果就开始趋于稳定，同时在第 2 轮以 727MB 为阈值的情况下得到最低的内存占用量。而正如 4.4.2 节 algorithm 1 所述，我们额外进行了第二次的搜索。在这次搜索中，阈值的变化将以初次搜索中的最佳阈值的 0.707 倍为下限，1.414 倍为上限，平均分为 7 份进行。在第二次的 7 轮搜索中，在第 5 轮得到最终最佳的内存占用表现。

表 4-1 内存重计算策略最低内存占用搜索

初次搜索							
轮数	1	2(best)	3	4	5		
閾值(MB)	0	727	569	592	592		
内存占用(MB)	1448	1154	1320	1320	1320		
二次搜索							
轮数	1	2	3	4	5(best)	6	7
閾值(MB)	514	587	660	733	806	879	952
内存占用(MB)	1224	1224	1240	1216	1144	1176	1176

阈值的变化影响着 checkpoint 的选择以及相应的 block 大小，从而决定最后

的内存占用。图 4.9 展示了 3 个不同阈值下，相应的 $MemAlloc_{Ti}$ 大小分布情况。 $MemAlloc_{Ti}$ 由 4.4.1 节(1)式定义，包括了当前 Block 的大小以及当前 Block 前所有 checkpoint 的大小之和。

图 4.9-(a)展示了初次搜索的第一轮，即阈值初始化为 0 时的 checkpoint 选择情况。由于阈值为 0，所有的候选点都会被选择为 checkpoint，共有 17 个。图中的散点显示了这 17 个 checkpoint 对应的 $MemAlloc_{Ti}$ 值，从(1)式可以得知，checkpoint 的数量越多，被累计的内存量也越多，也就不利于最终的整体内存占用。图 4.9-(b)则展示了初次搜索中的最佳情况，即阈值取 727MB 时的内存分布情况。此时 checkpoint 数量从 17 个减少到了 5 个，每个 checkpoint 之间的间隔更长，相对应的 $MemAlloc_{Ti}$ 值分布也更平均，所以得到了 1154MB 的最终内存占用。图 4.9-(c)则展示了二次搜索过程中的最佳内存表现时的内存分布情况，此时 checkpoint 数量只剩 2 个，阈值参考线 806MB 和最终占用的参考线 1144MB 也更加接近。

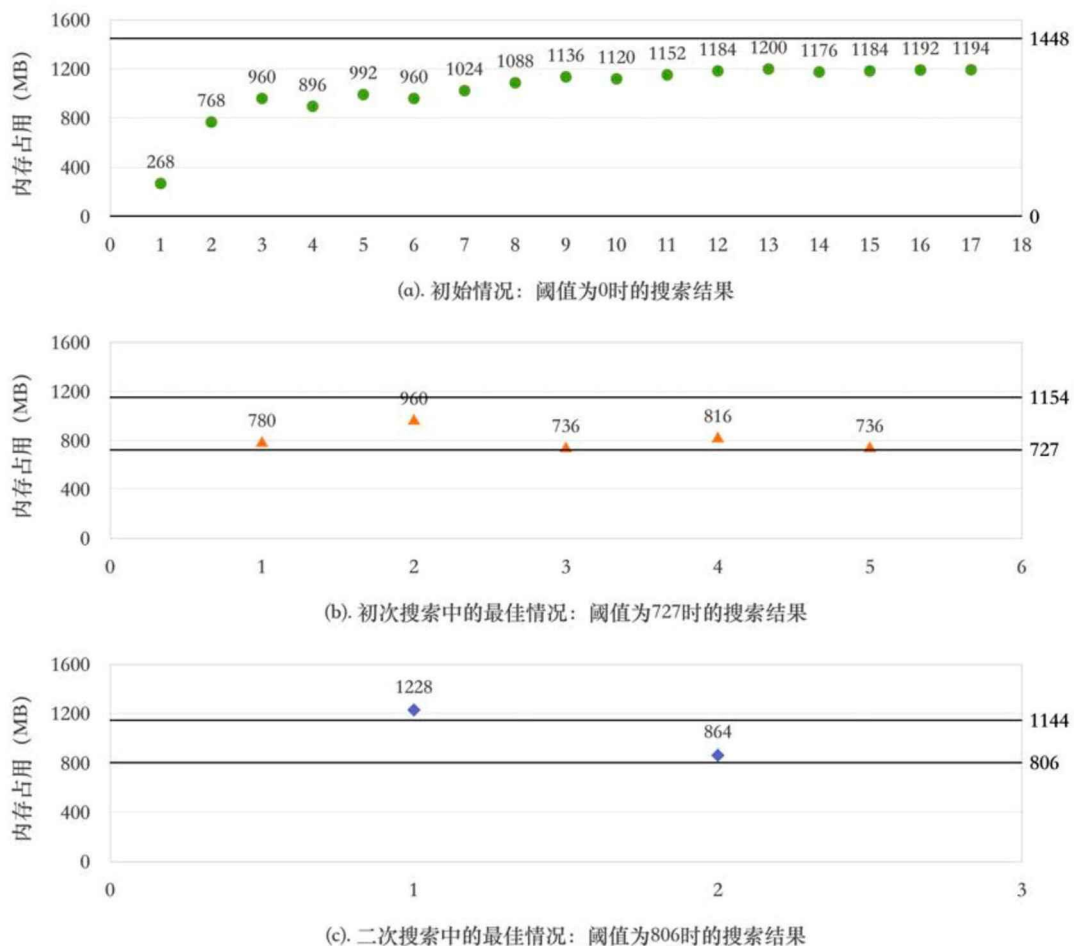


图 4.9 启发式 checkpoint 搜索算法不同阈值下内存占用散点图

可以看到,HelloDeep 的启发式变量重计算搜索策略,确实可以使得 checkpoint 方案迅速收敛(初次搜索),并且在细粒度的二次搜索中进一步地给出更接近局部最优解的 checkpoint 方案,有效地提升了 GPU 内存使用效率。

4.5 节将要介绍的内存申请算法还模拟了在一次完整的前向-反向过程中的内存申请情况,我们通过该算法来观察内存交换策略给 GPU 内存节省带来的影响。

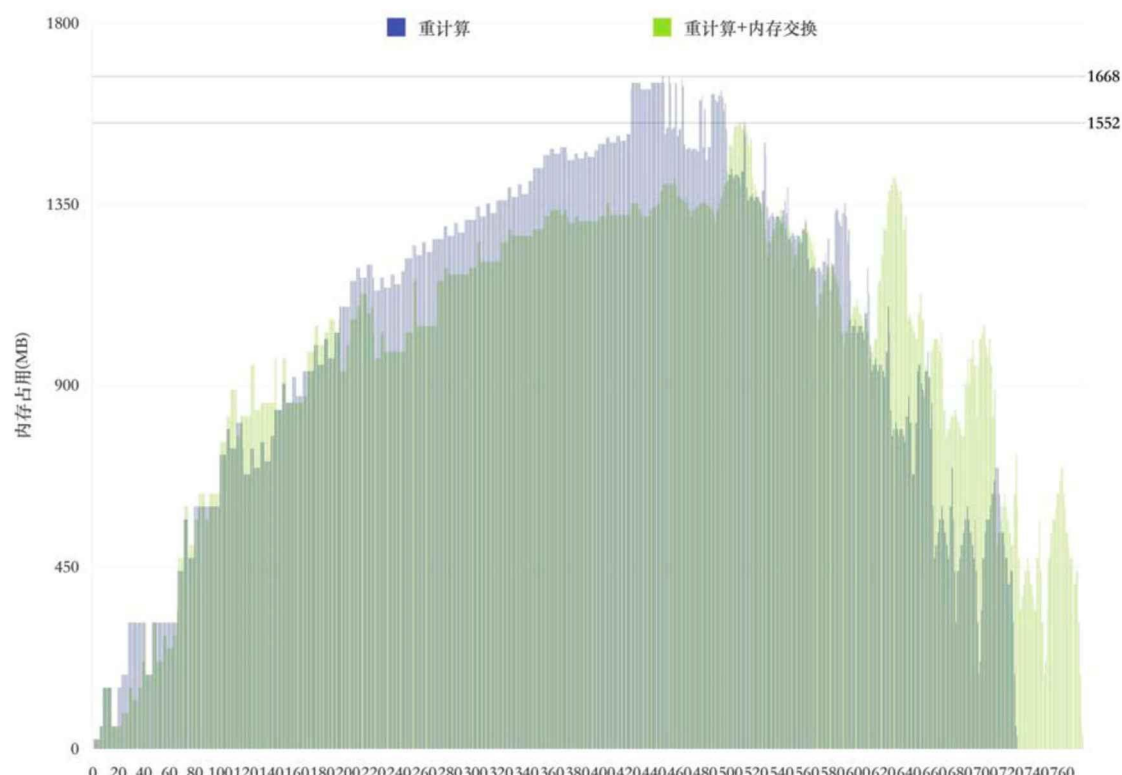


图 4.10 一次前向-反向传播过程中 GPU 内存使用量随时间变化柱状图

图 4.10 展示了在训练过程中一次前向-反向传播过程中的 GPU 内存使用量变化情况。本次测试采用 ResNet-50 模型,输入图像尺寸为 $3 \times 256 \times 256$, batchsize 为 32。可以看到,图中蓝紫色的部分是应用重计算策略后的内存变化情况,此时内存申请算法给出的峰值内存使用量为 1668MB;在重计算的基础上再添加了内存交换策略后,峰值内存使用量降低到了 1552MB,同时在代表时间的 x 轴上拉得更长。从图中绿色的部分可以观察到,在峰值之后出现了 3 个局部峰值,代表了 checkpoint 被分别交换回 GPU 并执行重计算的过程。由此可见,内存交换策略可以有效降低内存的峰值使用量,同时也付出了更多的计算时间。

4.5 基于静态计算图的内存申请方案

4.5.1 方法描述

在上节介绍的启发式搜索过程中，寻找的最佳方案的依据是本节将要介绍的、基于静态图的内存申请方案给出的实际 GPU 内存占用量。1.2.3 节中我们介绍过动态内存申请（DSA）技术，分为在线 DSA 和离线 DSA 两种：TensorFlow 与 Pytorch 使用动态计算图，与之对应地必须采用在线的动态内存申请方案，在不确定后续内存请求的大小和时间的情况下为每次内存申请分配内存块；HelloDeep 使用静态计算图实现神经网络计算，所以在内存申请时使用的也是离线 DSA 模型。

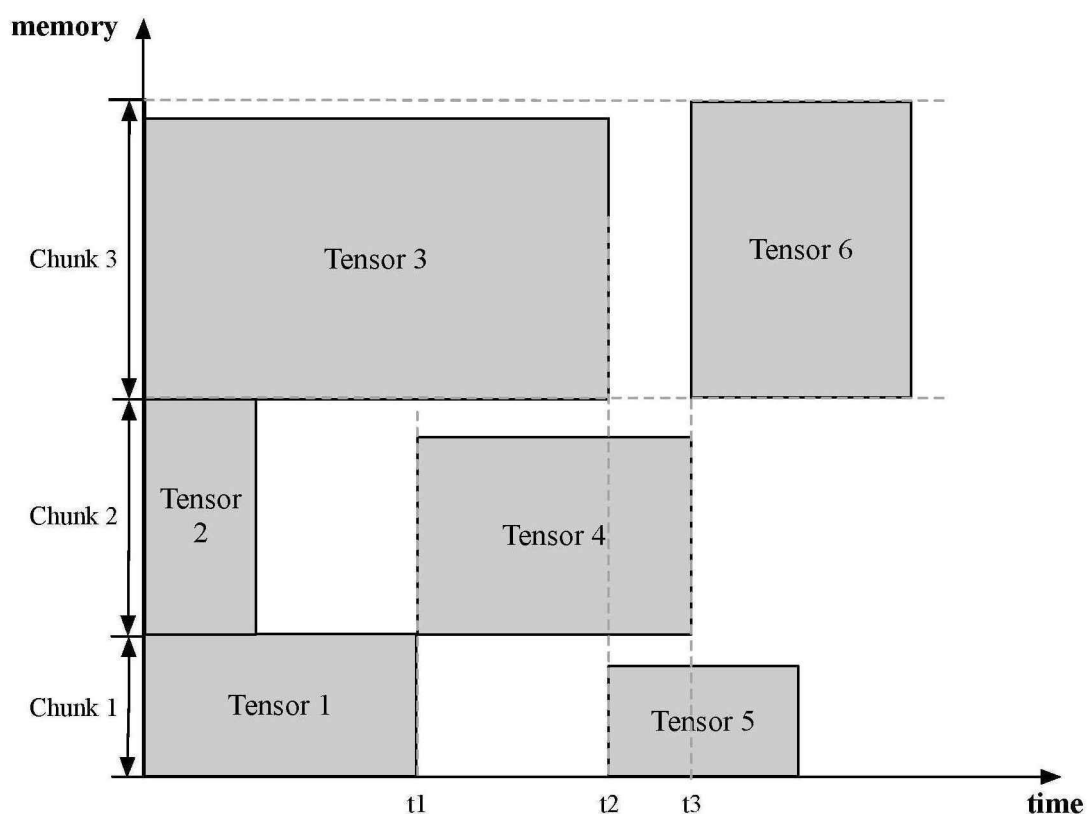


图 4.11 神经网络内存分配过程示意图

为了更好地理解在线 DSA 和离线 DSA 两种不同的内存申请方式的区别，考察图 4.11 所示的神经网络内存分配过程示意图，横轴为逻辑时间，纵轴为内存累计使用量。可以看到，Tensor 1, Tensor 2, Tensor 3 三个变量之间没有相互依赖，所以各自申请了一块内存； t_1 时刻，Tensor 1 完成计算，依赖其结果的 Tensor 4

开始进行计算，此时空闲的内存块有 **Chunk 1** 和 **Chunk 2**，**HelloDeep** 内存分配算法会将其安排至大小最接近的 **Chunk 2** 中， t_2 时刻的 **Tensor 5** 同理。值得注意的是 t_3 时刻，**Tensor 6** 的大小比空闲内存块 **Chunk 2** 和 **Chunk 3** 都要大，此时 **HelloDeep** 内存分配算法会将 **Chunk 3** 进行扩容，因为它的大小和 **Tensor 6** 更接近。需要强调，这个扩容不是在运行时进行的，而是经过算法的规划之后，**Chunk 3** 在一开始就会以 **Tensor 6** 的大小为基准进行申请。意即，**HelloDeep** 内存分配算法是对所有 **Tensor** 内存分配的一个模拟，产生出最后方案后，才会进行实际的内存申请，以此保证运行时不发生多余的内存申请、释放和扩容等操作。与 **TensorFlow** 的动态内存分配算法相比，**HelloDeep** 的内存分配是预先完成的，不会产生任何的运行时开销，这主要得益于静态计算图的设计。

按照流程图 4.12 所示，计算实际内存申请方案需要按照以下步骤进行：

- 1) 根据静态计算图中每个节点输出的数量，初始化相应 **Tensor** 的引用依赖：一个输出对应 1 份引用量；维护已申请内存块和空闲内存块两个数据结构，均初始化为空；
- 2) 以拓扑顺序考察每个 **Tensor**，模型的参数（输入，权值等）不参与内存分配；
- 3) 检查当前空闲内存块列表是否非空，若为空则为当前 **Tensor** 申请新的内存块；
- 4) 若存在空闲内存块，检查其大小和当前 **Tensor** 的大小之比是否落在合理区间内，若是，则将该空闲内存块分配给当前 **Tensor** 以避免新的内存申请，否则则为该 **Tensor** 申请新的内存空间；
- 5) 对当前 **Tensor** 的所有输入，减少它们的引用计数，表示当前 **Tensor** 已完成计算，不再依赖输入 **Tensor**。若输入 **Tensor** 列表中存在引用计数减少到 0 的对象，则释放该 **Tensor** 的内存，对应的内存块加入到空闲内存块列表中；
- 6) 重复 3)至 5)，直至完成对所有 **Tensor** 的考察，输出最终的内存申请方案。

需要说明，4)中所谓“合理区间”，是一个可搜索的值；该值分别取 1, 2, 4, 8, 16 时，各进行一次完整的内存申请方案推理，然后从中选择使得最终申请的内存总量最小的方案。如果当前 **Tensor** 比当前空闲内存块大，那么此时就会发生扩容。此处的取值是因为神经网络特征图之间的尺寸变换倍数通常是 2 的幂次，所以取 1, 2, 4, 8, 16 就分别对应了允许不同大小的特征图共享同一块内存的倍率限制。

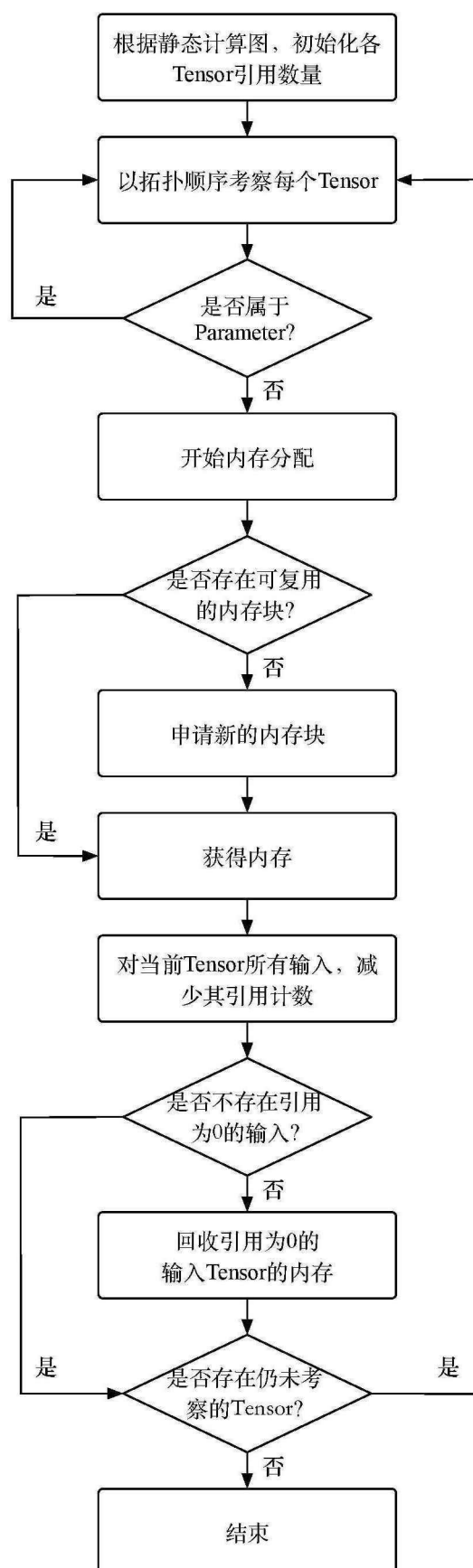


图 4.12 基于静态计算图的内存申请方案算法流程图

4.5.2 具体实现

基于静态计算图的内存申请方案生成的核心算法，如下伪代码 Algorithm2

MakeMemScheme 所示：

Algorithm 2. MakeMemScheme

Input: topu_graph, max_scale

Output: best_scheme

```

1:  ref_cnts ← vector(topu_graph.NumEdges(), 0)
2:  best_scheme ← []
3:  free_list ← []
4:  for i ← 0 to topu_graph.NumNodes() do
5:    node ← topu_graph.NodeAt(i)
6:    if node.IsParam() do continue
7:    for edge in node.Inputs() do ++ref_cnts[topu_graph.EdgeIdx(edge)]
8:  end for
9:  match_range ← 1
10: while match_range < max_scale do
11:   mem_chunks ← []
12:   for i ← 0 to topu_graph.NumNodes() do
13:     node ← topu_graph.NodeAt(i)
14:     if node.IsParam() do continue
15:     for chunk in free_list do
16:       if Match(node.Size(), chunk.Size(), match_range) do
17:         chunk.size ← max(node.Size(), chunk.Size())
18:         node.chunk_idx = chunk.Idx()
19:         break
20:       end for
21:       if node.chunk_idx == NULL do
22:         chunk ← Alloc(node.Size())
23:         node.chunk_idx = chunk.Idx()
24:         mem_chunks.Append(chunk)
25:       end if
26:       for edge in node.Inputs() do --ref_cnts[topu_graph.EdgeIdx(edge)]
27:       FreeZeroRef(node.Inputs(), ref_cnts, free_list)
28:     end for
29:     if mem_chunks.Size() < best_scheme.Size() or best_scheme.Empty() do
30:       best_scheme ← mem_chunks
31:     end if
32:     match_range ← match_range * 2
33:   end while
34: return best_scheme

```

在这份伪代码中，Node 代表计算图中各个节点，即神经网络的 layer；Edge 代表各个节点之间的边，即 layer 的输入输出 Tensor。第 4 至第 8 行展示了各个网络层的输入 Tensor 的初始化情况，其值存放于长度为所有 Tensor 个数且初始化为 0 的数组 ref_cnts 中；第 10 行至第 33 行的 while 循环，展现了匹配阈值从 1 到 max_scale 变化，以每个匹配阈值来进行一次内存方案规划，并记录最小内存占用方案的整体流程。该流程已在 4.5.1 节中介绍，在此不再赘述。

从这份代码中可以看出，MakeMemScheme()所做的，实际上只是返回了一个以 Chunk 为基本元素的 mem_chunks 数组，Chunk 中仅仅记录了该块内存所需要的空间大小以及哪些 Tensor 属于这块内存，而并不实际申请内存。所以函数 MakeMemScheme()仅仅是一次网络正式运行前的“预演”，等到该函数给出最佳内存方案后，HelloDeep 才会一次性地向 GPU 申请足够数量的内存块，并按照该方案进行划分，以实现运行时零开销的 Tensor 内存调度，规避了动态内存申请中频繁调用 CUDA 的 GPU 内存申请和释放接口带来的性能损失。

需要强调的是，从代码中可以看出 MakeMemScheme()的适用对象是整张计算图，包括原始计算图以及经过重计算和内存交换方案改写后的计算图，故本小节介绍的方法可以和 4.4 节介绍的方法很好地兼容。

4.5.3 实验验证

如 4.4 节所言，启发式搜索的最终目的是使得实际内存占用值最小，而实际内存占用值则由本节所述的基于静态计算图的内存申请方法给出。下面我们利用实验展示在给定计算图和 checkpoint 方案的前提下的内存申请过程。

图 4.13 展示了不同 match_range 取值下，基于静态计算图的内存申请算法所给出的 MemChunk 大小分布情况。当 match_range 取 1 时，每个 Tensor 都只能分配到和自身大小严格对应的 MemChunk，于是图 4.13-(a)的 MemChunk 总数最多；同理，match_range 取 16 时，MemChunk 总数最少。由于申请内存总数等于所有 MemChunk 的大小之和，在 Tensor 大小分布情况保持不变的情况下，MemChunk 数量越少，最终的 GPU 内存申请总数也就越少。但随着 match_range 的增大，单个 MemChunk 内部对应的 Tensor 数也会逐渐增加，这意味着内部内存碎片会更多，内存利用率下降，多个 Tensor 之间竞争同一个 MemChunk 所带来的同步开

销也会增大。因此, `match_range` 不能取太大的值。`match_range` 最大取 16, 即可在 GPU 总体占用量、GPU 碎片量、Tensor 同步开销三者之间取得一个平衡。

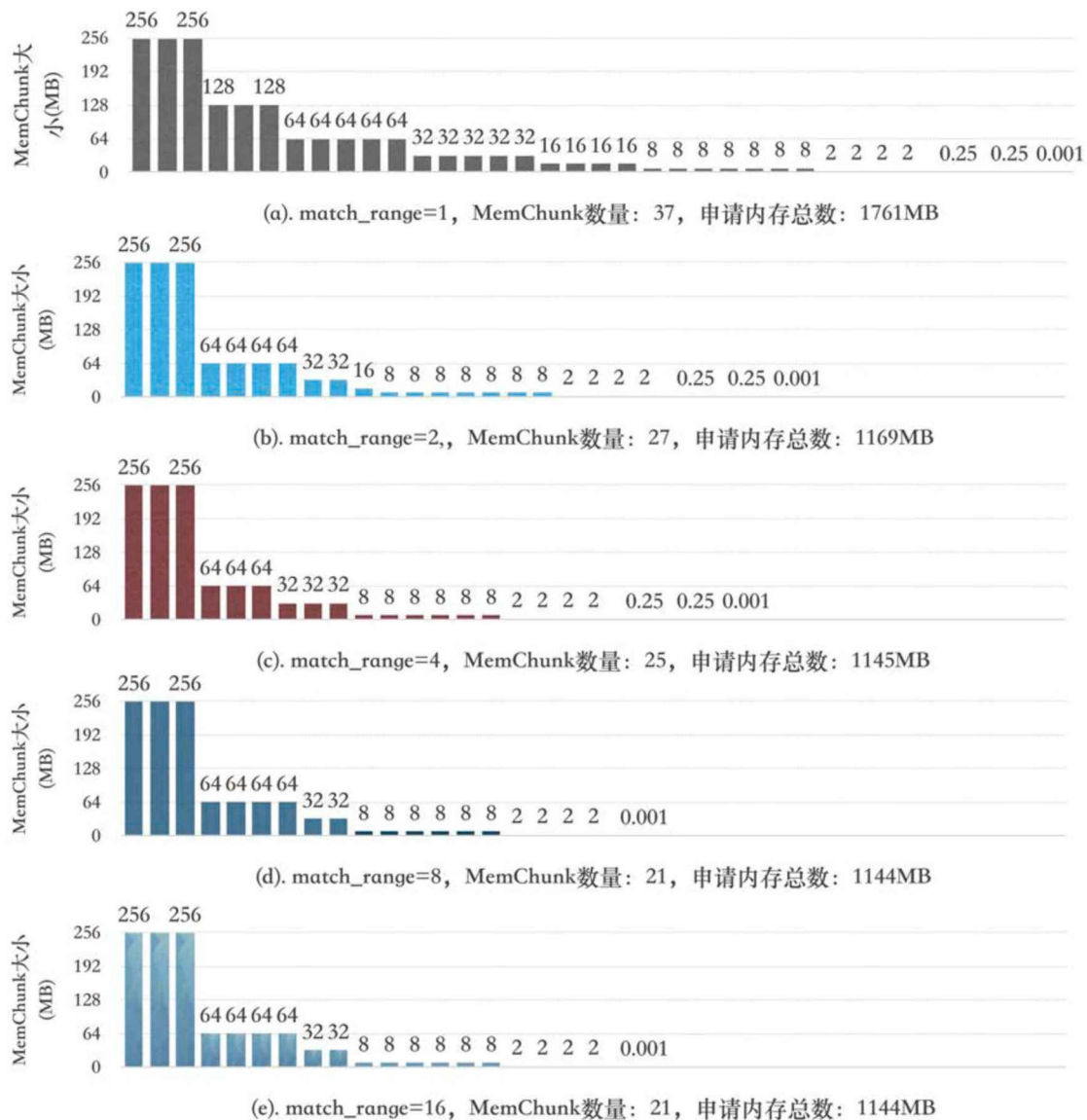


图 4.13 内存申请算法不同 `match_range` 取值下 MemChunk 大小分布柱状图

4.6 Cudnn 卷积算法辅助空间限制

4.6.1 方法描述

Cudnn^[56]是 Nvidia 推出的, 专门为 GPU 设计的深度神经网络算子加速库, 支持诸如卷积、池化、规范化等操作的加速。以卷积为例, Cudnn 支持 7 种卷积算法的实现, 包括 FFT^[57], WINOGRAD^[58], GEMM^[59]等, 其中的部分算法需要额

外的 GPU 内存作为中间结果的缓存。对于一张大小为 24.5MB 的特征图而言，为此进行卷积计算所需的 Cudnn 辅助空间从 0, 0.64MB, 443MB 到 2145MB 不等。为避免出现 GPU 内存因 Cudnn 辅助空间而出现不足的情况，HelloDeep 对其进行了限制。

限制 Cudnn 的辅助空间，出于这样的一项观察：在对 ResNet-50 进行不同 batchsize 的训练过程中，我们发现其内存使用峰值的出现具有周期性，且在不同的 batchsize 下，GPU 内存使用的峰值都稳定地比基础水平高 2.2GB 左右，如图 4.14 所示：

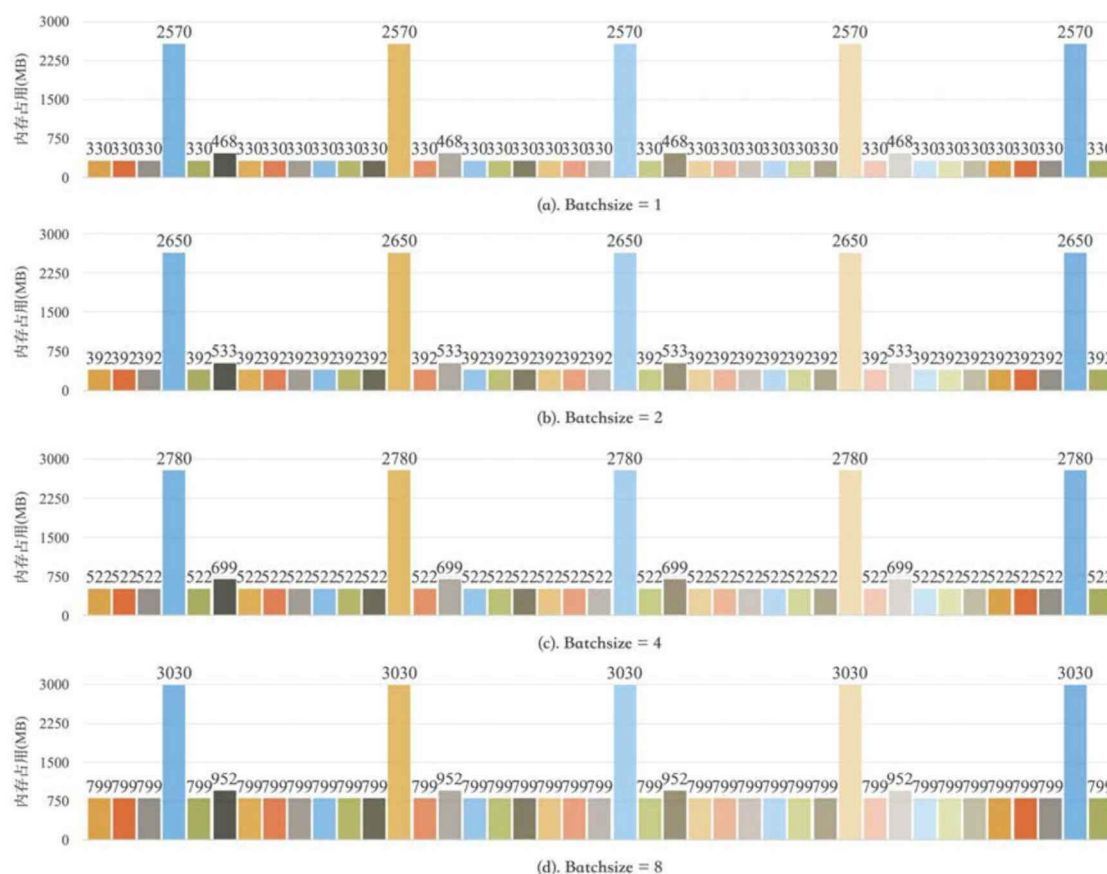


图 4.14 重计算策略下 ResNet-50 不同 batchsize GPU 内存占用对比图

从图 4.14 中可以看出，batchsize 不影响 cudnn 卷积算法占用的额外辅助空间大小；当 batchsize 越小时，cudnn 占用的额外空间就越主导着整体的内存占用。对于主流的 11GB GPU 内存（如 NVIDIA RTX 2080Ti）而言，2.2GB 也是不可忽视的大小。

Cudnn 提供了 `cudnnGetConvolutionForwardAlgorithm_v7()` 函数，该函数可以给出对于指定输入的卷积运算，不同的算法所需要的 Cudnn 辅助空间大小，

HelloDeep 以此为依据过滤掉超出限制的算法。表 4-2 记录了该函数给出的 Cudnn 各卷积算法所需的辅助空间大小，并以计算效率排序。如果不限辅助空间的大小，那么 Cudnn 就会默认采用最靠前的算法来进行卷积的运算，在表 4-2 中就是 winograd 算法。其中，implicit_gemm 算法不需要使用额外空间。Cudnn 保证对于所有的卷积运算，始终支持 0 额外空间的 implicit_gemm 算法。

表 4-2 Cudnn 卷积前向计算不同算法的辅助空间大小

input shape: 4, 128, 112, 112. filter shape: 128, 128, 3, 3. feature map size: 24.5MB	
CUDNN_CONVOLUTION_FWD_ALGO_WINOGRAD	1.56MB
CUDNN_CONVOLUTION_FWD_ALGO_IMPLICIT_PRECOMP_GEMM	0.64MB
CUDNN_CONVOLUTION_FWD_ALGO_WINOGRAD_NONFUSED	443MB
CUDNN_CONVOLUTION_FWD_ALGO_IMPLICIT_GEMM	0
CUDNN_CONVOLUTION_FWD_ALGO_GEMM	221MB
CUDNN_CONVOLUTION_FWD_ALGO_FFT_TILING	76.5MB
CUDNN_CONVOLUTION_FWD_ALGO_FFT	2145MB

为了兼顾计算效率和内存空间的节约，我们尝试了若干不同的 Cudnn 算法额外空间限制阈值，并找到比较合适的一组。表 4-3 展示了不同限制阈值下，对于一张 256*256 尺寸的输入图片，Cudnn 仍能选取最佳效率的卷积算法的占比情况，以及相应的辅助空间大小。可以看出，仅需付出一些计算上的效率，便可以将辅助空间的大小减少 97%至 100%。最终，我们得出如下结论：对于基础级别的优化，不加限制；对于通常级别的优化，将其限制为输入特征图大小的 2000%；对于极致级别的优化，将其限制为输入特征图大小的 5%。

表 4-3 Cudnn 卷积前向计算不同限制下的效率和辅助空间大小

辅助空间大小限制	结果
不限制大小	最高效卷积算法频率 100%，最大辅助空间 2193MB
特征图大小的 2000%	最高效卷积算法频率 86.7%，最大辅助空间 64.125MB
特征图大小的 5%	最高效卷积算法频率 25.8%，最大辅助空间 0.635MB

与前人工作相比，不论是 Memonger, SuperNeurons, Capuchin 抑或是 Sentinel 和 SwapAdvisor，都没有在 Cudnn 辅助空间上给予关注，而是集中于讨论网络特征图本身。这样的做法虽然有效减少了特征图大小，但是在模型的实际训练过程中，GPU 内存使用量占比显著的 Cudnn 辅助空间就被忽视了。MXNet 的卷积实现就对 Cudnn 辅助空间进行了限制，但是是固定的 1024MB 大小，缺乏灵活性，对此缺乏敏感性的用户很可能不会注意到这一点。HelloDeep 采用的不同等级内

存优化方法，简单而有效地实现了多级别的 Cudnn 辅助空间限制。

4.6.2 实验验证

本节对 Cudnn 辅助空间限制进行测试。表 4-4 展示了 ResNet-50 和 Vgg-11 模型在实际训练过程中，不同 Cudnn 辅助空间限制下的模型训练峰值内存占用和运行时间情况。具体的内存占用和运行时间情况通过 NVIDIA Nsight System 工具统计得出。其中，运行时间是从实际产生 GPU 内存申请开始，到所有计算结束为止，不包含前期的准备过程，如 checkpoint 方案搜索，内存规划等。

表 4-4 内存重计算策略最低内存占用搜索

ResNet-50, 输入尺寸 3*256*256, batchsize=32, batch=15			
辅助空间限制	无	2000%特征图	5%特征图
峰值内存占用(GB)	4.74	4.58(-3.37%)	4.45(-6.12%)
运行时间(s)	8.17	10.96(+34.1%)	12.29(+50.4%)
Vgg-11, 输入尺寸 3*256*256, batchsize=32, batch=15			
辅助空间限制	无	2000%特征图	5%特征图
峰值内存占用(GB)	5.90	5.91	5.10(-13.6%)
运行时间(s)	11.30	11.08	14.34(+26.9%)

可以看到，对于 ResNet-50 模型而言，三个不同等级的限制分别带来了不同结果，最高可以降低 0.29GB 的峰值内存占用，即在不加限制的基础上减少了 6.12% 的内存消耗，随之而来的代价是增加了 50.4% 的模型计算时间；而中间的 2000% 特征图限制的方案相较于其他二者而言，则正好取得了比较折中的效果。

与此同时，对于 Vgg-11 模型，2000%特征图大小的限制方案和不加限制的方案取得了相近的结果，说明即使不加限制，Cudnn 占用的内存空间也不会超过 2000%特征图大小。尽管如此，最后 5%特征图大小的严格限制之下，我们仍然取得了 13.6%峰值内存节省的有效结果。

4.7 总体方法与 baseline 对比实验

本节将对 HelloDeep GPU 内存优化方法进行总体测试，展示基础-通常-极致三个不同优化级别下的 GPU 内存占用情况，并将其与 TensorFlow, Pytorch, MXNet, 以及基于 MXNet 的 Memonger 作对比。其中，对于 MXNet，我们将环境变量 MXNET_MEMORY_OPT 的值设置为 1 来启用其内存优化模式，并

使用 MXNet 的性能更好的符号式编程接口进行测试。而对于 TensorFlow 以及 Pytorch, 则从各自的 modelzoo 中获取相应的网络模型来进行测试。

本节实验针对 ResNet-50, ResNet-101, VGGNet-11, VGGNet-19 四个网络进行, 输入图像的尺寸同样是 $3 \times 256 \times 256$, batchsize 统一设置为 32, 训练次数设置为 15 个 batch。

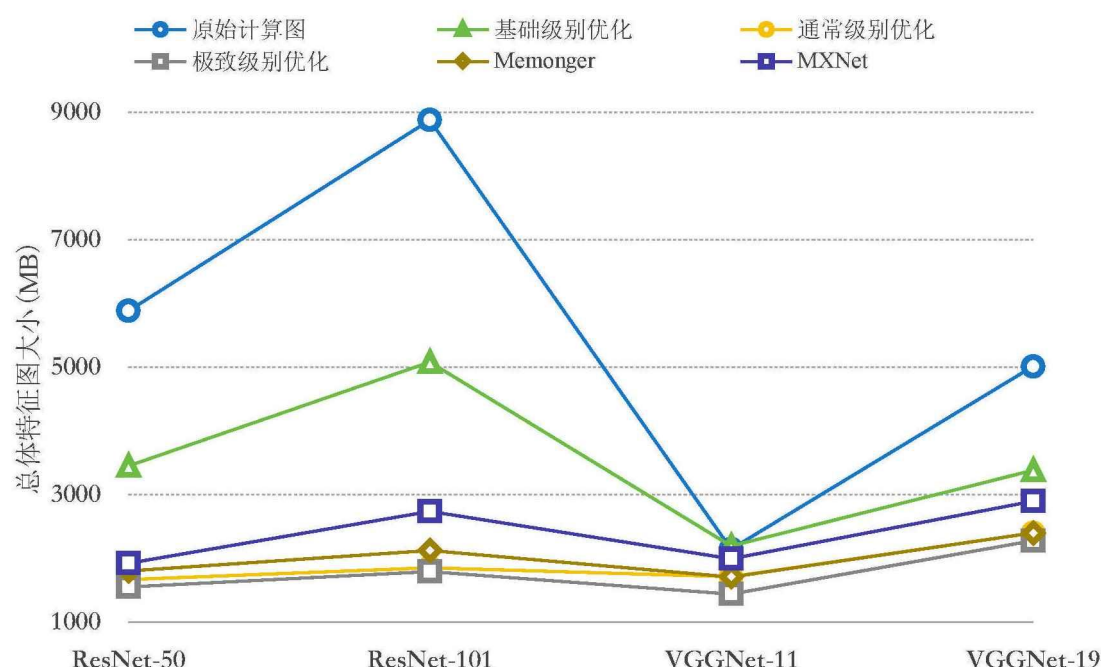


图 4.15 不同模型和不同方法下总体特征图大小值变化图

图 4.15 展示了不同神经网络模型下, 不同方法给出的总体特征图内存占用大小变化情况。总体特征图大小, 是指神经网络除输入和参数以外所有中间输出结果的内存占用值之和。除了原始计算图之外, 其余各个方法均在不同不同程度上进行了内存共享来减低总体特征图的大小。该部分测试主要针对 HelloDeep 框架, 支持静态图分析的 MXNet 以及基于 MXNet 的 Memonger 进行; 而 TensorFlow 和 Pytorch 以动态计算图为主要接口, 不适合加入特征图相关测试。可以看到, HelloDeep 通常级别优化的表现就可以优于 MXNet, 并且在多数情况下也优于 Memonger。而极致级别的优化则所有情况下都取得了最低的特征图大小。

图 4.16 则展示了网络训练峰值内存大小的变化情况。此时, 对 TensorFlow 以及 Pytorch 的测试作为重要的对比基线也被添加到实验过程中。其中, 由于 TensorFlow GPU 内存控制的特性, 本节实验对 TensorFlow 的测试分为了两个不同的模式: Limited 和 Fastest (非 TensorFlow 官方名称)。Limited 表示显式地使

用 `memory_limit` 参数对 TensorFlow 的 GPU 内存申请大小进行限制, 测试结果表明刚好能使得 TensorFlow 能执行 `Batchsize=32` 时的网络训练的 GPU 内存限制值; `Fastest` 则表示将 TensorFlow 的 GPU 内存控制模式设置为按需增长, 使其能发挥最好计算表现。

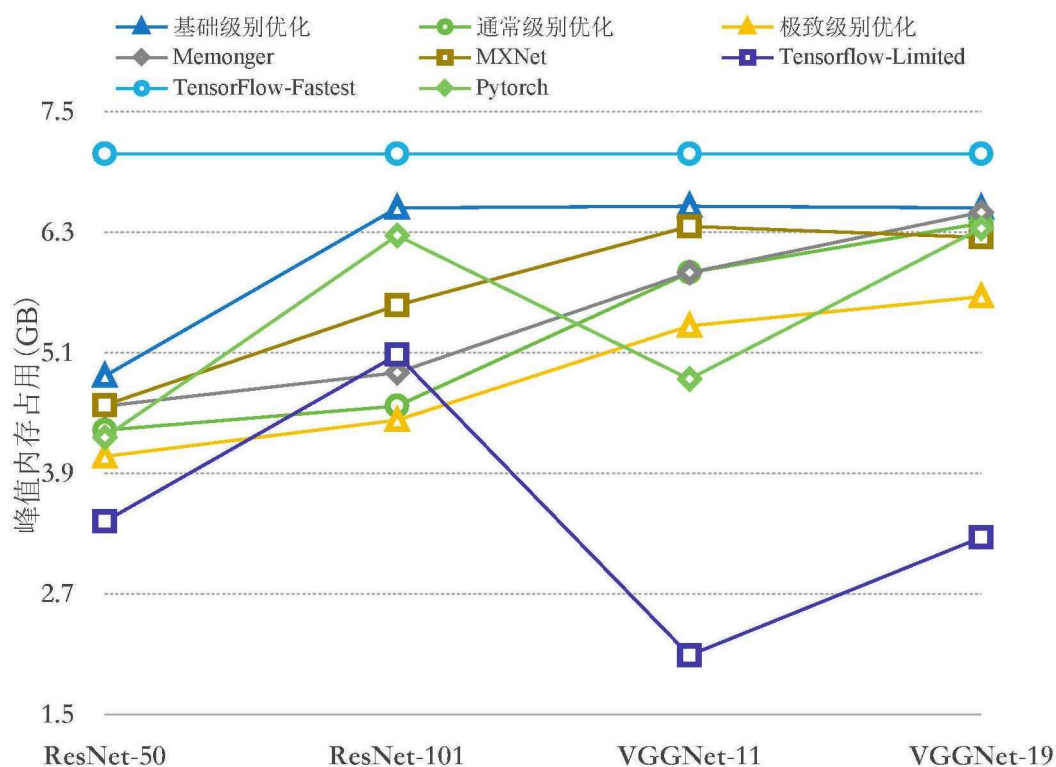


图 4.16 不同模型和不同方法下峰值内存大小值变化图

从结果中可以看出, HelloDeep 的极致级别优化对比 MXNet 和 Memonger, 总是有着更好的表现; 而对比 Pytorch 而言, 在多数情况下也能取得更低峰值内存占用的结果(ResNet-50, ResNet-101, VGGNet-19); 对比 TensorFlow 而言, HelloDeep 的所有模式都比 TensorFlow-Fastest 更节省内存, 而极致级别优化在部分情况下也能比 TensorFlow-Limited 使用更少的峰值内存(ResNet-101)。

HelloDeep 内存优化方法在部分模型上表现不如 TensorFlow-Limited, 主要原因在于: 本研究提出的内存优化方法旨在提出一个通用性的, 不依赖先验知识的自动化 GPU 内存优化方法, 而 TensorFlow 则对不同的网络模型做了专门的、针对性的优化; 同时, TensorFlow 有着庞大的开发人员数量以及广泛而优秀的开源社区贡献, 可以迅速应用 GPU 内存优化领域的最新技术, 因而在极限情况下可以取得比本研究更好的结果。尽管如此, 本研究提出的 GPU 内存优化

方法仍然在 ResNet-101 模型上取得了比实验中所有 baseline 都要好的结果。考虑到 TensorFlow 并没有给用户提供简单的极限 GPU 内存优化接口，本研究提出的三个不同等级优化在易用性上也更胜一筹。

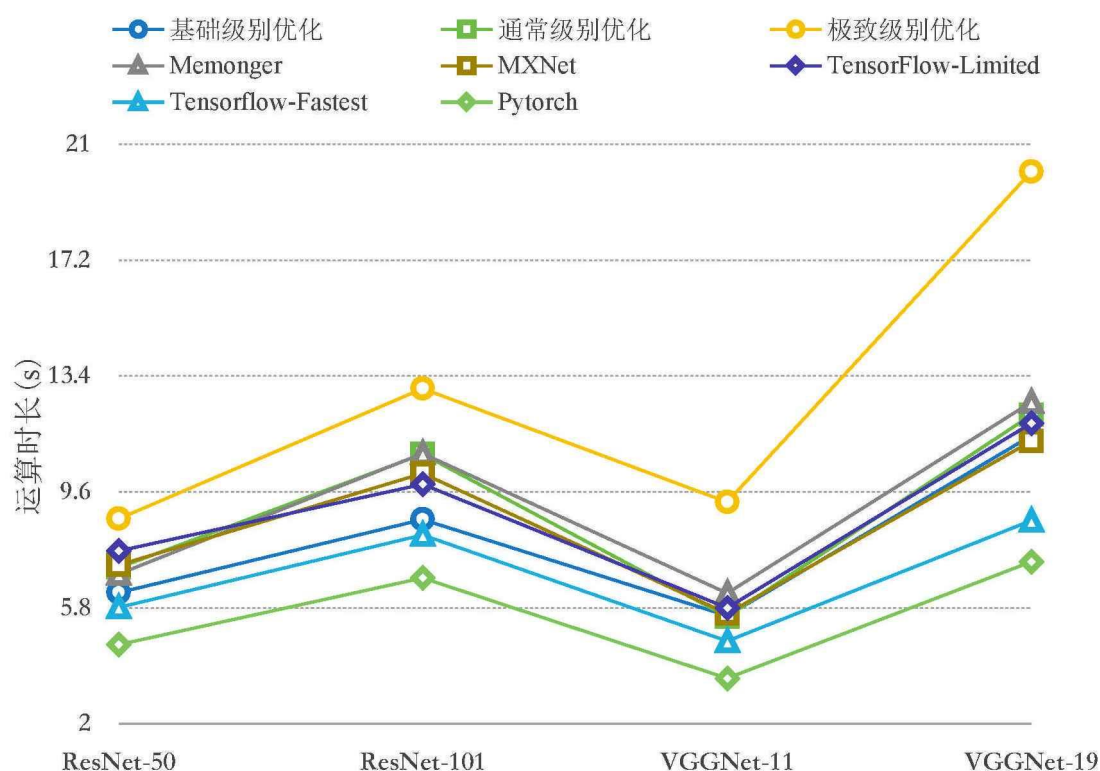


图 4.17 不同模型和不同方法下运算时长变化图

和图 4.16 的结果相对应地，图 4.17 展示了不同模型下运算时长的变化。与基线方法相比，HelloDeep 的运算速度基本能做到与 MXNet 齐平，略逊于 TensorFlow 和 Pytorch。将图 4.16 的结果和图 4.17 的结果综合来看，极致级别的优化有效减少了峰值内存占用，但同时也显著增加了运算时间；通常级别的优化则平衡了内存占用和运算时长；基础级别优化则给出了更快的运算速度。HelloDeep 的内存节省方案在三个不同等级之间做出了明显的区分度，用户可以根据训练网络规模和硬件资源的实际情况来选择合适的优化级别。

表 4-5 基线对比实验结果

总体特征图内存占用大小(MB)				
	ResNet-50	ResNet-101	VGGNet-11	VGGNet-19
原始计算图	5884	8876	2143	5011
基础级别优化	3448	5080	2194	3378
通常级别优化	1668	1856	1708	2400
极致级别优化	1552	1792	1440	2278
Memonger ^[14]	1804	2124	1708	2400
MXNet	1924	2740	1993	2897
峰值内存占用(GB)				
	ResNet-50	ResNet-101	VGGNet-11	VGGNet-19
基础级别优化	4.87	6.54	6.56	6.54
通常级别优化	4.33	4.57	5.90	6.39
极致级别优化	4.07	4.43	5.37	5.66
Memonger ^[14]	4.57	4.90	5.90	6.50
MXNet	4.58	5.57	6.36	6.25
TensorFlow-Limited	3.42	5.08	2.09	3.26
TensorFlow-Fastest	7.08	7.08	7.08	7.08
Pytorch	4.26	6.27	4.84	6.34
运算时长(s)				
	ResNet-50	ResNet-101	VGGNet-11	VGGNet-19
基础级别优化	6.29	8.69	5.54	11.44
通常级别优化	7.13	10.84	5.51	12.12
极致级别优化	8.72	12.99	9.26	20.09
Memonger ^[14]	6.90	10.87	6.28	12.54
MXNet	7.20	10.22	5.61	11.26
TensorFlow-Limited	7.67	9.86	5.80	11.85
TensorFlow-Fastest	5.82	8.20	4.71	8.66
Pytorch	4.60	6.79	3.49	7.31

以上基线对比实验的具体定量结果如表 4-5 所示。从总体特征图内存空间占用大小来看, HelloDeep 内存优化方法最高可以比原始计算图减少 79.8%的特征图大小(ResNet-101); 从运行时峰值内存占用来看, HelloDeep 内存优化方法最高可以比非 HelloDeep 方法中最低峰值内存占用 Memonger 还要再低 9.6%(ResNet-101), 同时在 ResNet-101 之外的模型中, HelloDeep 方法也保持了次低的峰值内存占用。

4.8 本章小结

本章介绍了 HelloDeep 中 GPU 的内存优化方法，总体分为 5 步：试运行、候选点选取、启发式变量重计算和内存交换方案搜索、基于静态计算图的内存申请方案生成、Cudnn 卷积算法辅助空间限制，4.2 节对此进行了详细描述。

在 4.3 节中，我们详细介绍了模型试运行的过程，通过例子论证了试运行的必要性，描述了如何通过试运行数据求取 checkpoint 候选点的方法。试运行使得 HelloDeep 的 checkpoint 选取过程基于运行时数据，而非先验知识，提高了总体方法的泛用性和正确性。

在 4.4 节中，我们分别介绍了重计算策略和内存交换策略，详细描述了其基本思想和具体实现，结合流程图和伪代码展现了其中值得注意的细节。HelloDeep 采用启发式搜索的方式，从 4.3 节中给出的候选点中选取使得 4.5 节的内存申请方案给出的内存总占用量最小的方案。4.4.3 节的实验展示了启发式搜索过程中最终内存占用值随阈值更新的变化，证明了启发式 checkpoint 方案搜索的有效性；同时，展示了应用内存交换策略前后模型计算过程中的 GPU 内存变化，证明了内存交换策略可以进一步降低 GPU 内存峰值。

在 4.5 节中，我们介绍了基于静态计算图的内存申请方案。与动态内存申请相比，基于静态计算图的申请由于预先知道所有变量的大小和生命周期，因此可以实现运行时零开销的内存复用；不仅如此，该方法还通过提前进行的、对不同匹配阈值的多次搜索，给出最低的内存占用方案。4.5.3 节的实验展示了不同 match_range 对最终申请的 MemChunk 数量和总体大小的影响，证明了对不同 match_range 取值进行搜索可以使得最终内存申请量更小。

4.6 节则介绍了在前人工作中经常被忽略，但对总体 GPU 内存占用又具有显著影响的 Cudnn 卷积算法辅助空间限制方法。通过对 Cudnn 辅助空间的自适应限制，我们可以进一步地以一定的计算效率换取明显的内存节约。4.6.2 节的实验展示了 Cudnn 辅助空间限制方法在 ResNet-50 和 VGGNet-11 两个模型中的效果，该方法最高可以降低 13.6% 的内存使用量，证明了 Cudnn 辅助空间限制方法降低运行时 GPU 内存的有效性。

在 4.7 节中，我们以主流深度学习框架 TensorFlow, Pytorch 以及 MXNet 为基线，对 HelloDeep 三个不同级别的优化等级进行了测试，并分别从定性和定量

两个角度给出了结果和分析。测试结果表明，相对于主流框架，HelloDeep 能最大限度地降低静态计算图特征图的大小，并且在多数情况下都能取得运行时 GPU 内存占用峰值的最低或者次低表现。同时，HelloDeep 的基础-通常-极致三个优化级别，也在峰值内存和运算时长方面有着很好的区分度，各自代表了不同的内存优化和计算速度之间的权衡。

本章的工作相对于前人的工作而言，更注重从整个网络训练运行时的角度来看待 GPU 内存，也更强调减少先验知识的影响，通过实际运行时的数据来进行自适应内存优化，以提升整体方法的泛用性和可靠性。

5 总结与展望

5.1 本文主要工作总结

本文主要提出了三个方面的工作。

首先,本文提出了一个新的、强调 GPU 高效利用的深度学习框架 HelloDeep, 该框架由 9 个模块组成, 采用静态计算图的方法来构建神经网络。HelloDeep 内部使用统一的内存分配器 **Storage** 模块来管理内存资源, 使用异步计算引擎 **Parallel** 模块提升计算并行度。

其次, 本文提出了一个高效 IO 模块, 该模块是 HelloDeep 高 GPU 使用率的保障。在 IO 模块中, 我们采用了生产者-消费者模型的设计来保证多线程下数据资源的同步, 采用多线程加载的方式提升批量数据读取速度, 应用了 GPU 数据传输最佳实践方法提升 CPU-GPU 间数据传输的效率, 内部实现则采取了面向对象的装饰器设计模式来提升模块的灵活性和扩展性, 同时提供了便于扩展更多数据集类型的 **Dataset** 类。在实践中, 高效 IO 模块成功将 HelloDeep 框架的运行时 GPU 使用率维持在 90% 以上。

最后, 本文提出了一个用于 HelloDeep 框架的 GPU 内存优化方法。该内存优化方法共分基础-通常-极致三个等级, 包含了基于静态计算图分析的内存共享、基于模型试运行数据的 **checkpoint** 候选点选取、基于变量重计算和内存交换策略的启发式 **checkpoint** 方案搜索, 以及基于特征图大小的 **Cudnn** 辅助空间限制等方法。将这些方法应用至实际神经网络训练当中时, 三个不同等级的优化取得了有效且差异化的效果。将该 GPU 内存优化方法和主流深度学习框架做对比, 其不同之处在于: 1. 通过试运行收集 **Tensor(Layer)** 级别运行数据来决定 **checkpoint** 候选点, 避免对先验知识的依赖; 2. 提供了不同级别的优化方案供用户选择, 使用更方便灵活; 3. 引入了以特征图大小为依据的自适应 **Cudnn** 辅助空间限制, 降低运行时 GPU 内存占用。实验结果表明, 本文提出的 GPU 内存优化方法, 在多数情况下都可以取得最优 (相对于 **MXNet**, **Memonger**, **Pytorch**) 或次优 (相对于 **TensorFlow**) 的 GPU 内存占用结果。

本文提出的工作相对前人工作而言, 创新之处主要有:

- 1) **新的深度学习框架**: 相较于已有框架, 本文提出的深度学习框架HelloDeep更强调GPU的高效利用;
- 2) **Cudnn辅助空间控制**: 前人工作主要关注神经网络特征图大小的优化, 忽视了训练过程中其他因素, 尤其是Cudnn辅助空间对GPU内存占用的影响。本文的工作将同时兼顾这些因素, 得出更好的GPU内存控制方案;
- 3) **多级别优化**: 前人工作没有考虑用户对可选优化级别的需要, 本文的工作提出了基础-通常-极致三个级别的GPU内存优化, 以不同等级的开销来实现不同等级的内存节省效果;
- 4) **实际环境验证**: 相较于部分工作在作者搭建的小型代码demo中进行测试(如SuperNeurons^[26]), 或是基于现有框架进行改造(如Capuchin^[28])的做法, 本文的工作在自主开发的深度学习框架上进行实现, 其性能测试表现更接近于实际生产环境, 说服力更强。

本文的贡献主要有以下几点:

- 1) 实现了一个新的、强调GPU高效利用的深度学习框架HelloDeep;
- 2) 提出了一个异步的深度学习数据加载模块, 避免IO成为神经网络训练过程中的瓶颈以致于降低GPU利用率;
- 3) 提出了一个启发式的GPU变量重计算和交换方案的搜索方法, 包括checkpoint候选点选择, checkpoint选取方案搜索, 以及基于静态计算图的GPU内存池;
- 4) 提出了一个根据特征图大小来进行Cudnn辅助空间限制的方法;
- 5) 提出了一个多级别的自动GPU内存优化方案, 提供了以不同级别的开销来获得不同级别GPU内存节省的功能。

5.2 未来工作展望

本文介绍了有关于深度学习框架中 GPU 高效利用问题的一些工作。受限于本人的学识水平, 当前工作还存在若干改进之处, 其中的细节可以在未来的工作中进行进一步的完善。

对于 HelloDeep 框架, 我们仅实现了基础的静态计算图和基本的数据并行功

能,未来可以进一步完善动态计算图和模型并行,同时结合即时编译等技术提高运行时效率,进一步提升框架的可用性、易用性和高效性。

对于 GPU 高效利用的 IO 模块,未来工作中可以将支持的数据集类型从简单的 jpeg、png 等图像数据集,扩展至二进制数据集、HDF5 数据集等高效数据集类型,将支持的文件系统从本地文件系统扩展至分布式文件系统,使得 HelloDeep 不仅在单机模型下可以实现 GPU 的高效利用,同时,在多机分布式模型下也能完成对 GPU 的高效利用。

对于 GPU 内存优化方法,本工作中仅展示了针对单机单卡模型下的方法介绍和实验分析,未来可以进一步扩展至单机多卡模型、多机多卡模型。对于 GPU 内存优化方法中的具体步骤,未来的工作中可以设计更巧妙的 checkpoint 搜索方法,使得搜索到的局部最优点更接近于全局最优;同时,进一步加强变量重计算和模型交换两个方法之间的融合,用更巧妙的同步方式减少多线程下二者带来的同步开销;而除了 Cudnn 辅助空间之外,未来的工作中还可以对神经网络模型特征图大小和实际运行时 GPU 内存占用之间巨大差距的来源进行研究,更进一步地减少除计算图数据本身之外的 GPU 内存消耗。

参考文献

- [1] GOODFELLOW I, BENGIO Y, COURVILLE A. Deep learning[M]. MIT press, 2016.
- [2] RAINA R, MADHAVAN A, NG A Y. Large-scale deep unsupervised learning using graphics processors[C]. International Conference on Machine Learning. 2009: 873-880.
- [3] NVIDIA . NVIDIA TURING: 显卡技术的颠覆性创新[EB/OL]. [2021-11-29]. <https://www.nvidia.cn/design-visualization/technologies/turing-architecture/>.
- [4] ABADI M, AGARWAL A, BARHAM P, et al. Tensorflow: A system for large-scale machine learning[C]. Symposium on Operating Systems Design and Implementation. 2016: 265-283.
- [5] PASZKE A, GROSS S, MASSA F, et al. Pytorch: An imperative style, high-performance deep learning library[J]. Advanced in Neural Information Processing Systems. 2019, 32: 8026-8037.
- [6] CHEN T, LI M, LI Y, et al. MXNet: A flexible and efficient machine learning library for heterogeneous distributed systems[Z]. ArXiv preprint arXiv:1512.01274, 2015.
- [7] HE K, ZHANG X, REN S, et al. Deep residual learning for image recognition[C]. IEEE Conference on Computer Vision and Pattern Recognition. 2016: 770-778.
- [8] SIMONYAN K, ZISSERMAN A. Very deep convolutional networks for large-scale image recognition[C]. International Conference on Learning Representations. 2015.
- [9] BROWN T B, MANN B, RYDER N, et al. Language models are few-shot learners[Z]. ArXiv preprint arXiv:2005.14165, 2020.
- [10] GAO Y, LIU Y, ZHANG H, et al. Estimating gpu memory consumption of deep learning models[C]. ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2020: 1342-1352.
- [11] JIA Y, SHELHAMER E, DONAHUE J, et al. Caffe: Convolutional architecture for fast feature embedding[C]. ACM International Conference on Multimedia. 2014: 675-678.
- [12] COLLOBERT R, KAVUKCUOGLU K, FARABET C. Torch7: A matlab-like environment for machine learning[C]. Advanced in Neural Information Processing Systems Workshop. 2011.
- [13] DOUG LEA. A Memory Allocator[EB/OL]. (2000-04-04)[2021-11-29]. <http://gee.cs.oswego.edu/dl/html/malloc.html>.
- [14] CHEN T, XU B, ZHANG C, et al. Training deep nets with sublinear memory cost[Z]. ArXiv preprint arXiv:1604.06174, 2016.
- [15] HOLT J L, BAKER T E. Back propagation simulations using limited precision calculations[C]. International Joint Conference on Neural Networks: vol. 2. 1991: 121-126.
- [16] PRESLEY R K, HAGGARD R L. A fixed point implementation of the backpropagation learning algorithm[C]. SoutheastCon. 1994: 136-138.
- [17] COURBARIAUX M, BENGIO Y, DAVID J P. Training deep neural networks with low precision multiplications[Z]. 2015. arXiv: 1412.7024.
- [18] M. COURBARIAUX, Y. BENGIO, J. DAVID, et al. Low precision arithmetic for deep learning[J]. CoRR, abs/1412.7024, vol. 4, 2014.
- [19] SA C D, LESZCZYNSKI M, ZHANG J, et al. High-Accuracy Low-Precision Training[Z]. 2018. arXiv: 1803.03383.
- [20] BENDER M A, FARACH-COLTON M, FEKETE S P, et al. Cost-oblivious storage reallocation[J]. ACM Transactions on Algorithms. 2017, 13(3): 1-20.

- [21] GAREY M R, JOHNSON D S, STOCKMEYER L. Some simplified NP-complete problems[C]. ACM Symposium on Theory of Computing. 1974: 47-63.
- [22] H. A. KIERSTEAD. A polynomial time approximation algorithm for dynamic storage allocation[J]. Discrete Mathematics. 1991, 88(2-3): 231-237.
- [23] ZHANG J, YEUNG S H, SHU Y, et al. Efficient memory management for gpu-based deep learning systems[Z]. arXiv preprint arXiv:1903.06631, 2019.
- [24] REN J, LUO J, WU K, et al. Sentinel: Efficient tensor migration and allocation on heterogeneous memory systems for deep learning[C]. IEEE International Symposium on High-Performance Computer Architecture. 2021: 598-611.
- [25] LATTNER C, ADVE V. Automatic pool allocation: improving performance by controlling data structure layout in the heap[J]. ACM Sigplan Notices, 2005, 40(6): 129-142.
- [26] WANG L, YE J, ZHAO Y, et al. Superneurons: Dynamic GPU memory management for training deep neural networks[C]. ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. 2018: 41-53.
- [27] CUI H, ZHANG H, GANGER G R, et al. Geeps: Scalable deep learning on distributed gpus with a gpu-specialized parameter server[C]. European Conference on Computer Systems. 2016: 1-16.
- [28] PENG X, SHI X, DAI H, et al. Capuchin: Tensor-based gpu memory management for deep learning[C]. International Conference on Architectural Support for Programming Languages and Operating Systems. 2020: 891-905.
- [29] HUANG C C, JIN G, LI J. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping[C]. International Conference on Architectural Support for Programming Languages and Operating Systems. 2020: 1341-1355.
- [30] MA Y, YU D, WU T, et al. PaddlePaddle: An open-source deep learning platform from industrial practice[J]. Frontiers of Data and Computing. 2019, 1(1): 105-115.
- [31] MINDSPORE. MindSpore[EB/OL]. [2021-12-01]. https://gitee.com/mindspore/mindspore/blob/master/README_CN.md.
- [32] SHI-MIN, HU D, LIANG G Y, et al. Jittor: a novel deep learning framework with meta-operators and unified graph execution[J]. Science China(Information Sciences). 2020, v.63(12):118-138.
- [33] ALIBABA-INC. X-DeepLearning: An industrial deep learning framework for high-dimension sparse data[EB/OL]. [2021-12-01]. <https://github.com/alibaba/x-deeplearning>.
- [34] TENCENT-INC. NCNN: A high-performance neural network inference framework optimized for the mobile platform[EB/OL]. [2021-12-01]. <https://github.com/Tencent/ncnn>.
- [35] STEVEN WHITE, et al. Using fused operators to improve performance[EB/OL]. [2021-12-01]. <https://docs.microsoft.com/en-us/windows/ai/directml/dml-fused-activations>.
- [36] PATRIC. MXNet Graph Optimization and Quantization based on subgraph and MKL-DNN [EB/OL]. (2019-03-05)[2021-12-01]. <https://cwiki.apache.org/confluence/display/MXNET/MXNet+Graph+Optimization+and+Quantization+based+on+subgraph+and+MKL-DNN>.
- [37] CHEN T, MOREAU T, JIANG Z, et al. TVM: end-to-end optimization stack for deep learning[Z]. ArXiv preprint arXiv:1802.04799, 2018, 11: 20.
- [38] GOOGLE CLOUD. Cloud Tensor Processing Units (TPUs) [EB/OL]. (2021-11-26) [2021-12-01]. <https://cloud.google.com/tpu/docs/tpus>.
- [39] 上海海思技术有限公司. 昇腾芯片[EB/OL]. [2021-12-01]. <https://www.hisilicon.com/cn>

- /products/Ascend.
- [40] 中科寒武纪科技股份有限公司. 智能加速卡-思元 370 系列[EB/OL]. [2021-12-01]. <https://www.cambricon.com/index.php?m=content&c=index&a=lists&catid=360>.
- [41] THAKUR R, GROPP W, LUSK E. A case for using MPI's derived datatypes to improve I/O performance[C]. ACM/IEEE Conference on Supercomputing. 1998: 1-1.
- [42] THAKUR R, GROPP W, LUSK E. Data sieving and collective I/O in ROMIO[C]. Symposium on the Frontiers of Massively Parallel Computation. 1999: 182-189.
- [43] REW R, DAVIS G. NetCDF: an interface for scientific data access[J]. IEEE Computer Graphics and Applications. 1990, 10(4): 76-82.
- [44] FOLK M, HEBER G, KOZIOL Q, et al. An overview of the HDF5 technology suite and its applications[C]. EDBT/ICDT 2011 Workshop on Array Databases. 2011: 36-47.
- [45] PUMMA S, SI M, FENG W, et al. Towards scalable deep learning via I/O analysis and optimization[C]. International Conference on Data Science and Systems. 2017: 223-230.
- [46] MARK HARRIS. How to Optimize Data Transfers in CUDA C/C++[EB/OL]. (2012-12-04)[2021-12-02]. <https://developer.nvidia.com/blog/how-optimize-data-transfers-cuda-cc/>.
- [47] MARK HARRIS. How to Overlap Data Transfers in CUDA C/C++[EB/OL]. (2012-12-13)[2021-12-02]. <https://developer.nvidia.com/blog/how-access-global-memory-efficiently-cuda-c-kernels/>.
- [48] MARK HARRIS. How to Access Global Memory Efficiently in CUDA C/C++ Kernels[EB/OL]. (2013-01-07)[2021-12-02]. <https://developer.nvidia.com/blog/how-access-global-memory-efficiently-cuda-c-kernels/>.
- [49] MARK HARRIS. Using Shared Memory in CUDA C/C++[EB/OL]. (2013-01-28)[2021-12-02]. <https://developer.nvidia.com/blog/using-shared-memory-cuda-cc/>.
- [50] PCI-SIG. PCI Express Base Specification Revision 5.0[EB/OL]. (2019)[2021-12-11]. <https://pcisig.com/specifications>.
- [51] NVIDIA. NVIDIA NVLINK High-Speed Interconnect[EB/OL]. (2019)[2021-12-11]. <https://www.nvidia.com/en-us/data-center/nvlink/>.
- [52] LARSEN S, LEE B. Survey on system I/O hardware transactions and impact on latency, throughput, and other factors[J]. Advances in Computers, 2014, 92: 67-104.
- [53] NICULESCU V. Efficient Decorator Pattern Variants through C++ Policies.[C]. Evaluation of Novel Approaches to Software Engineering. 2020: 281-288.
- [54] NVIDIA. NVIDIA Nsight Compute Profiling Tool [EB/OL]. (2021-03-26)[2021-12-07]. <https://docs.nvidia.com/nsight-compute/NsightCompute/index.html>.
- [55] YANG C. Hierarchical roofline analysis: How to collect data using performance tools on intel cpus and nvidia gpus[Z]. arXiv preprint arXiv:2009.02449, 2020.
- [56] CHETLUR S, WOOLLEY C, VANDERMERSCH P, et al. Cudnn: Efficient primitives for deep learning[Z]. arXiv preprint arXiv:1410.0759, 2014.
- [57] MATHIEU M, HENAFF M, LECUN Y. Fast training of convolutional networks through ffts[Z]. arXiv preprint arXiv:1312.5851, 2013.
- [58] WINOGRAD S. Arithmetic complexity of computations[M]. Siam, 1980.
- [59] GEORGANAS E, AVANCHA S, BANERJEE K, et al. Anatomy of high-performance deep learning convolutions on simd architectures[C]. International Conference for High Performance Computing, Networking, Storage and Analysis. 2018: 830-841.

攻读硕士期间所取得科研成果

1. 发明专利：一种降低深度神经网络训练过程中 GPU 内存占用的方法. 专利申请号：202210006073.6. 欧岳枫，杜歆。